

Содержание

Реферат	2
Введение	4
1. Анализ архитектуры Mamba и проблемы устойчивости моделей машинного обучения ..	7
1.1 Обзор существующих архитектур моделей последовательностей и их ограничений	7
1.2 Обзор моделей пространства состояний и архитектуры Mamba.....	10
1.3 Анализ уязвимостей моделей машинного обучения и методов состязательных атак	18
1.4 Исследование методов повышения устойчивости моделей и анализ устойчивости Mamba	25
1.5 Выводы.....	32
1.6 Постановка цели и задач курсовой работы	32
2. Моделирование метода обучения устойчивой к состязательным атакам модели на основе Mamba	34
2.1. Постановка задачи повышения устойчивости модели Mamba	34
2.2. Разработка метода повышения устойчивости для Mamba	42
2.3. Выбор метрик оценки устойчивости и качества модели	47
2.4. Выводы.....	51
3. Проектирование и программная реализация фреймворка для обучения устойчивых моделей на основе Mamba	52
3.1. Формулирование требований к фреймворку	52
3.2. Описание архитектуры и функционала системы	54
3.3. Реализация программного модуля для обучения, тестирования и генерации состязательных примеров	56
3.4. Тестирование разработанного программного модуля	63
3.5. Выводы.....	67
4. Экспериментальное исследование метода повышения устойчивости моделей на основе Mamba	69
4.1. Описание используемых наборов данных	69
4.2. Генерация состязательных примеров для обучения модели.....	72
4.3. Схема экспериментального исследования и методология тестирования	77
4.4. Результаты экспериментального исследования	82
4.5. Выводы.....	98
Заключение.....	101
Список литературы	103
Приложение	106

Реферат

Пояснительная записка содержит: 113 страниц, 25 использованных источников, 26 рисунков, 41 формулу и 14 таблиц.

Ключевые слова: Mamba, модели пространства состояний (State Space Models, SSM), состязательные атаки, устойчивость моделей, состязательное обучение, регуляризация скрытых состояний, State-Aware Adversarial Training, HiSPA, робастность нейронных сетей.

Целью данной работы является разработка метода обучения модели на основе Mamba устойчивого к состязательным атакам, учитывающего архитектурные особенности моделей пространства состояний.

В первом разделе проводится анализ предметной области, включающий исследование существующих архитектур моделей последовательностей (RNN, LSTM, Transformer) и их ограничения, а также модели пространства состояний и архитектура Mamba. Рассмотрены уязвимости моделей машинного обучения к состязательным атакам. Выполнено сравнительное исследование методов атак и существующих подходов к повышению устойчивости.

Во втором разделе выполняется моделирование методов обучения устойчивых к состязательным атакам модели на основе Mamba. Формализуется задача повышения устойчивости, разрабатываются адаптированные под архитектурные особенности SSM-моделей методы состязательного обучения. Предложен метод State-Aware Adversarial Training (SA-AT) с регуляризатором дрейфа скрытых состояний, стратегия гибридного состязательного обучения Attack-Mix с двухэтапной генерацией возмущений и SSM-специфичная атака HiSPA. Разработаны механизмы входной защиты. Обосновывается выбор метрик для оценки устойчивости и качества модели.

В третьем разделе рассматриваются вопросы проектирования специализированного фреймворка для обучения устойчивых моделей на основе Mamba. Формулируются требования к разрабатываемому фреймворку, обосновывается выбор программных средств и технологий. Описывается реализация модулей для обучения, тестирования и генерации состязательных примеров, а также приводятся результаты тестирования разработанного программного модуля.

В четвёртом разделе представлено экспериментальное исследование предложенного метода повышения устойчивости моделей на трёх архитектурах (Mamba, Transformer,

CNN), шести наборах данных различной природы и семи типах состязательных атак. Описываются используемые наборы данных и процедура генерации состязательных примеров. Проводится сравнительный анализ поведения моделей в нескольких режимах, основанных на разработанных методах. Выполняется оценка эффективности разработанных методов по выбранным метрикам. По результатам экспериментов формулируются выводы об эффективности предложенных подходов.

Введение

На современном этапе развития интеллектуальных систем всё чаще внедряются методы машинного обучения для обработки последовательных данных. К такому типу данных относятся текстовая информация, аудио- и видеосигналы, а также многие другие виды информации, где порядок элементов особенно важен для понимания контекста. Работа с этими данными востребована для решения задач обработки естественного языка, прогнозирования временных рядов, разработки систем мониторинга и других прикладных областей. Эффективность решения подобных задач во многом определяется типом выбранной модели, поэтому в настоящее время наиболее распространёнными являются нейросетевые архитектуры, способные учитывать связи между элементами последовательности.

В последние годы широкое распространение получили трансформерные архитектуры, демонстрирующие высокую точность в задачах моделирования последовательностей. Однако их главными недостатками являются высокая вычислительная сложность и значительные требования к объёму памяти, что ограничивает их применение в ресурсно-ограниченных средах. Современной альтернативой выступают модели пространства состояний (State Space Models, SSM), и в частности архитектура Mamba, которая сочетает эффективность обработки длинных контекстов с линейной сложностью вычислений.

Несмотря на растущую популярность новой архитектуры, вопросы её устойчивости к состязательным воздействиям остаются недостаточно изученными. Состязательные атаки, заключающиеся в целенаправленном внесении малозаметных возмущений во входные данные, способны существенно снижать точность работы нейросетевых моделей, что создаёт серьёзные риски при внедрении таких решений в критически важные системы. Существующие методы повышения устойчивости, разработанные для свёрточных нейронных сетей и трансформеров, не учитывают архитектурные особенности SSM-моделей и не показывают удовлетворительных результатов при применении к Mamba.

Таким образом, актуальность данной работы обусловлена необходимостью исследования уязвимостей архитектуры Mamba к состязательным атакам и разработки эффективных методов повышения её устойчивости, поскольку обеспечение робастности моделей является важным условием для их безопасного применения в реальных сценариях.

Теоретическая значимость заключается в адаптации концепций состязательного обучения к архитектуре Mamba и в выявлении закономерностей влияния состязательных

возмущений на качество работы подобных моделей. Результаты исследования помогут понять механизмы распространения ошибок в рекуррентных SSM-моделях и определить факторы, определяющие их чувствительность к входным возмущениям.

Практическая значимость определяется созданием программного фреймворка, позволяющего пользователям обучать устойчивые версии Mamba-моделей, проводить их тестирование и оценивать поведение под воздействием различных типов атак.

В рамках данной работы будет проведён комплексный анализ архитектуры Mamba и существующих методов состязательных атак, на основе которого будет разработан метод повышения устойчивости, учитывающий архитектурные особенности SSM-моделей. Предлагаемый подход будет направлен на преодоление нижней границы ошибки, характерной для детерминированных SSM, и предотвращение эффекта «взрыва» ошибки, возникающего при адаптивной параметризации. Для реализации метода будет разработан фреймворк, интегрируемая с существующими фреймворками глубокого обучения. Экспериментальное исследование позволит оценить эффективность предложенного подхода, а также выявить специфические уязвимости Mamba, обусловленные её рекуррентной природой и механизмом распространения состояния.

В первом разделе проводится анализ проблемной области: рассматриваются архитектуры моделей последовательностей и их ограничения, затем исследуется архитектура SSM и Mamba. Исследуются известные уязвимости моделей машинного обучения и методы состязательных атак, а также существующие способы защиты от них. На основе проведённого анализа формулируются цели и задачи курсовой работы.

Во втором разделе выполняется моделирование метода обучения устойчивой к состязательным атакам модели на основе Mamba. Формулируется постановка задачи повышения устойчивости, разрабатывается адаптированный метод состязательного обучения, а также обосновывается выбор метрик для оценки устойчивости и качества модели.

В третьем разделе рассматриваются вопросы проектирования и программной реализации фреймворка для обучения устойчивых моделей. Формулируются требования к разрабатываемому фреймворку, обосновывается выбор программных средств и технологий, описывается реализация модулей для обучения, тестирования и генерации состязательных примеров, а также приводятся результаты тестирования разработанного программного модуля.

В четвёртом разделе представлено экспериментальное исследование предложенного метода повышения устойчивости моделей на основе Mamba. Описываются используемые наборы данных, процедура генерации состязательных примеров, исследуется поведение базовой архитектуры Mamba под воздействием атак, а также проводится количественная и качественная оценка эффективности разработанного метода. По результатам экспериментов формулируются выводы об эффективности предложенного подхода.

1. Анализ архитектуры Mamba и проблемы устойчивости моделей машинного обучения

1.1 Обзор существующих архитектур моделей последовательностей и их ограничений

На протяжении многих лет доминирующими архитектурами для решения подобных задач были рекуррентные нейронные сети (RNN) и их модифицированные версии — сети с долгой краткосрочной памятью (LSTM) и управляемые рекуррентные блоки (GRU) [1]. Однако с появлением архитектуры Transformer в 2017 году фокус сместился на быстро набирающую популярность идею применения self-attention для обработки длинных последовательностей [2].

1.1.1 Рекуррентные нейронные сети (RNN) и их варианты (LSTM/GRU)

Одними из классических моделей, созданных специально для работы с последовательностями, являются рекуррентные нейронные сети. Их главным отличием от сетей прямого распространения выступает наличие внутренней памяти (состояния), которая позволяет учитывать информацию о предыдущих элементах последовательности при обработке текущего [2]. На каждом временном шаге сеть применяет одну и ту же операцию с общими параметрами. При этом за счёт передачи скрытых состояний результат зависит как от текущего входа, так и от более ранних. Теоретически RNN способны моделировать зависимости на произвольной длине, так как каждый шаг обработки зависит от всех предыдущих [2]. Это свойство сделало их удобным выбором для задач обработки естественного языка (NLP), распознавания речи, анализа временных рядов и других приложений, где важен контекст [2].

Однако на практике простые RNN столкнулись с проблемой, известной как затухание градиента (vanishing gradient). В процессе обучения методом обратного распространения ошибки во времени (Backpropagation Through Time, BPTT) градиенты, передаваемые через множество временных шагов, экспоненциально уменьшаются. Это существенно затрудняет обучение сети на длинных последовательностях, так как она перестаёт «помнить» информацию из начала последовательности [1].

Чтобы преодолеть эти ограничения, были предложены более продвинутые архитектуры, такие как LSTM (Long Short-Term Memory) и её компактная альтернативная версия GRU (Gated Recurrent Unit) [2]. LSTM имеет специальные механизмы управления памятью — «вентили» (gate, гейт), которые регулируют поток информации. Структура модели включает в себя три типа вентилях: входной (input gate), забывающий (forget gate)

и выходной (output gate) [2]. Эти вентили позволяют сети решать, какую информацию из предыдущего состояния нужно сохранить, какую обновить, а какую — удалить соответственно. Благодаря этому LSTM стали намного эффективнее справляться с проблемой затухания градиента и успешно применяются для анализа длинных контекстов, например, для классификации музыкальных жанров на основе аудиоданных [2] и прогнозирования временных рядов [1].

Несмотря на значительный прогресс, LSTM и GRU всё ещё обладают рядом фундаментальных недостатков. Главный из них — рекуррентная составляющая. Обработка данных происходит строго последовательно, где на каждом новом шаге необходима информация с предыдущего. Это создаёт узкое место с точки зрения скорости обучения и инференса (затрудняет эффективную параллелизацию и усложняет моделирование очень дальних зависимостей). Автор работы [2] отмечает, что хотя LSTM и могут удерживать информацию сотни шагов, на практике они не позволяют эффективно связывать «старые» токены по мере поступления новых данных. Эти ограничения стимулировали поиск принципиально новых архитектурных решений.

1.1.2 Архитектура Transformer и её фундаментальные преимущества

В 2017 году группа исследователей из Google представила архитектуру Transformer в статье «Attention Is All You Need» [1]. Это стало поворотным событием, поскольку создатели Transformer отказались от рекуррентной обработки в пользу механизма самовнимания (self-attention). Вместо последовательного прохода по элементам, Transformer обрабатывает всю последовательность параллельно, вычисляя веса взаимодействия между всеми парами элементов. Так решаются две ключевые проблемы RNN/LSTM: последовательная обработка без возможности распараллеливания вычислений и низкое качество моделирования дальних связей [1].

Механизм самовнимания (self-attention) в своей основе использует модель «Запрос-Ключ-Значение» (Query-Key-Value, QKV). Для каждого элемента последовательности вычисляются три вектора. Вес внимания между двумя элементами определяется как степень соответствия запроса одного элемента ключу другого. Итоговое представление элемента вычисляется как взвешенная сумма значений всех элементов последовательности, где веса определяются функцией внимания [1]. Это позволяет каждому элементу напрямую взаимодействовать с любым другим элементом последовательности, обеспечивая теоретически бесконечную память при условии достаточных вычислительных ресурсов [2]. Важным улучшением является использование многоголового внимания (multi-head

attention), которое позволяет модели одновременно фокусироваться на различных аспектах входных данных, например, на ритме и высоте тона в аудиосигнале [2].

Для сохранения информации о порядке следования элементов в Transformer используется *позиционное кодирование* (positional encoding). В первой версии это были фиксированные синусоидальные функции [1], однако в последующих работах стали применять и обучаемые позиционные эмбединги. Параллелизация вычислений в Transformer позволяет обучать значительно более глубокие и объемные модели на огромных наборах данных [1].

1.1.3 Основные проблемы и ограничения Transformer при работе с временными рядами

Несмотря на триумф в NLP и CV (computer vision), прямое применение стандартного Transformer к задачам анализа временных рядов сталкивается с рядом проблем, которые могут сделать его не самым оптимальным выбором. В работах [1; 2], отражены следующие специфические проблемы трансформерной архитектуры:

Вычислительная сложность. Самовнимание в его каноническом виде имеет квадратичную временную и пространственную сложность относительно длины входной последовательности, $O(N^2)$, где N — количество временных шагов (или токенов) [1]. Обработка длинных последовательностей становится слишком затратной с точки зрения памяти и времени вычислений, что делает обучение на длинных данных невозможным в условиях ограниченных ресурсов.

Неэффективное использование длинного входа. Исследования показывают, что даже модифицированные для снижения сложности трансформеры на практике не могут эффективно использовать длинные входные последовательности. Исследователи из [1] сделали вывод, что при увеличении длины входного окна качество прогнозирования большинства моделей (Transformer, Informer, Reformer) значительно ухудшается.

Проблемы с масштабируемостью. В NLP и CV увеличение глубины Transformer (количества слоев) является стандартным способом повышения качества модели. Однако для временных рядов эта закономерность не работает. В эксперименте, приведенном в обзоре [1], увеличение числа слоев с 3 до 6 давало приемлемые результаты, но дальнейшее увеличение глубины (до 12 и более слоев) не приводило к росту качества, а напротив даже

ухудшало его. Это свидетельствует о том, что для временных рядов требуются иные подходы к масштабированию, отличные от таковых в NLP.

Неучет специфической структуры временных рядов. Стандартный Transformer не содержит встроенных предположений (индуктивных смещений) о структуре данных. Временные ряды, в отличие от текста, обладают ярко выраженными свойствами, такими как *тренд* (направленное долговременное изменение) и *сезонность* (периодические колебания) [1]. Как показывают эксперименты, простой учет сезонно-трендовой декомпозиции позволяет улучшить качество прогнозирования на 50–80% [1], ярко демонстрируя полезность интеграции дополнительных специализированных знаний о предметной области в архитектуру.

Подводя итог, переход от RNN/LSTM к Transformer в задачах анализа последовательностей был обусловлен необходимостью преодоления ограничений рекуррентных моделей, связанных с последовательной обработкой и памятью. Однако архитектура Transformer так же требует адаптации для эффективного применения к временным рядам. Основные усилия исследователей в этой области направлены на снижение вычислительной сложности, улучшение работы с длинными последовательностями и интеграцию индуктивных смещений [2], но трансформеры всё ещё не полностью удовлетворяют требованиям эффективности и устойчивости. Указанные ограничения стимулировали поиск альтернативных архитектур, способных сочетать высокую точность моделирования последовательностей с эффективностью вычислений.

1.2 Обзор моделей пространства состояний и архитектуры Mamba

1.2.1 State Space Models: определение и формулировка

В поисках альтернативы, исследователи обратились к классической теории управления — моделям состояния (State Space Models, SSM) [3; 4]. SSM отображают последовательность в виде динамической системы, в которой текущее состояние зависит от предыдущего состояния и входных данных. Такой подход позволяет эффективно учитывать временные зависимости, обладая линейной сложностью.

Модели состояния представляют собой математический аппарат, связывающий входной сигнал $x(t)$, скрытое состояние $h(t)$ и выходной сигнал $y(t)$ с помощью системы дифференциальных уравнений. В непрерывной форме классическая SSM задаётся следующими уравнениями (1.1) [4; 5]:

$$\begin{cases} \dot{h}(t) = \mathbf{A}h(t) + \mathbf{B}x(t) \\ y(t) = \mathbf{C}h(t) + \mathbf{D}x(t) \end{cases} \quad (1.1)$$

где $\mathbf{A} \in R^{N \times N}$ — матрица переходов состояния, $\mathbf{B} \in R^{N \times 1}$ — матрица входа, $\mathbf{C} \in R^{1 \times N}$ — матрица выхода, а $\mathbf{D}$ - параметр skip connection, часто принимается нулевым [5]. Параметр N (размерность скрытого состояния) определяет объем памяти модели.

Для применения в глубоком обучении непрерывная SSM дискретизируется, при помощи преобразования с удержанием нулевого порядка (Zero-Order Hold, ZOH). Второе уравнение дискретизируется напрямую: $y_k = y(k\Delta) \rightarrow y_k = Ch_k + Dx_k$.

Рассмотрим подробнее для первого:

Умножим исходное уравнение на e^{-At} :

$$e^{-At} \dot{h}(t) - e^{-At} \mathbf{A}h(t) = e^{-At} \mathbf{B}x(t)$$

$$\frac{d}{dt} (e^{-At} h(t)) = e^{-At} \mathbf{B}x(t)$$

Тогда общее решение для непрерывной модели имеет вид (1.2):

$$h(t) = e^{At} h(0) + \int_0^t e^{A(t-\tau)} \mathbf{B}x(\tau) d\tau \quad (1.2)$$

Шаг дискретизации $\Delta \Rightarrow x_k = x(k\Delta), h_k = h(k\Delta)$

$$h_k = e^{A k\Delta} h(0) + \int_0^{k\Delta} e^{A(k\Delta-\tau)} \mathbf{B}x(\tau) d\tau$$

$$h_{k+1} = e^{A\Delta} \left[e^{A k\Delta} h(0) + \int_0^{k\Delta} e^{A(k\Delta-\tau)} \mathbf{B}x(\tau) d\tau \right] + \int_{k\Delta}^{(k+1)\Delta} e^{A((k+1)\Delta-\tau)} \mathbf{B}x(\tau) d\tau =$$

Подставляем выражение для h_k , вводим $v = (k+1)\Delta - \tau$ и учитываем, что $x = \text{const}$ внутри интервала Δ , следовательно $x(\tau) = x_k$ на всём интервале:

$$= e^{A\Delta} h_k + \left[\int_0^{\Delta} e^{Av} dv \right] \mathbf{B}x_k = e^{A\Delta} h_k + \frac{1}{A} (e^{A\Delta} - I) \mathbf{B}x_k$$

Это позволяет перейти к дискретной рекуррентной форме (1.3), которая может быть вычислена на компьютере [5]:

$$\begin{cases} \mathbf{h}_k = \bar{\mathbf{A}}\mathbf{h}_{k-1} + \bar{\mathbf{B}}\mathbf{x}_k \\ \mathbf{y}_k = \bar{\mathbf{C}}\mathbf{h}_k + \bar{\mathbf{D}}\mathbf{x}_k \end{cases} \quad (1.3)$$

где $\mathbf{A} = \exp(\Delta\mathbf{A})$, $\mathbf{B} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot \Delta\mathbf{B} \approx \Delta\mathbf{B}$, а Δ — шаг дискретизации. Важно отметить, что в этой формулировке параметры модели являются линейно-инвариантными во времени (Linear Time-Invariant, LTI) — они не зависят от входных данных и остаются постоянными для всех временных шагов [5].

Удобным свойством дискретных SSM является возможность их реализации как в рекуррентной, так и в свёрточной форме. Благодаря линейности системы, выходная последовательность может быть вычислена как свёртка входа с ядром $\bar{\mathbf{K}} = (\bar{\mathbf{C}}\bar{\mathbf{B}}, \bar{\mathbf{C}}\bar{\mathbf{A}}\bar{\mathbf{B}}, \dots)$. Таким способом можно эффективно распараллеливать обучение на GPU [4; 5]. При этом во время инференса модель может переключаться в рекуррентный режим, обрабатывая последовательность токенов за токеном [4; 5].

1.2.2 Ограничения линейно-инвариантных SSM и переход к селективности

Несмотря на эффективность LTI SSM, они имеют существенный недостаток при моделировании дискретных и информационно-насыщенных данных, таких как текст: отсутствует *селективность* (selectivity) — способность избирательно запоминать или забывать информацию в зависимости от контекста [5].

В стандартной задаче Copying, где интервалы между элементами постоянны, LTI SSM легко справляются, используя свёртки фиксированной длины. Однако ограничение ярко проявляется в Selective Copying, где интервалы между элементами случайны, и модель должна игнорировать шумовые токены, запоминая только релевантные. LTI SSM не могут решить эту задачу, поскольку их динамика не зависит от содержимого входа [5].

Другой важной задачей, демонстрирующей недостаток LTI SSM, является Induction Heads — механизм, лежащий в основе способности LLM к обучению в контексте (in-context learning). Для успешного выполнения этой задачи модель должна уметь ассоциативно запоминать и воспроизводить информацию на основе контекста, что требует селективного управления состоянием [5].

1.2.3 Mamba: архитектура и ключевые инновации

Mamba, предложенная Альбертом Гу и Три Дао в 2023 году, стала первой моделью на основе SSM, достигшей качества, сопоставимого с Transformer, на задачах обработки естественного языка, сохранив при этом линейную вычислительную сложность [3;6]. Основная инновация Mamba заключается в введении *механизма селективности* (Selective State Space Model, S6), который представлен на рисунке 1.1.

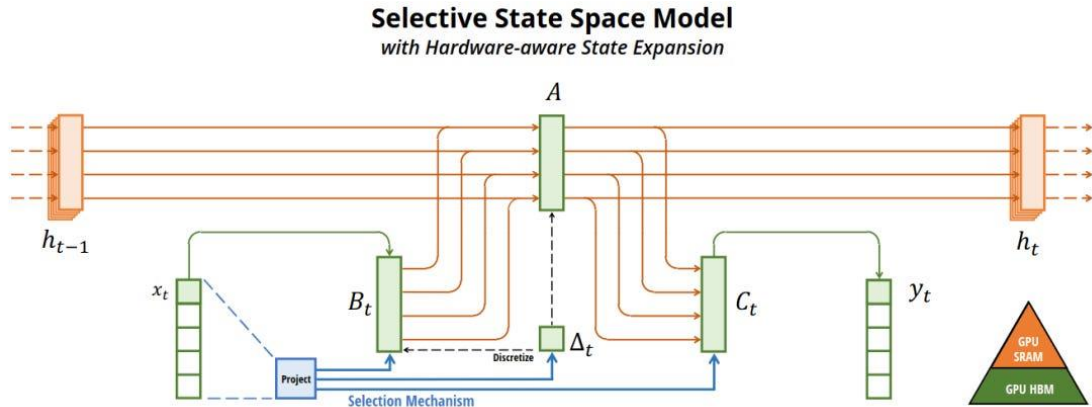


Рисунок 1.1 — Схема селективной модели пространства состояний (Selective State Space Model) в архитектуре Mamba.

Механизм селективности. В отличие от классических SSM, где параметры B , C и шаг дискретизации Δ являются фиксированными, в Mamba эти величины вычисляются как функции от текущего входа x_t [3; 6], как показано в (1.4):

$$\begin{aligned} B_t &= s_B(x_t) \\ C_t &= s_C(x_t) \\ \Delta_t &= \tau_\Delta(\text{bias} + s_\Delta(x_t)) \end{aligned} \quad (1.4)$$

где $s_B(x)$, $s_C(x)$ и $s_\Delta(x)$ — линейные проекции в пространство размерности d , а τ_Δ — функция `softplus`. Такая параметризация позволяет модели динамически регулировать поток информации: большие значения Δ «сбрасывают» состояние, фокусируясь на текущем входе, тогда как малые значения сохраняют предыдущее состояние, игнорируя новый вход [3; 6].

Важно отметить связь механизма селективности с классическими RNN-гейтами. Теоретически показано, что при $N = 1$, $A = -1$, $B = 1$ селективная SSM сводится к гейтированной рекуррентной формуле (1.5) [6]:

$$g_t = \sigma(\text{Linear}(x))$$

$$h_t = (1 - g_t)h_{t-1} + g_tx_t \quad (1.5)$$

Аппаратно-ориентированный алгоритм. Введение селективности создаёт вычислительную проблему: параметры становятся зависимыми от входа, и использование эффективной свёрточной формулировки становится невозможным. Для решения этой проблемы Mamba использует аппаратно-ориентированный алгоритм, основанный на параллельном ассоциативном сканировании (parallel associative scan) и технике перевычисления (recomputation) [3; 6].

Главная идея заключается в выполнении дискретизации и рекуррентного обновления состояния непосредственно в быстрой памяти SRAM, минимизируя обмен данными с медленной HBM. Это позволило достигнуть 20–40-кратного ускорения по сравнению со стандартной реализацией сканирования [6].

Архитектура блока Mamba. Блок Mamba использует дизайн, объединяющий идеи предшествующих SSM-архитектур (таких как H3) с современными практиками построения Transformer-блоков [3; 6]. Основной блок включает следующие компоненты [3]:

1. **Линейная проекция и расширение.** Вход проецируется в пространство с увеличенной размерностью (коэффициент расширения $E=2$), генерируя два потока: один для главной ветви, другой для гейтинга. Это позволяет увеличить ёмкость модели без значительного роста числа параметров.
2. **Свёрточный слой.** Одномерная свёртка (ядро размера 4) захватывает локальные паттерны и улучшает стабильность обучения.
3. **Селективная SSM (S6).** Ядро блока. Выполняет дискретизацию и рекуррентное обновление состояния с использованием аппаратно-ориентированного сканирования.
4. **Гейтирование и выходная проекция.** Результат умножается на гейт, активируется функцией SiLU/Swish и проецируется обратно в исходную размерность.

Важной особенностью архитектуры является отсутствие отдельного MLP-блока — его роль объединена с SSM. Это не только упрощает структуру, но и, как показывают эксперименты, улучшает эффективность использования параметров по сравнению с чередованием SSM- и MLP-блоков [6].

Mamba-2: State Space Duality. Развитием Mamba стала Mamba-2, представленная в 2024 году. Её отличием является концепция *структурной дуальности пространства состояний* (Structured State Space Duality, SSD), которая устанавливает теоретическую связь между селективными SSM-моделями и различными формами механизмов внимания [3; 4].

В рамках SSD как внимание в Transformer, так и линейная SSM могут быть представлены как преобразования с полуразделимыми матрицами, что позволило перенести на SSM многие оптимизации, разработанные для Transformer.

Mamba-2 вводит новую структуру матрицы A , ограничивая её скалярным множителем единичной матрицы (вместо диагональной в Mamba-1), и использует алгоритм, основанный на разбиении последовательности на блоки. Внутри каждого блока вычисления выполняются параллельно с использованием матричного умножения, а взаимодействие между блоками — через рекуррентные связи. Это дало заметное ускорение в 2–8 раз по сравнению с оригинальной Mamba [3].

1.2.4 Преимущества Mamba по сравнению с другими подходами

Mamba имеет ряд преимуществ по сравнению с традиционными архитектурами, что подтверждается теоретическим анализом и эмпирическими исследованиями.

Линейная вычислительная сложность. В отличие от Transformer с квадратичной сложностью $O(L^2)$, Mamba сохраняет линейную сложность $O(L)$ как при обучении, так и при инференсе [3; 6]. На последовательностях длиной 32K токенов эффективный SSM-скан Mamba работает до 7 раз быстрее, чем оптимизированная реализация FlashAttention-2 [6].

Высокая скорость инференса. Благодаря рекуррентной природе, Mamba не требует хранения KV-кэша, как Transformer, что позволяет использовать значительно большие размеры батча при генерации. Mamba имеет в 4–5 раз более высокую пропускную способность (throughput) по сравнению с Transformer аналогичного размера [6]. На рисунке 1.2 показаны результаты сравнения скорости scan-операции и пропускной способности инференса Mamba с альтернативными подходами. Например, при генерации текста Mamba-1.4B достигает пропускной способности выше, чем Transformer-1.3B, а Mamba-6.9B превосходит Transformer-1.3B в 5 раз [6].

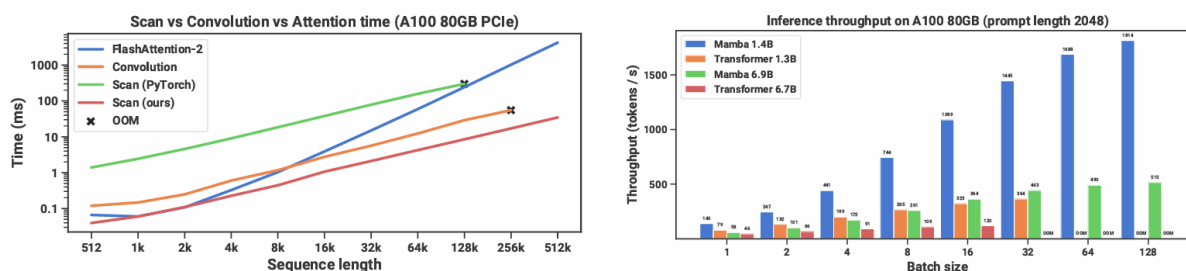


Рисунок 1.2 — Показатели эффективности Mamba: скорость scan-операции при обучении и пропускная способность при инференсе

Эффективное использование памяти. Аппаратно-ориентированная реализация Mamba позволяет существенно сократить потребление памяти. При обучении модели на 125M параметров с размером батча 32 Mamba использует 38.2 ГБ памяти, в то время как Transformer с FlashAttention-2 — 34.5 ГБ, что сопоставимо, но при значительно большей скорости обработки. Однако при увеличении длины последовательности разрыв в потреблении памяти становится более заметным в пользу Mamba [6].

В контексте аудио-суперразолюции замена Attention и LSTM на Mamba в архитектуре AERO позволила сократить потребление GPU-памяти в 2–4 раза при обучении и в 5 раз при инференсе, а также получить 14-кратное ускорение работы [7].

Способность моделировать сверхдлинные последовательности. Благодаря механизму селективности, Mamba эффективно обрабатывает последовательности длиной до 1M токенов. В экспериментах с ДНК-моделированием Mamba демонстрирует улучшение качества с увеличением длины контекста до 1M токенов, тогда как модели на основе свёрток (НуенаDNA) ухудшают качество на длинных последовательностях [6].

Анализ производительности Mamba-2 на последовательностях различной длины показывает, что компонент SSM является доминирующим с точки зрения вычислительных затрат (до 60–70% от общего времени выполнения) и потребления памяти. При этом Mamba-2 демонстрирует на 28.6% меньше FLOPs по сравнению с Mamba-1 при длине последовательности 2048 токенов благодаря блочной обработке состояний [8].

Преодоление проблем рекуррентных сетей. В отличие от RNN и LSTM, Mamba не страдает от проблемы затухания градиента благодаря структурированной формулировке и инициализации HiPPO. На задаче Induction Heads Mamba способна обобщать знания на последовательности в 4000 раз длиннее, чем использовались при обучении, в то время как LSTM и другие RNN-подобные модели не справляются с этой задачей [6].

Сравнение с Transformer в языковом моделировании. В языковом моделировании на датасете Pile Mamba-1.4B превосходит Pythia-1.4B (на 4.5 пункта по средней точности на нулевых задачах) и достигает качества Transformer-2.8B. При сопоставимом количестве параметров Mamba демонстрирует более низкую перплексию и лучшие результаты на задачах рассуждения (HellaSwag, PIQA, ARC) [6].

Преимущества в задачах обработки аудио. В задаче аудио-суперразолюции модель AEROMamba (на основе Mamba) превзошла AERO (на основе Attention и LSTM) по субъективным оценкам слушателей: 66.47 против 60.03 на шкале 0-100 для датасета MUSDB18, и 84.41 против 76.89 для PianoEval (использовался AEROMamba-HQ). При этом AEROMamba продемонстрировала более высокие объективные метрики (ViSQOL: 2.93 против 2.90). Слушатели отмечали, что AEROMamba создаёт более яркий и естественный звук с лучшим восстановлением высоких частот, что подтверждает способность модели к более точному восстановлению спектральных компонентов [7].

1.2.5 Развитие механизма селективности: SeRpEnt

Следующим этапом развития идеи селективности стала архитектура SeRpEnt (Selective Resampling for Expressive State Space Models), которая предлагает дополнительный механизм сжатия последовательностей на основе информационной значимости элементов [9]. Основная идея заключается в использовании значений Δ_t (шагов дискретизации) как индикаторов информационной ценности каждого элемента последовательности. Элементы с большими значениями Δ_t считаются информационно-значимыми и сохраняются, тогда как элементы с малыми Δ_t агрегируются с соседними [9].

В архитектуре SeRpEnt каждый блок сжимает входную последовательность с несколькими коэффициентами сжатия, обрабатывает сжатые версии с помощью SSM, а затем декомпрессирует и объединяет результаты. Это позволяет модели одновременно работать с информацией на разных уровнях детализации, аналогично пирамидальным структурам в компьютерном зрении, но с адаптивным выбором важных элементов [9].

Этот подход позволяет сжимать входные последовательности до заданной длины, сохраняя наиболее важную информацию. В экспериментах на бенчмарке Long Range Arena SeRpEnt улучшил производительность базовых SSM-моделей (S4 и S5) на задачах ListOps (на 0.65% для S5), Text Retrieval (на 1.78% для S4 и 2,99% для S5) и в среднем по всем задачам, за исключением задач, связанных с обработкой изображений, где структурное смещение данных оказалось иным [9].

1.3 Анализ уязвимостей моделей машинного обучения и методов состязательных атак

1.3.1 Введение: новая проблема на фоне прогресса

Глубокое обучение совершило революцию в области искусственного интеллекта, позволив создавать модели, демонстрирующие выдающиеся результаты в распознавании изображений, обработке естественного языка, диагностике заболеваний и многих других приложениях [10; 11]. Эти достижения стали возможны благодаря способности глубоких нейронных сетей извлекать сложные закономерности из больших объемов данных. В результате такие модели получили широкое распространение в критически важных системах, включая автономное вождение, биометрическую аутентификацию и финансовый мониторинг [12].

Однако наряду с впечатляющими успехами была выявлена фундаментальная проблема: модели машинного обучения остаются уязвимыми к специально сконструированным входным данным, получившим название состязательных примеров (adversarial examples) [12; 10].

Работа К. Сегеди и соавторов (2014) впервые продемонстрировала, что добавление едва заметных, целенаправленно рассчитанных возмущений к изображению может привести к полной ошибке классификации глубокой нейронной сетью, хотя для человека такие изменения остаются практически незаметными [10; 14]. Это открытие не только поставило под сомнение надежность моделей глубокого обучения, но и дало импульс развитию новой области исследований — состязательного машинного обучения (adversarial machine learning), в рамках которого изучаются как уязвимости моделей, так и методы их защиты.

1.3.2 Уязвимости моделей машинного обучения

Состязательные примеры как проявление уязвимости. Состязательный пример (adversarial example) определяется как модифицированная версия исходного образца x , к которому добавлено небольшое возмущение δ , такое, что результирующий образец $x^{adv} = x + \delta$ приводит к ошибочному предсказанию модели f , в то время как для человека этот образец практически неотличим от исходного [10; 15]. Формально, для классификационной задачи это можно записать следующим образом, в виде неравенства (1.6):

$$f(x^{adv}) \neq f(x), \text{ при условии } \|\delta\| \ll \|x\| \quad (1.6)$$

Важно отметить, что состязательные примеры не являются случайным шумом, а представляют собой специально сконструированные образцы, которые заставляют модель ошибаться с высокой степенью уверенности [10]. Особенно опасным такие атаки делает то, что модель нередко демонстрирует «ложную уверенность» (false confidence) в своих ошибочных предсказаниях. Состязательные примеры встречаются не только в задачах классификации изображений, но и в обработке текста, аудио, временных рядов и других областях, что подчеркивает фундаментальный характер данной уязвимости [12; 14].

Причины уязвимости моделей. Для объяснения феномена состязательной уязвимости, исследователями был предложен ряд взаимодополняющих гипотез.

Высокая размерность данных рассматривается одним из главных факторов. Как отмечается в [10], в пространствах высокой размерности даже малые возмущения по каждой координате могут суммироваться и приводить к существенному смещению точки в пространстве признаков. Иными словами, объем пространства, занимаемого состязательными примерами, растет экспоненциально вместе с размерностью. Для модели, обученной на ограниченной выборке, большая часть такого пространства остается неисследованной, а поведение модели в этих областях может быть нестабильным и легко поддаваться манипуляции [12].

Другим объяснением является линейность поведения моделей в локальных областях пространства признаков. По результатам работы И. Гудфеллоу и его коллеги (2015) сделали вывод, что уязвимость моделей может быть связана с их квазилинейным поведением в локальных областях пространства признаков [10; 15]. Несмотря на использование нелинейных функций активации, глубокие нейронные сети во многих аспектах ведут себя как линейные модели. Если изменить вход в направлении градиента функции потерь, выход модели может существенно измениться даже при очень малом возмущении. Этот эффект наиболее заметен в пространствах высокой размерности, где существует множество направлений, в которых можно увеличить значение функции потерь.

Еще одно направление исследований связывает уязвимость моделей с особенностями обучения и обобщения. Было показано, что модели часто полагаются на «хрупкие» (non-robust) признаки — корреляции в данных, которые не являются инвариантными к малым возмущениям, но тем не менее позволяют достичь высокой точности на тестовой выборке [14; 15]. Такое обучение ориентировано на минимизацию эмпирического риска на обучающей выборке, однако не обеспечивает устойчивости к состязательным возмущениям (adversarial perturbations). В результате модель усваивает

сложные и нередко неинтуитивные для человека закономерности, которые сравнительно легко нарушаются.

Наконец, не менее важным аспектом уязвимости выступает отсутствие у моделей подлинного понимания семантики данных. Как показано в [10], нейронные сети могут быть «обмануты» текстурами и случайным шумом, демонстрируя высокую уверенность в заведомо неправильных классификациях. Это закономерно связано с тем, что модели нередко полагаются на поверхностные статистические закономерности, а не на глубокое понимание объектов и их свойств.

1.3.3 Методы состязательных атак

Для систематического выявления и оценки уязвимостей моделей были разработаны методы состязательных атак. Такие методы классифицируются по различным критериям: по объему информации о модели, доступной атакующей стороне, по целям атаки, по стадии воздействия и по используемым стратегиям генерации состязательных примеров [16; 14].

Классификация по знаниям атакующего. Основной характеристикой атак является объем информации о модели, доступной атакующему [10; 13].

White-box-атаки предполагают, что атакующий имеет полный доступ к архитектуре модели, её параметрам и градиентам. Это позволяет применять наиболее эффективные методы, основанные на точном вычислении градиента [10]. *White-box-атаки* задают верхнюю границу уязвимости модели и используются для оценки её устойчивости в худшем случае.

Black-box атаки, напротив, реализуются в условиях отсутствия доступа к внутреннему устройству модели. Атакующий может только отправлять запросы и получать ответы (score или метки классов) [10]. Такие атаки считаются более реалистичными, поскольку соответствуют распространенным сценариям использования моделей через API. *Black-box-атаки* могут основываться на переносимости (transferability) состязательных примеров, сгенерированных для surrogate-модели, либо на методах оценки градиента по запросам (query-based methods) [10].

Grey-box-атаки занимают промежуточное положение, когда атакующий обладает частичной информацией: например, знает архитектуру модели, но не точные веса, или имеет доступ к аналогичному набору данных для обучения surrogate-модели [10].

Более детально классификация состязательных примеров представлена на рисунке 1.3.

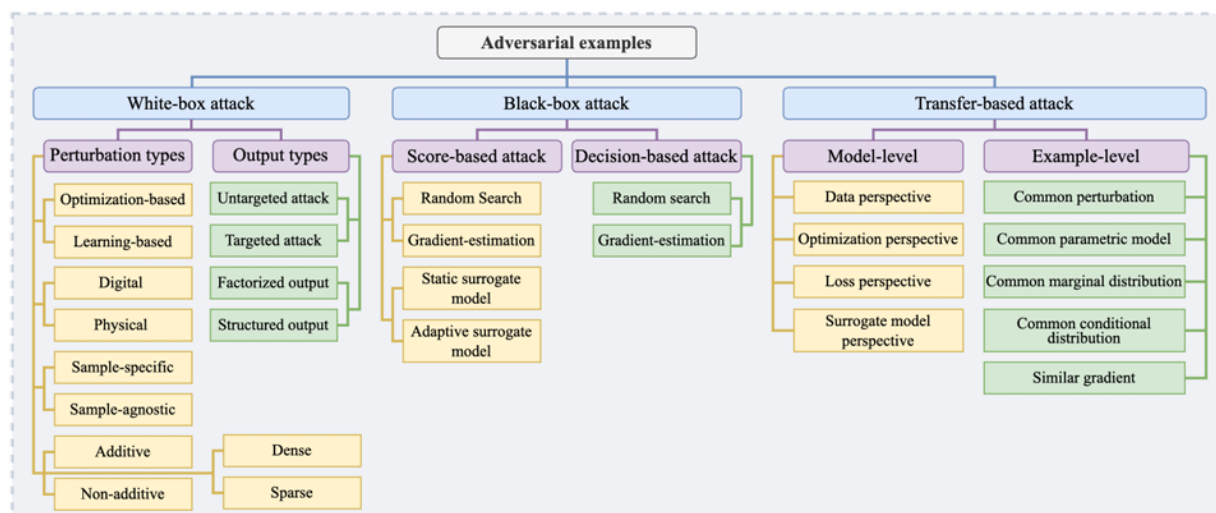


Рисунок 1.3 — Классификация состязательных примеров на этапе инференса

Классификация атак по стадии воздействия. Наиболее существенное различие проводится между атаками, происходящими на разных этапах жизненного цикла модели [16; 14].

Evasion-атаки (атаки уклонения) — это наиболее распространенный класс атак, которые происходят на этапе тестирования (инференса). Атакующий не вмешивается в процесс обучения, а модифицирует входные данные уже на этапе использования модели, чтобы вызвать ошибочную классификацию [13; 14]. К этому классу относятся все методы, генерирующие состязательные примеры путем добавления возмущений к тестовым образцам. Evasion-атаки представляют наибольшую практическую угрозу, так как для их реализации достаточно иметь доступ к входным данным модели.

Poisoning-атаки (атаки отравлением) — воздействуют на модель на этапе обучения. Атакующий внедряет специально подготовленные образцы в обучающий набор данных, искажая тем самым сам процесс обучения [13; 11]. В результате обученная модель начинает допускать систематические ошибки, которые проявляются уже во время тестирования. Poisoning-атаки бывают как незаметными (например, внедрение образцов с сохранением правильных меток), так и явными (изменение меток) [10].

Backdoor-атаки (атаки с закладкой) — представляют собой особый, более сложный вид poisoning-атак. В ходе такой атаки в модель внедряется скрытый «триггер» — определенный паттерн в данных, при появлении которого в тестовом образце модель начинает предсказывать заданный атакующим целевой класс [16; 14]. При этом на

обычных, «чистых» данных модель продолжает работать корректно, что делает такую атаку особенно скрытной. Backdoor-атаки могут быть реализованы как через модификацию обучающих данных, так и через прямое вмешательство в процесс обучения [16].

Основные различия между backdoor-атаками, weight-атаками и adversarial examples по входным данным, результату атаки и условиям реализации приведены в таблице 1.1.

Таблица 1.1 — Сравнение основных парадигм состязательных атак в машинном обучении

Attack paradigm	Inputs	Outputs	Stealthiness		Benign consistency		Adversarial inconsistency	
			AML.S _x	AML.S _w	AML.C _A	AML.C _B	AML.I _A	AML.I _B
Backdoor attack	Training dataset D_0 or control of training process	D_ε or $f_{w_\varepsilon}(\cdot)$	•		•		•	
Weight attack	$f_{w_0}(\cdot)$ & a few benign data & control of device memory	$f_{w_\varepsilon}(\cdot)$	•	•	•	•	•	
Adversarial example	$(x_0, y_0, y_\varepsilon)$ & weights or access permission of $f_{w_0}(\cdot)$	x_ε	•			•		•

Классификация по целям атакующего. По своим целям атаки делятся на два основных типа [10; 16]:

Untargeted-атаки (ненаправленные) — атакующий стремится вызвать любую ошибку классификации, независимо от того, какой именно класс будет предсказан.

Targeted-атаки (направленные) — атакующий стремится заставить модель предсказать конкретный, заранее заданный класс. Такие атаки являются более сложными, так как требуют не просто нарушения работы модели, а точного управления её выводом.

Классификация по области применения. Атаки различаются и по среде, в которой они реализуются [10; 15].

Digital-атаки — реализуются в цифровом пространстве, где атакующий может точно контролировать значения пикселей или иных признаков входного объекта. Это наиболее распространенный класс атак в исследовательской литературе.

Physical-атаки — реализуются в физическом мире, где состязательный пример должен сохранять свою эффективность после печати, фотографирования или иных физических преобразований. Примерами являются специальные наклейки на дорожных знаках, способные вводить в заблуждение системы автономного вождения, или специальные очки для обхода систем распознавания лиц [10].

1.3.4 Методы генерации состязательных примеров

Методы генерации состязательных примеров разделяются по используемым подходам к решению оптимизационной задачи поиска минимального возмущения, способного вызвать ошибку модели [10; 15].

Градиентные методы (Gradient- based). Наиболее известные и широко используемые методы основанные на использовании градиента функции потерь по входным данным.

Fast Gradient Sign Method (FGSM) — известен среди первых и наиболее простых методов, предложенный в исследовании И. Гудфеллоу и коллегами [10; 12]. Состязательный пример генерируется за один шаг (1.7):

$$x^{adv} = x - \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)) \quad (1.7)$$

где ϵ — величина возмущения, θ — параметры модели, а $\nabla_x J(\cdot)$ — градиент функции потерь. FGSM отличается высокой скоростью, но требует относительно больших возмущений для успешной атаки.

Basic Iterative Method (BIM) и Projected Gradient Descent (PGD) — итеративные методы, которые считаются одними из сильнейших градиентных методов [10; 12]. PGD выполняет несколько шагов градиентного обновления, после каждого из которых проецирует результат обратно в допустимую область (шар радиуса ϵ), как показано в (1.8):

$$x_{t+1}^{adv} = P_{B_\epsilon(x)}(x_t^{adv} + \alpha \cdot \text{sign}(\nabla_x J(x_t^{adv}, y))) \quad (1.8)$$

По сравнению с одношаговыми методами PGD позволяет находить более сильные состязательные примеры и часто используется для оценки максимальной уязвимости моделей.

Momentum Iterative FGSM (MI- FGSM) — метод, улучшающий сходимость итеративных градиентных атак за счет внедрения импульса, что позволяет сглаживать шум и избегать локальных минимумов [12].

Оптимизационные методы (Optimization- based). Эти методы формулируют задачу поиска состязательного примера как задачу оптимизации с ограничениями [10; 12].

Классическим примером является *метод Карлини-Вагнера (C&W)*, нацеленный на минимизацию нормы возмущения при условии успешной атаки (1.9):

$$\min_{\delta} \|\delta\|_p + c \cdot L(x + \delta, y_{target}) \quad (1.9)$$

C&W-атаки считаются одними из наиболее эффективных и способны обходить многие защитные механизмы, включая defensive distillation [10].

Генеративные методы (Generative model- based). Альтернативный подход заключается в обучении генеративной модели (например, GAN) создавать состязательные примеры за один проход [12]. В отличие от градиентных и оптимизационных методов, которые работают с каждым образцом индивидуально, генеративные методы позволяют создавать состязательные примеры для целого набора данных без итеративного поиска для каждого образца.

Sparse-атаки. Особая группа атак, которые стремятся модифицировать минимальное количество элементов входного образца, а не ограничивать общую норму возмущения [10]. Одной из экстремальных форм является *One-Pixel Attack*, которая модифицирует всего один пиксель изображения, чтобы вызвать ошибку классификации. Такие атаки демонстрируют, что уязвимость моделей может быть сосредоточена в очень небольшой части входных данных.

1.3.5 Значение состязательных атак для исследования уязвимостей

Методы состязательных атак в области машинного обучения выполняют двойную функцию. С одной стороны, они представляют реальную угрозу для безопасности систем, особенно в таких весомых приложениях, как автономное вождение, биометрическая аутентификация и медицинская диагностика [13]. С другой стороны, они служат важнейшим инструментом для исследования и оценки уязвимостей моделей. Только подвергая модель сильным состязательным атакам, можно объективно оценить её устойчивость и выявить скрытые недостатки, которые могут проявиться в реальных условиях эксплуатации [17].

Как отмечается в [12], анализ состязательных атак позволяет глубже понять ограничения современных алгоритмов машинного обучения и выявить слабые места в понимании механизмов работы нейронных сетей. Таким образом, развитие методов атак и методов защиты происходит параллельно, открывая возможности для создания более надежных и безопасных систем искусственного интеллекта.

1.4 Исследование методов повышения устойчивости моделей и анализ устойчивости Mamba

1.4.1 Методы повышения устойчивости моделей

В ответ на уязвимости, описанные в предыдущем разделе, исследовательским сообществом был разработан широкий спектр методов защиты, направленных на повышение устойчивости моделей к состязательным атакам. Однако эти методы, как будет показано в данном разделе, имеют существенные ограничения, включая высокую вычислительную сложность, узкую применимость и отсутствие универсальных решений, способных защитить модель от всех типов атак [18; 19]. При этом сами угрозы в состязательном машинном обучении обычно рассматриваются в контексте сценариев атак из предыдущего раздела, что важно учитывать при выборе защитных механизмов [20]. На основе анализа, представленного в [18; 19], методы защиты могут быть классифицированы по стадиям жизненного цикла модели, на которых они применяются и по способу смягчения угроз. В частности, в обзоре [19] защита систематизируется по пяти стадиям AML life-cycle: pre-training, training, post-training, deployment и inference [19].

Состязательное обучение (Adversarial Training). Состязательное обучение является одним из наиболее распространенных и эффективных подходов к повышению устойчивости моделей [18; 19]. Его основная идея состоит во включении состязательных примеров в обучающую выборку, что позволяет модели адаптироваться к искаженным входным данным [18; 19].

Формально, задача состязательного обучения может быть сформулирована как задача минимизации эмпирического риска с учетом наихудших возмущений, представленная в (1.10):

$$\min_{\theta} E_{(x,y) \sim D} [\max_{\|\delta\| \leq \epsilon} L(f_{\theta}(x + \delta), y)] \quad (1.10)$$

где δ — состязательное возмущение, ϵ — максимально допустимая величина возмущения, L — функция потерь [18].

Наиболее известными реализациями данного подхода являются PGD-AT (Projected Gradient Descent Adversarial Training) и TRADES (TRadeoff-inspired Adversarial Defense via Surrogate-loss minimization) [18; 22]. PGD-AT строится на многошаговом градиентном методе для генерации сильных состязательных примеров, а TRADES добавляет

регуляризатор, контролирующий баланс между устойчивостью и точностью на чистых данных [18].

Несмотря на эффективность, состязательное обучение имеет ряд существенных недостатков: оно значительно увеличивает вычислительные затраты, так как требует генерации состязательных примеров на каждой итерации обучения [18], что приводит к снижению точности на чистых данных — так называемому компромиссу между устойчивостью и обобщением [19], а также модели, обученные с помощью состязательного обучения, демонстрируют устойчивость преимущественно к тем типам атак, которые использовались при обучении, и могут быть легко уязвимы к новым, неизвестным атакам [18].

Обнаружение состязательных примеров (Detection). Альтернативным подходом является обнаружение состязательных примеров, при котором основная модель дополняется детектором, идентифицирующим потенциально опасные входные данные [18; 19]. Такие детекторы могут быть основаны на различных принципах: анализе активаций нейронов, оценке статистических отклонений, анализе согласованности предсказаний при различных преобразованиях входа [12; 18].

Основное преимущество методов обнаружения заключается в том, что они не требуют модификации обучаемой модели и могут быть добавлены к уже обученным системам [18]. Однако и у этого подхода есть серьезные ограничения. Как показано в [18], многие методы обнаружения можно обойти адаптивными атаками, специально сконструированными для «обмана» детектора. Также, методы обнаружения могут давать ложные срабатывания, что снижает их практическую ценность в критических приложениях.

Предобработка входных данных (Preprocessing). Методы предобработки направлены на удаление или ослабление состязательных возмущений из входных данных до их подачи в модель [18; 19]. К таким методам относятся JPEG-компрессия, сглаживание, уменьшение битовой глубины, использование автоэнкодеров для очистки изображений и другие [18]. Общая идея таких методов заключается в том, что состязательные возмущения, как правило, имеют специфические частотные или статистические характеристики, которые могут быть удалены без существенного ухудшения качества полезного сигнала [18, 21].

Однако такие преобразования могут удалять не только состязательные возмущения, но и важные детали исходных данных, ухудшая их качество и снижая точность модели на чистых образцах [18, 21]. Кроме того, эти методы часто неэффективны против сильных адаптивных атак, которые учитывают применяемые преобразования.

1.4.2 Ограничения существующих методов защиты

Анализ существующих методов защиты позволяет выделить ряд фундаментальных ограничений, которые сохраняются даже при использовании наиболее продвинутых подходов.

Высокая вычислительная сложность. Состязательное обучение требует многократной генерации состязательных примеров, что на порядок увеличивает время обучения в сравнении со стандартным режимом [18]. Как показано в [18], применяя PGD для модели WideResNet-28-10 на CIFAR-10 состязательное обучение требует более 11 часов, в то время как стандартное обучение занимает менее часа. Эта проблема делает применение таких методов затруднительным при ограниченных вычислительных ресурсах.

Узкая применимость. Большинство существующих методов защиты разработаны для конкретных типов атак и архитектур. Например, методы, эффективные против ℓ_∞ – ограниченных атак, могут быть неэффективны против ℓ_2 -ограниченных или пространственных атак [18]. Более того, методы, разработанные для свёрточных нейронных сетей, иногда не работают для трансформеров или SSM-моделей [20].

Отсутствие универсальных решений. Не существует единого метода, способного обеспечить устойчивость ко всем типам состязательных атак одновременно. Как отмечается в [18; 19], всегда должен присутствовать компромисс как между устойчивостью и точностью на чистых данных, так и между устойчивостью к различным типам возмущений.

Проблема переносимости. Модели, обученные с использованием состязательного обучения на одном типе атак, могут сохранять уязвимость к другим типам атак. Защита осложняется тем, что атакующий может действовать на разных стадиях жизненного цикла модели и с различным объемом знаний о системе [20; 19]. Помимо этого, состязательные примеры, сгенерированные для одной модели, часто переносятся на другие модели, что делает защиту сложной задачей в условиях, когда заранее не известна архитектура [20].

1.4.3 Устойчивость моделей на основе SSM и Mamba

Существующие исследования устойчивости SSM. Устойчивость моделей на основе пространства состояний (SSM) к состязательным атакам пока остается сравнительно малоизученной областью [5; 6]. Первые исследования устойчивости SSM, такие как [22], показывают, что чистые SSM-структуры (S4, DSS, S5) практически не выигрывают от состязательного обучения. Применение PGD-AT и TRADES к этим моделям приводит к значительному снижению точности на чистых данных (до 15% на CIFAR-10) при незначительном повышении устойчивости. Наблюдается явный компромисс между устойчивостью и обобщением, который характерен и для традиционных архитектур, но проявляется в SSM даже более выражено.

Особый интерес представляет исследование, проведенное в [22], где сравнивались различные варианты SSM-архитектур (S4, DSS, S5, Mega и Mamba) в условиях состязательного обучения. Соответствующая динамика обучения, тестирования и состязательного тестирования представлена на рисунках 1.4 и 1.5.

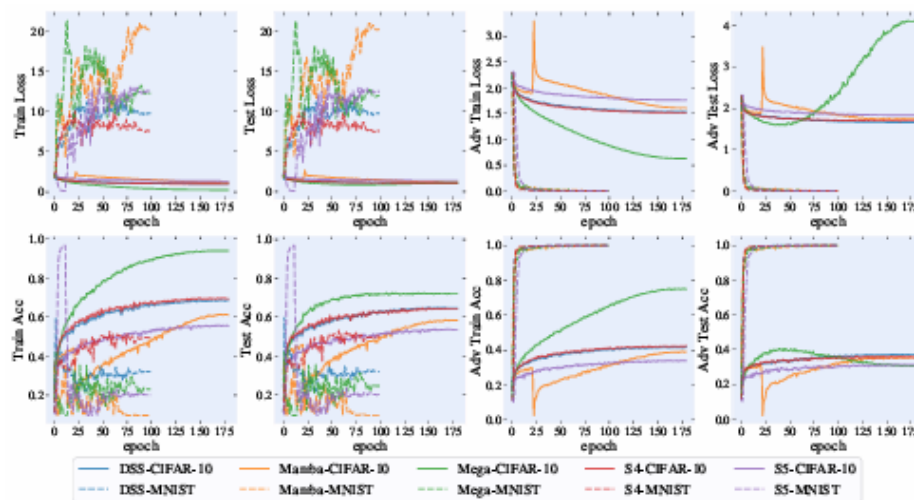


Рисунок 1.4 — Динамика обучения, тестирования и состязательного тестирования (PGD-10) для моделей S4, DSS, S5, Mega и Mamba на наборах данных CIFAR-10 и MNIST при использовании PGD-AT

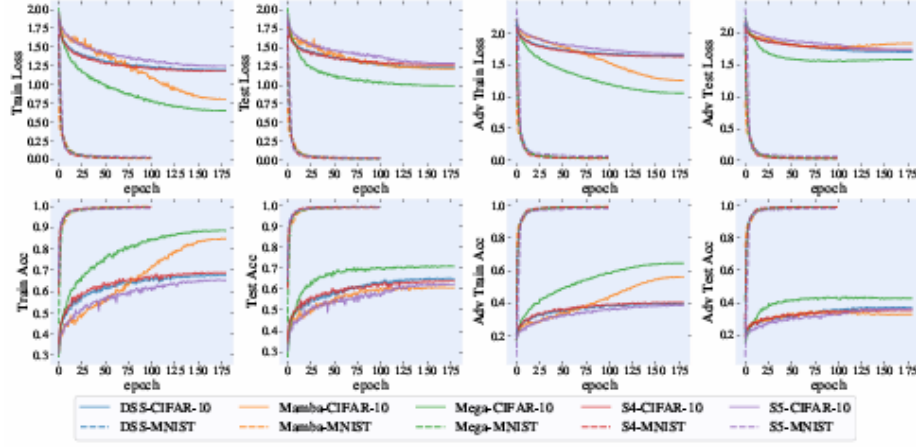


Рисунок 1.5 — Динамика обучения, тестирования и состязательного тестирования (PGD-10) для моделей S4, DSS, S5, Mega и Mamba на наборах данных CIFAR-10 и MNIST при использовании TRADES

Полученные результаты показывают, что чистые SSM-структуры не выигрывают от состязательного обучения. Модели S4, DSS и S5 демонстрируют значительное падение точности на чистых данных после состязательного обучения, при этом устойчивость к атакам остается низкой. Более того, Расширение до data-dependent SSM (Mamba) не решает проблему. Несмотря на введение адаптивных параметров, зависящих от входных данных, Mamba не показывает существенного преимущества перед фиксированными SSM в условиях состязательного обучения. Только интеграция механизмов внимания дает улучшение. Модель Mega, сочетающая SSM и attention-блоки, демонстрирует лучший баланс между устойчивостью и точностью, однако это сопровождается проблемой переобучения на состязательных примерах (robust overfitting).

Особенности архитектуры Mamba, влияющие на устойчивость. Для понимания причин низкой эффективности состязательного обучения для Mamba необходимо обратиться к анализу её архитектурных особенностей, проведенному в [22].

Фиксированная нижняя граница ошибки. Как показано в теоретическом анализе [22], для фиксированных SSM (S4, DSS) ошибка вывода при наличии состязательных возмущений имеет фиксированную нижнюю границу (1.11), зависящую от параметров модели:

$$\mathbb{E}_\epsilon[\|y' - y\|_2^2] \geq \mu^2 c_1 \sum_{j=1}^L \left(\prod_{i=1}^{j-1} |\tilde{\lambda}_i^{\min}| \right) (\bar{C}_j \bar{B}_j)^2 \quad (1.11)$$

Эта граница возрастает с увеличением длины последовательности L , что ведёт к неизбежному накоплению ошибки. Даже при минимизации ошибки в процессе состязательного обучения, фиксированные SSM не могут опуститься ниже этого предела.

Проблема адаптивных параметров Mamba. Mamba использует адаптивные параметры $B_k(u_k)$, $C_k(u_k)$, $\Delta t_k(u_k)$ зависящие от входных данных [5; 6]. В процессе дискретизации с удержанием нулевого порядка (ZOH) при $\Delta t_k \rightarrow 0$ возникает вычислительная неустойчивость, приводящая к $\|B_k\| \rightarrow \infty$. Как следствие, верхняя граница ошибки (1.12) может стремиться к бесконечности:

$$\max_u \bar{C}_k(u_k) \bar{B}_k^2(u_k) \leq \|\bar{C}_k(u_k)\|_2^2 \|\bar{B}_k(u_k)\|_2^2 \rightarrow \infty \quad (1.12)$$

Это объясняет, почему Mamba не только не выигрывает от состязательного обучения, но и демонстрирует аномально высокие значения ошибки при наличии состязательных возмущений.

Роль внимания и линейных слоев. Экспериментально показано [22], что включение линейных слоев после SSM и, особенно, механизмов внимания, позволяет значительно снизить расхождение между чистыми и состязательными признаками. В частности, механизм внимания обеспечивает адаптивное масштабирование выхода SSM, что позволяет преодолеть фиксированную нижнюю границу ошибки. Однако это же приводит к переобучению на состязательных примерах из-за высокой сложности модели.

1.4.4 Проблема недостаточной изученности устойчивости Mamba

Проведённый анализ современных исследований в области устойчивости SSM архитектур позволил выявить ряд пробелов в доступной литературе.

Прежде всего, так как данное направление относительно новое и только набирает свою популярность большинство существующих работ по SSM сосредоточены на повышении эффективности и точности моделей на чистых данных [5; 6], оставляя вопросы безопасности и устойчивости в качестве будущего возможного направления, не затрагивая его напрямую [19; 22]. Сильно заметно отсутствие систематических исследований в этой области и крайне ограниченное число работ, посвященных устойчивости SSM-моделей к состязательным атакам. Как отмечают авторы исследования [22], их работа фактически стала первой попыткой оценить устойчивость различных вариантов SSM в условиях

состязательного обучения. Отсутствие накопленного опыта и установленных практик существенно затрудняет разработку эффективных защитных механизмов.

Также важно отметить неадаптированность существующих методов повышения устойчивости к особенностям SSM-архитектур. Как показано в [22], прямое применение классических методов повышения устойчивости разработанных для свёрточных нейронных сетей и трансформеров, таких как PGD-AT и TRADES, к моделям типа Mamba не дает ожидаемых результатов. Как показано в теоретическом анализе [5; 6], SSM обладают специфическими уязвимостями, связанными с их рекуррентной природой и механизмом распространения состояния. Состязательные возмущения могут накапливаться в процессе рекуррентных вычислений, что делает SSM особенно чувствительными к атакам на длинных последовательностях.

Эти пробелы могут повлиять на качество существующих систем в основе которых лежат SSM-модели и существенно ограничить возможности их безопасного применения в будущих проектах.

1.5 Выводы

Проведенный анализ позволяет сформулировать основную проблему, определяющую направление настоящего исследования: несмотря на высокую эффективность Mamba в задачах обработки длинных последовательностей и её растущую популярность, устойчивость этой архитектуры к состязательным атакам остается недостаточно изученной. Существующие методы повышения устойчивости, разработанные для свёрточных нейронных сетей и трансформеров, не учитывают архитектурные особенности SSM-моделей и не показывают удовлетворительных результатов при применении к Mamba. В настоящее время отсутствуют как глубокое понимание механизмов уязвимости, так и практические методы защиты, адаптированные к специфике данной архитектуры. Это создаёт явный разрыв между возможностями Mamba при внедрении в прикладные области и отсутствием гарантий безопасного функционирования таких систем в условиях возможных состязательных воздействий. Предполагается, что исследование этого направления позволит не только повысить безопасность конкретной архитектуры, но и способствовать развитию более надежных моделей в целом. Обнаруженные недостатки определяют цели и задачи дальнейшего исследования.

1.6 Постановка цели и задач курсовой работы

Целью настоящей работы является разработка метода повышения устойчивости модели Mamba к состязательным атакам, учитывающего архитектурные особенности SSM-моделей и позволяющего достичь более высоких показателей устойчивости по сравнению с прямым применением существующих подходов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Формализовать задачу повышения устойчивости моделей на основе Mamba.
2. Разработать метод повышения устойчивости, учитывающий архитектурные особенности модели и самих данных. Метод должен быть направлен на преодоление нижней границы ошибки, характерной для детерминированных SSM, и предотвращение эффекта «взрыва» ошибки, возникающего при адаптивной параметризации.
3. Выбрать метрики для оценки устойчивости и качества модели.
4. Сформулировать требования к реализуемому фреймворку состязательного обучения моделей на основе Mamba.

5. Реализовать предложенный метод в виде программного модуля, интегрируемого с существующими фреймворками глубокого обучения.
6. Провести экспериментальную оценку эффективности метода на стандартных наборах данных.
7. Исследовать поведение Mamba под воздействием различных типов состязательных атак. На различных наборах данных определить специфические уязвимости Mamba, обусловленные рекуррентной природой и механизмом распространения состояния.
8. Сравнить разработанный метод с существующими подходами по стандартным метрикам, а также по вычислительной эффективности.

2. Моделирование метода обучения устойчивой к состязательным атакам модели на основе Mamba

Раздел посвящен формализации задачи повышения устойчивости архитектуры Mamba, разработке адаптированного метода состязательного обучения и обоснованию выбора метрик. В рамках раздела выполняется анализ теоретических ограничений, присущих SSM-моделям. На основе выявленных ограничений и существующих методов защиты моделей разрабатывается метод, включающий стабилизацию параметра дискретизации и регуляризацию скрытого состояния. Также определяются метрики для количественной оценки устойчивости обученной модели.

2.1. Постановка задачи повышения устойчивости модели Mamba

2.1.1 Формализация задачи

Введём обозначения:

- Пусть $D = \{(x_i, y_i)\}_{i=1}^N$ — исходная выборка данных, где $x_i \in X$ — исходный объект, подающийся на вход, а $y_i \in Y$ — его истинная метка.
- Отображение $f_\theta : X \rightarrow Y$ — исходная модель, на базе архитектуры Mamba с параметрами θ , возвращающая предсказанный класс.
- Пусть для каждого исходного объекта x_i задано состязательное возмущение δ_i , сгенерированное в рамках допустимого множества $\Delta = \{\delta : \|\delta\|_p \leq \epsilon\}$ с помощью определённого алгоритма атаки. Тогда состязательный пример обозначается как:

$$x'_i = x_i + \delta_i$$

- Дополнительно введём S — область допустимых возмущений, определяемая ограничением $\|\delta\|_p < \epsilon$. В рамках данной работы рассматривается l_∞ - норма, как наиболее репрезентативная для задач компьютерного зрения (MNIST, CIFAR-10) и обработки аудио (Speech commands dataset).

Обучение на исходном датасете D . Задача минимизации эмпирического риска для базового обучения (2.1) формулируется как поиск параметров θ^* , минимизирующих кросс-энтропию L_{CE} на множестве D :

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N L_{CE}(f_\theta(x_i), y_i) \quad (2.1)$$

Расширение до состязательного набора данных D' . Обозначим $A(x_i, f_\theta, y_i)$ — оператор состязательной атаки, который для каждого объекта x_i генерирует пример x_i' при помощи возмущения δ_i такого, что $x_i' = x_i + \delta_i \in \{x + \delta : \|\delta\|_p \leq \epsilon\}$.

На основе этого введем состязательно-расширенный датасет D' (2.2):

$$D' = \{ (x_i', y_i) \mid x_i' = A(x_i, f_\theta, y_i), \forall (x_i, y_i) \in D \} \quad (2.2)$$

Формализация устойчивости. Задача повышения устойчивости (робастности) модели формулируется как поиск параметров θ^* , минимизирующих суммарный риск на объединённом множестве $D \cup D'$ с учётом стабилизации внутренней динамики скрытых состояний, что отражено в (2.3).

$$\theta^* = \min_{\theta} \left[\frac{1}{|D|} \sum_{(x_i, y_i) \in D} L_{CE}(f_\theta(x_i), y_i) + \frac{\alpha}{|D'|} \sum_{(x_i', y_i) \in D'} \max_{\delta \in S} L_{total}(f_\theta, x_i', \delta, y_i) \right] \quad (2.3)$$

где первое слагаемое обеспечивает сохранение качества классификации на чистых данных D , а второе — минимизацию риска на состязательно-расширенном датасете D' в условиях наихудшего допустимого возмущения $\delta \in S$. Расширенная функция потерь L_{total} включает в себя как задачу корректной классификации состязательных примеров, так и требование инвариантности внутренних представлений модели к возмущению δ и имеет вид (2.4):

$$L_{total} = L_{CE}(f_\theta(x_i + \delta), y_i) + \lambda \cdot \Phi(f_\theta, x_i, \delta) \quad (2.4)$$

Здесь $\lambda \geq 0$ — коэффициент регуляризации, а $\Phi(f_\theta, x_i, \delta)$ — штрафной функционал, измеряющий степень дестабилизации внутренних состояний модели под воздействием δ . При $\lambda = 0$ задача сводится к стандартному состязательному обучению на $D \cup D'$, где устойчивость обеспечивается выбором оператора A , а также предобработкой входа и ограничениями архитектуры модели. При $\lambda > 0$ дополнительно минимизируется Φ . В отличие от базового обучения, где минимизируется только L_{CE} на чистых данных D , введение Φ и максимизация по $\delta \in S$ обеспечивают устойчивость модели к наихудшему допустимому возмущению. Конкретный вид Φ , определяемый архитектурными особенностями Mamba, задаётся в подразделе 2.2.

Предложенная формализация является модификацией подхода, описанного в работе [23]: в отличие от исходной постановки, целевая функция расширена за счёт введения

штрафного функционала $\Phi(f_\theta, x_i, \delta)$, учитывающего специфику SSM-архитектуры через регуляризацию дрейфа скрытых состояний, а также явно разделена на слагаемые качества на чистых данных D и робастности на состязательно-расширенном датасете D' .

2.1.2 Требования к оператору атаки при формировании набора данных D'

Из формулы (2.3) следует, что качество состязательно-расширенного датасета D' непосредственно определяет второе слагаемое целевой функции. Чем точнее оператор A находит наихудшее $\delta \in S$ для текущей модели f_θ , тем более информативными оказываются примеры в D и тем эффективнее минимизация L_{total} снижает реальный риск. Для обеспечения этого сначала формулируются общие требования к оператору A , а затем на основе анализа архитектуры Mamba устанавливается, каким образом эти требования конкретизируются для SSM-специфичного оператора A_{SSM} .

Требования к оператору A . Для формирования информативного D' оператор A должен удовлетворять следующим требованиям:

- Оператор A должен находить такое $\delta_i \in S$, которое максимизирует расширенную функцию потерь. В зависимости от значения λ максимизируемый функционал принимает вид (2.5):

$$A(x_i, f_\theta, y_i) = \arg \max_{\|\delta\|_p < \epsilon} \begin{cases} L_{CE}(f_\theta(x_i + \delta), y_i), & \lambda = 0 \\ L_{CE}(f_\theta(x_i + \delta), y_i) + \lambda \cdot \Phi(f_\theta, x_i, \delta), & \lambda > 0 \end{cases} \quad (2.5)$$

- Для покрытия условий наихудшего случая оператор A использует полную информацию об архитектуре и параметрах модели θ , что позволяет вычислять градиент максимизируемого функционала напрямую, то есть модель рассматривается в режиме “белого ящика” (white-box). Из (2.4) следует, что при $\lambda > 0$ градиент включает оба слагаемых L_{total} :

$$\nabla_\delta L_{total} = \nabla_\delta L_{CE}(f_\theta(x_i + \delta), y_i) + \lambda \cdot \nabla_\delta \Phi(f_\theta, x_i, \delta) \quad (2.6)$$

при $\lambda = 0$ оператор использует только $\nabla_\delta L_{CE}(f_\theta(x_i + \delta), y_i)$. Рассмотрение данного режима необходимо, поскольку оператор, обладающий полным знанием о θ , порождает возмущения, с наибольшей вероятностью выявляющие уязвимости f_θ .

- Все сгенерированные возмущения должны удовлетворять ограничению $\delta_i \in S$ вне зависимости от значения λ , что обеспечивает семантическую близость x'_i к исходному объекту x_i .
- Датасет D' должен формироваться с использованием операторов из разных классов $A \in \{A_{grad}$ (градиентные методы генерации шума), A_{SSM} (SSM – специфичные методы генерации шума)), что предотвращает переобучение f_θ к конкретному методу генерации возмущений. Формальные определения классов операторов приведены в подразделе 2.1.3.

Архитектурная уязвимость Mamba и конкретизация требований к A_{SSM} . Для понимания того, каким образом воздействие A_{SSM} наиболее эффективно реализуется в Mamba, необходимо рассмотреть рекуррентную природу её скрытых состояний (2.7). Специфика Mamba заключается в зависимости скрытого состояния h_t от входного тензора x_t и параметра дискретизации Δ_t :

$$h_t = SSM(h_{t-1}, x_t, \Delta_t) \quad (2.7)$$

В отличие от трансформеров, где входные данные обрабатываются с ограниченной глубиной пути внимания, в Mamba малые изменения δ на ранних шагах ($t \rightarrow 0$) могут экспоненциально усиливаться через механизм селективного сканирования Δ_t . Чувствительность штрафного функционала Φ к состязательному воздействию формализуется через градиент скрытого состояния (2.8):

$$\frac{\partial h_t}{\partial \delta} = \sum_{k=0}^t \left(\prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \right) \cdot \frac{\partial h_k}{\partial x_k} \cdot \frac{\partial x_k}{\partial \delta} \quad (2.8)$$

Данное выражение показывает, что операторы A_{SSM} дестабилизируют h_t одним из двух способов: либо находя в режиме белого ящика такое направление $\delta \in S$, которое максимизирует норму градиента скрытого состояния и провоцирует лавинообразный рост $\Phi(f_\theta, x_i, \delta)$ вдоль временной оси, либо внося в последовательность структурные возмущения, которые накапливают ошибку в h_t через рекуррентные связи без явной градиентной максимизации. Следовательно, задача повышения устойчивости из подраздела 2.1.1 требует, чтобы минимизация L_{total} по θ подавляла хотя бы один механизм дестабилизации скрытых состояний из описанных в (2.9):

$$\min_{\theta} \left\| \frac{\partial h_t}{\partial \delta} \right\|_p, \quad \forall \delta \in S \quad (2.9)$$

$$\min_{\theta} \|h_t(x) - h_t(x')\|_2^2$$

Таким образом, мы переходим от наблюдения к проектированию инвариантной динамики скрытых состояний. Мы обучаем модель так, чтобы механизм селективного сканирования был невосприимчив к возмущениям, внесенным оператором A , не позволяя ошибке накапливаться в h_t . Это определяет конкретный вид Φ и методы D' с использованием A_{SSM} , описываемые в подразделе 2.1.3 и 2.2.

2.1.3 Используемые методы для генерации возмущений

В соответствии с требованиями описанными в подразделе 2.1.2 предлагается использование следующих различных по своей природе и оптимальной сфере применения типов атак.

1. Градиентные методы A_{grad}

Для проведения анализа по выявлению падения устойчивости в сравнении с Transformer и CNN были выбраны атаки PGD (Projected Gradient Descent), CW(Carlini-Wagner), R-FGSM (Fast Gradient Sign Method с рандомным стартом) и AutoAttack. Последняя является ансамблем из четырёх наиболее эффективных для выявления всесторонних уязвимостей методов: двух версий адаптивных PGD, минимизации расстояний до границы решения (FAB) и SquareAttack. Данные методы направлены на поиск возмущения δ , минимизирующего вероятность предсказания верного класса путем итеративного спуска по градиенту функции потерь.

Для формализации используемых операторов атак $A_{grad}(x, f_{\theta}, y)$ в рамках оценки робастности модели f_{θ} , приведем их краткое математическое описание:

- PGD является итеративным итерационным методом, использующим градиент функции потерь для поиска оптимального возмущения в ϵ — окрестности. $x^{(k+1)} = \text{proj}_S \left(x^{(k)} + \alpha \cdot \text{sign} \left(\nabla_x L_{CE}(f_{\theta}(x^{(k)}), y) \right) \right)$. В контексте Mamba итеративная максимизация ошибки позволяет оценить устойчивость модели f_{θ} к накопленным отклонениям в рекуррентных слоях.

- CW (Carlini-Wagner): Атака, минимизирующая функцию потерь $L_{CW} = \|\delta\|_2^2 + c \cdot \max_{j \neq y} (\max Z(x + \delta)_j - Z(x + \delta)_y, -\kappa)$, где $Z(\cdot)$ — логиты модели, а κ — параметр уверенности. Она находит минимально необходимое возмущение для изменения решения модели, что позволяет выявить предельные границы устойчивости логитов f_θ в Mamba-архитектуре.
- R-FGSM (Randomized Fast Gradient Sign Method): Метод, добавляющий случайный старт перед выполнением одного градиентного шага. $x' = x + \eta + \alpha \cdot \text{sign}(\nabla_x L_{CE}(f_\theta(x + \eta), y))$, где $\eta \sim \mathcal{U}(-\epsilon, \epsilon)$. Это позволяет избежать ловушек в ландшафте функции потерь и проверить, насколько модель чувствительна к быстрым, грубым возмущениям в начале последовательности.
- AutoAttack: Комплексный ансамбль $A_{AA} = \{A_{PGD}, A_{FAB}, A_{Square}\}$, исключаяющий эффект «маскировки градиентов». Использование AutoAttack необходимо для того, чтобы подтвердить, что модель устойчива не к конкретному градиентному методу, а к спектру подходов к генерации состязательных примеров.

2. SSM-специфичные методы A_{SSM}

Помимо стандартных градиентных методов, также были выбраны специализированные атаки, направленные на специфику SSM-архитектур: HiSPA (Hidden State Poisoning), Sliding Patch и Informational Gap. Выбор данных методов обусловлен их направленным воздействием на скрытые состояния модели, что позволяет наглядно продемонстрировать существенный разрыв в показателях робастности между Mamba и классическими архитектурами. Данные атаки принципом действия заметно отличаются от градиентных:

- HiSPA (Hidden State Poisoning Attack) — целенаправленная дестабилизация скрытых состояний моделей класса SSM путем вычисления градиентов функции потерь относительно внутренних векторов скрытых состояний h_t (2.10):

$$A_{HiSPA}(x, f_\theta, y) \Rightarrow \delta^* = \arg \max_{\|\delta\|_p \leq \epsilon} \|\nabla_{h_t} L(f_\theta(x + \delta), y)\|_2 \quad (2.10)$$

Метод минимизирует δ при максимальном влиянии на латентный дрейф, вызывая лавинообразное накопление ошибки через рекуррентные связи.

Главным отличием метода от стандартных методов состязательных атак является перенос точки градиентного воздействия с входного тензора на внутреннее латентное пространство модели, что проиллюстрировано на рисунке 2.1.

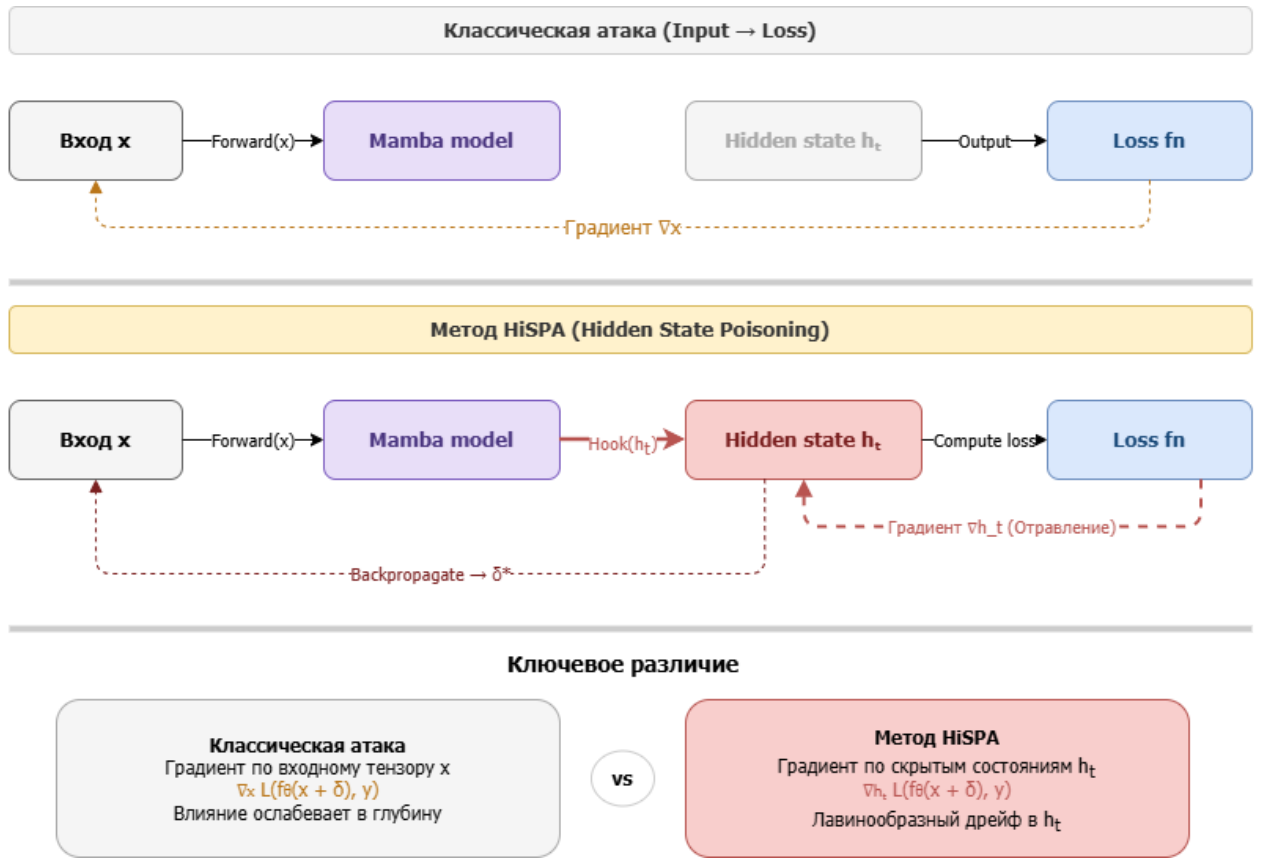


Рисунок 2.1 — Сравнительная схема последовательности генерации градиентов: базовый подход и метод HiSPA

- Sliding Patch (Скользящий блок шума) — алгоритм проверки контекстной устойчивости. Метод (2.11): добавляет шум высокой амплитуды на фиксированный сегмент последовательности (35–45% от её общей длины) и случайно динамически смещает его вдоль временной оси

$$A_{patch}(x) \Rightarrow x'_t = x_t + \mathbb{I}(t \in T_{sub}) \cdot \omega \quad (2.11)$$

где $\omega \sim \mathcal{N}(0, \sigma^2)$, а T_{sub} — случайно смещаемый временной интервал. Метод позволяет выявить, в каких фазах T модель теряет контекстную устойчивость.

- Info Gap (Информационный разрыв) — атака на механизм гейтования Δ_t , с помощью эмуляции «информационной стены» (переполнения гейтов) путем принудительной замены фрагмента данных на константные значения (заполнители) (2.12):

$$A_{gap}(x) \Rightarrow x'_t = c, \forall t \in T_{sub}, \text{ где } c \text{ — константа.} \quad (2.12)$$

Для Mamba этот метод позволяет оценить, насколько эффективно механизм селективного сканирования способен компенсировать неинформативные резкие провалы в плотности входных данных, не позволяя им испортить итоговое скрытое состояние.

2.2. Разработка метода повышения устойчивости для Mamba

По результатам анализа в разделе 2.1 можно сформулировать вывод, что: задача повышения устойчивости допускает два режима в зависимости от λ : при $\lambda = 0$ устойчивость обеспечивается выбором оператора A и методами защиты на уровне предобработки и архитектуры модели, при $\lambda > 0$ дополнительно минимизируется дрейф скрытых состояний через Φ , на основании чего были разработаны два комплексных метода повышения устойчивости.

2.2.1 Адаптивный метод повышения устойчивости на уровне обучения

Методы на уровне обучения применяются при $\lambda > 0$: введение регуляризатора Φ в состав L_{total} является ключевым отличием от стандартного состязательного обучения и определяет необходимость использования данной группы методов. Для формирования устойчивых внутренних представлений в рамках метода предлагается комплексный подход к оптимизации.

Центральным компонентом данной группы является *State-Aware Adversarial Training (SA-AT)* реализующий минимаксную оптимизацию из подраздела 2.1.1 и конкретизирующий вид штрафного функционала Φ . Исходя из анализа рекуррентной природы Mamba в подразделе 2.1.2, Φ задаётся в (2.13) как регуляризатор дрейфа скрытых состояний h_t :

$$\Phi(f_{\theta}, x_i, \delta) = L_{reg} = \|h_t(x_i) - h_t(x_i + \delta)\|_2^2, \delta \in S \quad (2.13)$$

Тогда расширенная функция потерь из подраздела 2.1.1 принимает вид (2.14):

$$L_{total} = L_{CE}(f_{\theta}(x_i + \delta), y_i) + \lambda \cdot L_{reg} \quad (2.14)$$

где L_{reg} минимизирует «дрейф» памяти при подаче входа с состязательным возмущением. Это способствует тому, чтобы модель сохраняла инвариантность скрытых векторов h_t даже при наличии шума.

Целевая задача оптимизации реализуется через минимаксную процедуру: на внешнем контуре оператор A максимизирует ошибку L_{total} по δ для формирования D' , в то время как на внутреннем контуре модель минимизирует общую функцию потерь L_{total} по θ . Полный пайплайн метода на уровне обучения, включая внешний контур генерации

возмущений (max по δ) и внутренний контур оптимизации параметров (min по θ), представлен на рисунке 2.2.

Для повышения эффективности SA-AT применяются следующие дополнительные подходы:

Гибридный ансамблевый подход (Attack-Mix Training). Для реализации требования разнородности операторов из подраздела 2.1.2 каждый батч B_{mix} из D' формируется операторами A_{grad} (PGD) и A_{SSM} (HiSPA) с вероятностью $p = 0.5$. Дополнительно к PGD добавляется Momentum-фактор вида (2.15), который накапливает градиентную историю предыдущих итераций, делая итоговое возмущение более агрессивным и сложным для подавления:

$$\delta_{total} = Momentum(\nabla_x L_{CE} + \nabla_{h_t} L_{reg}) \quad (2.15)$$

Прогрессивное расширение области возмущений (Curriculum Training). Прогрессивное расширение области S и сложности операторов $A^{(epoch)}$ по формуле (2.16):

$$\epsilon^{(epoch)} = \epsilon_{start} + \frac{epoch}{epoch_{max}} (\epsilon_{final} - \epsilon_{start}) \quad (2.16)$$

что обеспечивает постепенное усложнение второго слагаемого целевой функции, позволяя весам модели адаптироваться к пиковым минимумам функции потерь без риска сильной деградации точности на чистых данных D .

Двухэтапная генерация возмущений (Two-stage training). Для нахождения возмущений, максимизирующих оба слагаемых L_{total} , применяется двухэтапный процесс, при котором сначала генерируется грубо оценённое возмущение через быстрый градиентный шаг (FGSM) по L_{CE} , а затем оно итеративно уточняется с учетом динамики скрытых состояний (2.17):

$$\begin{aligned} \delta_{init} &= \epsilon \cdot sign(\nabla_x L_{CE}) \\ \delta_{adv} &= \Pi_S (\delta_{init} + \alpha \cdot \nabla_{\delta} L_{reg}(h_t(x), h_t(x + \delta_{init}))) \end{aligned} \quad (2.17)$$

Это позволяет находить более успешные состязательные примеры D' , заставляя модель обучаться даже на тех возмущениях, которые глубоко проникают в скрытую память SSM-архитектур.

Адаптивный метод повышения устойчивости — этап обучения

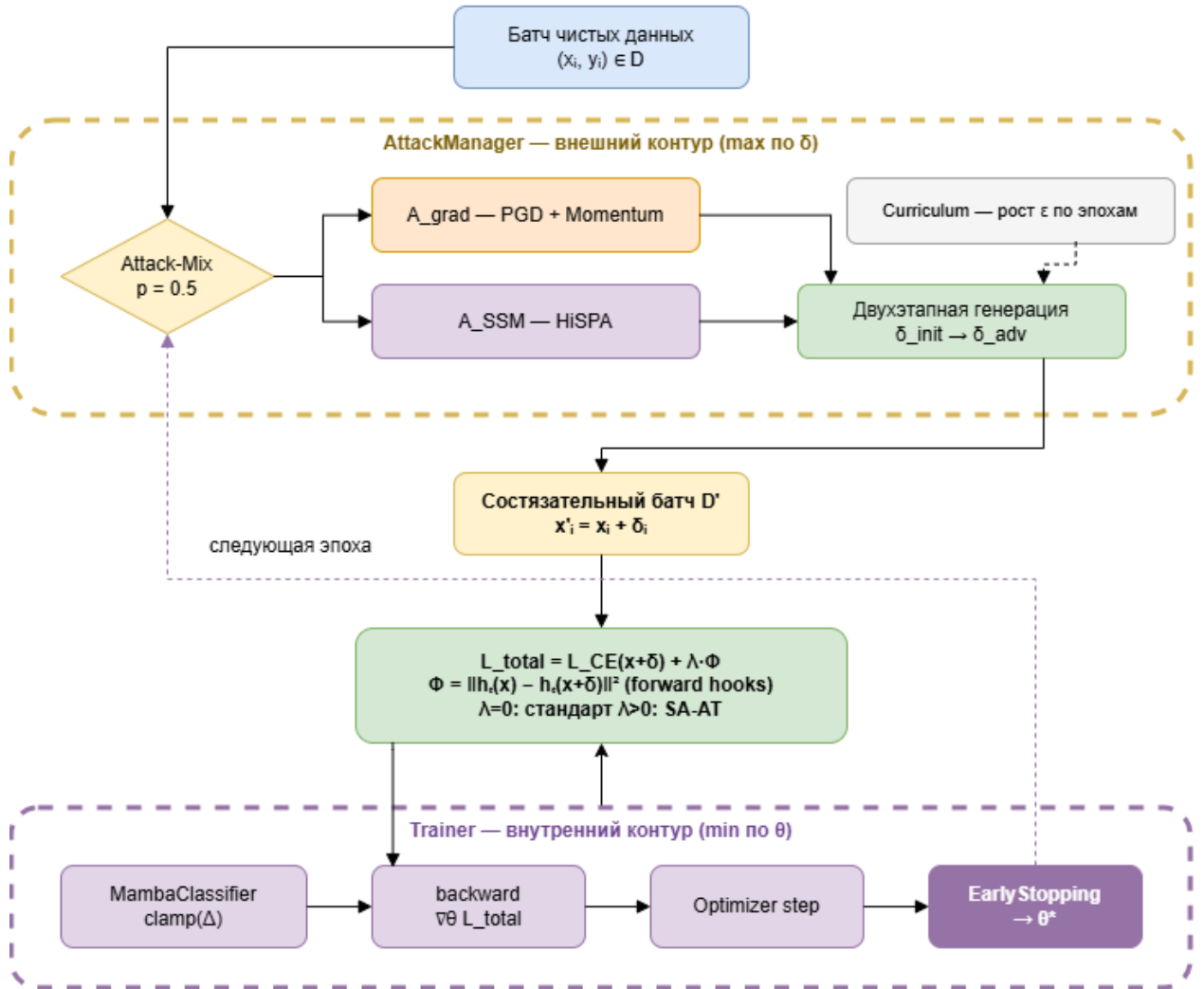


Рисунок 2.2 — Пайплайн и поток данных адаптивного метода повышения устойчивости на уровне обучения (SA-AT)

2.2.2 Метод повышения устойчивости на уровне предобработки и архитектуры модели

Метод данной группы дополняет минимаксную оптимизацию из подраздела 2.1.1 и применяется при любом λ , снижая эффективность оператора A при вычислении $\nabla_{\delta} L_{total}$ непосредственно в момент работы модели. При $\lambda = 0$ данная группа является основным источником устойчивости модели совместно с выбором оператора A .

Входное стохастическое сглаживание (Input Randomization). При инференсе к эмбедингам входной последовательности добавляется контролируемый уровень случайного шума: $f_{\theta}(x + \eta), \eta \sim \mathcal{N}(0, \sigma^2)$, который эффективно «размывает» градиенты

$\nabla_x L_{CE}$, используемые A_{grad} и препятствует успешному воздействию атаки, делая систему более устойчивой к градиентным методам.

Механизм адаптивного Gating-фильтра. В качестве защитного слоя внедряется «пре-фильтр» на основе 1D-свертки (2.18), который сглаживает скачки амплитуды в данных, подавляя локальные всплески амплитуды, созданные алгоритмами типа Sliding Patch:

$$x_{filt} = Conv1D(x; \omega) \quad (2.18)$$

где ω — сверточный фильтр, снижающий эффективность максимизации Φ через сегментные возмущения, на которых базируется A_{patch} .

Стабилизация параметра дискретизации Δ . Поскольку Δ определяет, сколько информации из текущего входа пропускается в скрытое состояние, к параметрам, генерирующим Δ , применяется ограничение пространства параметров дискретизации (2.19):

$$\Delta' = clamp(\Delta, \Delta_{min}, \Delta_{max}) \quad (2.19)$$

Это исключает резкие скачки в механизме селективного сканирования, делая процесс более предсказуемым. При $\lambda > 0$ данный механизм дополнительно снижает чувствительность Φ к малым изменениям входа, усиливая эффект регуляризатора скрытых состояний из подраздела 2.2.1. Общая последовательность применения защитных механизмов на этапе инференса представлена на рисунке 2.3.

Предложенный комплекс мер обеспечивает инвариантность f_θ на двух уровнях: через принудительную стабилизацию внутренней динамики h_t при обучении и через снижение чувствительности к аномальным входным данным при инференсе, что обеспечивает комплексную защиту Mamba от градиентных и структурных атак при сохранении эффективности реализации.

Методы повышения устойчивости — этап инференса

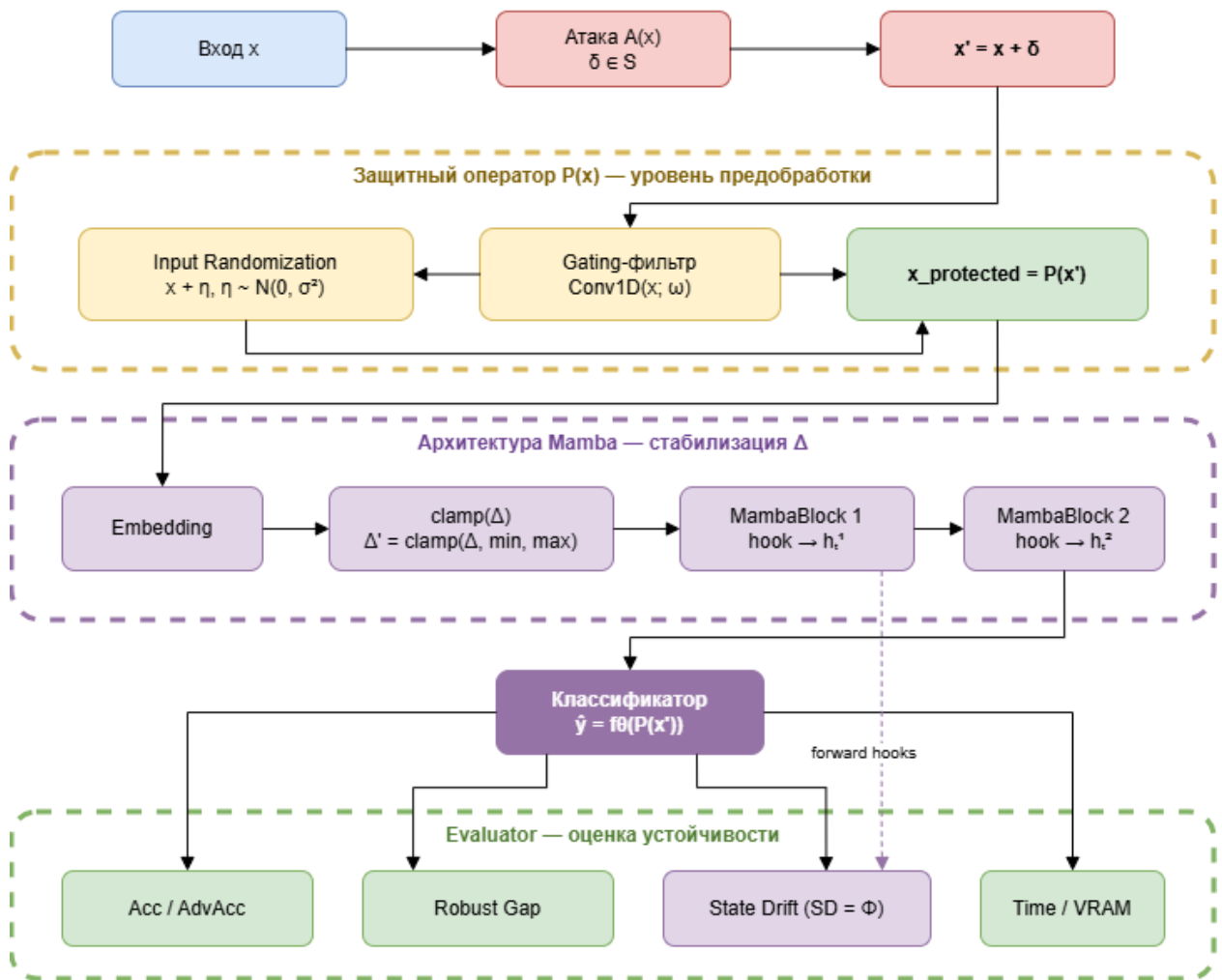


Рисунок 2.3 — Пайплайн и поток данных методов повышения устойчивости на уровне предобработки и архитектуры модели

2.3. Выбор метрик оценки устойчивости и качества модели

Подраздел посвящен обоснованию выбора метрик для количественной оценки эффективности разработанного метода. Рассматриваются как стандартные метрики качества классификации и эффективности, так и специализированные показатели, позволяющие оценить влияние состязательных возмущений на работу модели на основе Mamba.

Для удобства введём несколько новых обозначений:

- $f(x)$ — исходная модель, возвращающая предсказанный класс.
- $f_{adv}(x)$ — модель, обученная с применением методов повышения устойчивости (состязательного обучения).

2.3.1 Формализованное описание метрик

Для проведения количественного анализа и оценки эффективности разработанного метода введем формальные определения используемых метрик.

Метрики качества (Performance) — такие метрики позволяют оценить базовую предсказательную способность моделей на чистых (невозмущенных) данных D .

- Стандартная точность (Standard Accuracy) (2.20): Отражает долю корректных предсказаний на чистых данных предсказаний моделей на чистых данных x_i для исходной модели f :

$$Acc(f) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(f(x_i) = y_i) \quad (2.20)$$

где $\mathbb{I}(\cdot)$ — индикаторная функция, равная 1, если условие истинно, и 0 в противном случае. Метрика рассчитывается как для исходной модели $Acc(f)$, так и для состязательно обученной модели $Acc(f_{adv})$. Важным критерием успешности состязательного обучения является минимизация падения базового качества: $Acc(f) \approx Acc(f_{adv})$.

- F1-Score (2.21): Гармоническое среднее между точностью (Precision) и полнотой (Recall), используемое для учета возможного дисбаланса классов:

$$F1(f) = \frac{2 \cdot Precision(f) \cdot Recall(f)}{Precision(f) + Recall(f)} \quad (2.21)$$

Метрики состязательной устойчивости (Robustness) — данные метрики позволят оценить способность Mamba и других моделей противостоять направленным состязательным атакам.

- Состязательная точность (Adversarial Accuracy) (2.22): Точность работы модели f на данных x' , подвергнутых состязательным возмущениям.

Для состязательно обученной модели f_{adv} генерация возмущения происходит с учетом её собственных градиентов, поэтому её состязательный пример обозначается как $x'_{i,adv}$

$$\begin{aligned} AdvAcc(f, A) &= \frac{1}{N} \sum_{i=1}^N \mathbb{I}(f(A(x_i, f, y_i)) = y_i) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(f(x'_i) = y_i) \\ AdvAcc(f_{adv}, A) &= \frac{1}{N} \sum_{i=1}^N \mathbb{I}(f_{adv}(A(x_i, f_{adv}, y_i)) = y_i) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(f_{adv}(x'_{i,adv}) = y_i) \end{aligned} \quad (2.22)$$

Основная цель предложенного метода — обеспечение условия $AdvAcc(f_{adv}, A) \gg AdvAcc(f, A)$.

- Робастный разрыв (Robust Gap) (2.23): Разница между точностью модели на чистых и на состязательных данных. Характеризует чувствительность конкретной архитектуры к атакам.

$$\begin{aligned} Gap(f) &= Acc(f) - AdvAcc(f, A) \\ Gap(f_{adv}) &= Acc(f_{adv}) - AdvAcc(f_{adv}, A) \end{aligned} \quad (2.23)$$

Меньшее значение $Gap(f_{adv})$ свидетельствует о более высокой стабильности модели.

2.3.2 Специфические метрики архитектуры Mamba и эффективности

Специфика пространства состояний (SSM) в архитектуре Mamba накладывает необходимость отслеживания внутренних параметров сети.

Дрейф состояния (State Drift, SD) — показывает среднеквадратичное отклонение непрерывного скрытого состояния h_t модели Mamba под воздействием состязательного

шума по сравнению с чистым сигналом. Пусть $h_t(x_i)$ — скрытое состояние на шаге t при подаче чистого объекта, а $h_t(x'_i)$ — скрытое состояние при подаче атакованного. Тогда для последовательности длины T метрика дрейфа для модели f определяется как (2.24):

$$SD(f) = \frac{1}{N \cdot T} \sum_{i=1}^N \sum_{t=1}^T \|h_t(x_i) - h_t(x_i^{adv})\|_2^2 \quad (2.24)$$

Аналогично рассчитывается показатель $SD(f_{adv})$ для защищенной модели на основе её скрытых состояний и её состязательных примеров $x'_{i,adv}$. Сравнение этих показателей позволяет математически доказать, что состязательное обучение стабилизирует внутренние переменные состояния Mamba, препятствуя накоплению ошибки вдоль длинных последовательностей. Метрика SD непосредственно измеряет величину штрафного функционала $\Phi(f_\theta, x_i, \delta)$ введённого в подразделе 2.1.1, что позволяет количественно оценить эффективность его минимизации в процессе обучения.

В разработанном программном комплексе сбор скрытых траекторий $h_t(x_i)$ и $h_t(x'_i)$ реализован с помощью динамических перехватчиков сессий (forward hooks). Поскольку архитектура Mamba состоит из каскада повторяющихся блоков, итоговое значение SD вычисляется путем усреднения квадратов отклонений на выходах первых двух блоков пространства состояний (SSM). Процедура извлечения скрытых представлений и алгоритм вычисления метрики представлены на рисунке 2.4, где схематично отражен процесс параллельного прохождения чистого и состязательного сигналов через модель.

Выбор именно начальных слоев SSM в качестве опорных точек обоснован необходимостью зафиксировать формирование и локализацию начального состязательного дрейфа. Оценка скрытых переменных на ранних этапах обработки последовательности позволяет наглядно показать, как именно состязательное обучение f_{adv} подавляет амплитуду возмущения в латентном пространстве, не позволяя шуму лавинообразно накапливаться и распространяться на последующие слои и финальный классификатор.

Метрики вычислительной эффективности (Efficiency Metrics) (2.25) — используются для сравнения Mamba модели с трансформерами при сохранении робастности фиксируются аппаратные показатели инференса в условиях ограниченных ресурсов (среда Kaggle):

$$\begin{aligned}
 \text{Inference Time} &= \Delta \tau \text{ (сек/объект)} \\
 \text{Peak VRAM} &= \max(\text{Mem}_{\text{allocated}}) \text{ (Мб)}
 \end{aligned}
 \tag{2.25}$$

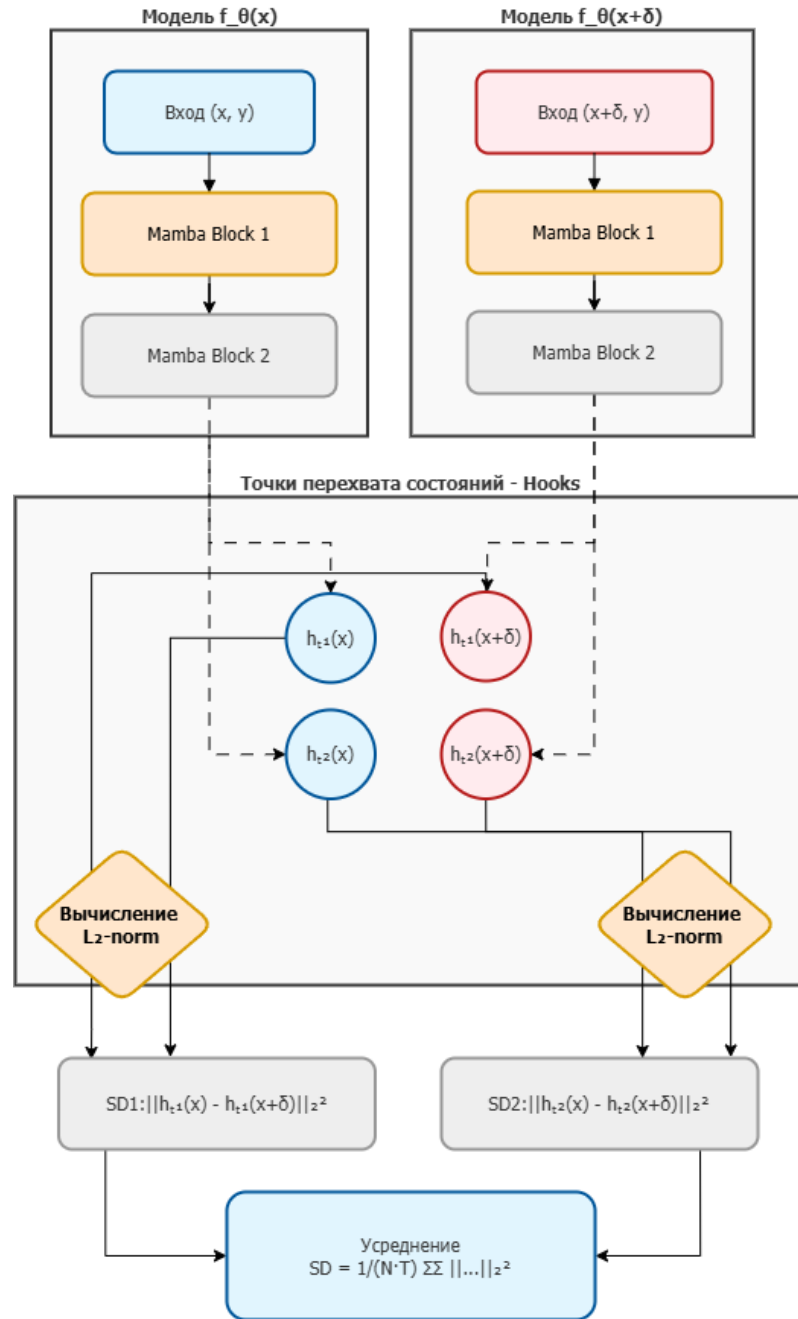


Рисунок 2.4 — Механизм оценки дрейфа скрытых состояний с использованием метода forward hooks на выходах первых двух блоков SSM

2.4. Выводы

По итогам моделирования в разделе были сформированы теоретические основы и разработан метод повышения состязательной устойчивости моделей на базе архитектуры Mamba. Метод основан на генерации возмущений алгоритмами PGD и HiSPA непосредственно в процессе обучения, что позволяет сформировать у модели устойчивые латентные представления и снизить её чувствительность к малым изменениям входных данных.

Для проведения последующей экспериментальной оценки качества классификации выбраны метрики стандартной и состязательной точности (Accuracy), взвешенная оценка F1-Score, а также показатель робастного разрыва (Robust Gap). Вычислительная эффективность и аппаратные затраты оцениваются через время инференса (Inference Time) и пиковое потребление видеопамати (Peak VRAM).

Математически обоснован и программно реализован метод оценки дрейфа скрытых состояний (State Drift, SD). Как было отмечено в подразделе 2.3, использование динамических перехватчиков (forward hooks) на выходах первых двух блоков SSM позволяет зафиксировать зарождение состязательного дрейфа на ранних этапах обработки последовательности. Такой подход позволяет наглядно доказать, что состязательное обучение эффективно подавляет амплитуду возмущений внутри модели, препятствуя их лавинообразному накоплению вдоль длинных контекстов.

Разработанная методология и программный комплекс измерительных модулей позволяют перейти к этапу проведения вычислительных экспериментов для сравнительного анализа робастности моделей Mamba и классических архитектур, таких как Transformer и CNN.

3. Проектирование и программная реализация фреймворка для обучения устойчивых моделей на основе Mamba

Раздел посвящен проектированию и программной реализации фреймворка, позволяющего обучать устойчивые к состязательным атакам Mamba-модели. В рамках раздела формулируются требования к разрабатываемому фреймворку, обосновывается выбор методологии проектирования и инструментальных средств, описывается архитектура системы и состав основных компонентов. Рассматривается стратегия разработки, основанная на принципах модульности и расширяемости, что позволяет адаптировать фреймворк под различные задачи и типы атак. Особое внимание уделяется интеграции с существующими библиотеками глубокого обучения, а также обеспечению вычислительной эффективности при работе с длинными последовательностями. Разрабатываемый фреймворк поддерживает гибкую настройку параметров состязательного обучения, включая выбор типа атаки, параметры регуляризации и метрики оценки. В результате проектирования формируется структура программного модуля, которая затем реализуется и тестируется.

3.1. Формулирование требований к фреймворку

Для практической реализации предложенной методики состязательного обучения и анализа робастности моделей семейства Mamba был разработан специализированный экспериментальный фреймворк. Его основными задачами являются сквозная автоматизация процессов токенизации исходных данных, обучение нейросетевых моделей при настраиваемых параметрах, генерация различных видов состязательных атак, вычисление метрик робастности и комплексного сравнительного анализа трех классов архитектур: моделей пространства состояний (Mamba/SSM), моделей на базе механизмов внимания (Transformer) и свёрточных нейросетей (CNN), а также визуализация результатов. Система реализована как самостоятельный программный блок с модульной структурой, предоставляющей унифицированный интерфейс для проведения сравнительных экспериментов.

Ниже приведен перечень функциональных и нефункциональных требований, сформированных для обеспечения гибкости разработанного фреймворка, его стабильности и вычислительной эффективности.

Функциональные требования:

- Система должна поддерживать различные архитектуры моделей, уметь работать с моделями этих типов архитектур в рамках единого пайплайна через общий интерфейс.
- Фреймворк должен предоставлять возможность генерации специализированного шума, искажающего исходные данные и последующего построения состязательного примера для проверки устойчивости классификаторов.
- Система должна поддерживать несколько режимов состязательного обучения, для тренировки модели как на чистых, так и на атакованных данных, для влияния на устойчивость.
- Требуется наличие механизмов для автоматизированного сбора статистики и сравнения метрик (точности, уровня ошибок) для базовых и защищенных версий моделей.
- Пользователь должен иметь возможность через конфигурационные параметры задавать настройки наборов данных и моделей, гиперпараметры обучения, типы атак и уникальные параметры конкретных видов атакующих воздействий (силу, количество шагов и др.)

Нефункциональные требования:

- Система должна быть оптимизирована для работы с длинными последовательностями, минимизируя потребление оперативной и видеопамяти, чтобы обеспечивать достаточную производительность и эффективность.
- Архитектура должна состоять из независимых блоков (загрузка данных, модели, атаки, обучение), что позволит легко заменять или масштабировать отдельные части системы.
- Фреймворк должен быть расширяемым: предусматривать простое добавление новых типов архитектур или алгоритмов атак без существенных изменений в основном программном ядре.

3.2. Описание архитектуры и функционала системы

Исходя из сформулированных требований, функциональная архитектура фреймворка состоит из следующих, взаимодействующих друг с другом, абстрактных модулей, каждый из которых решает свою изолированную задачу. Общая структура представлена на рисунке 3.1.

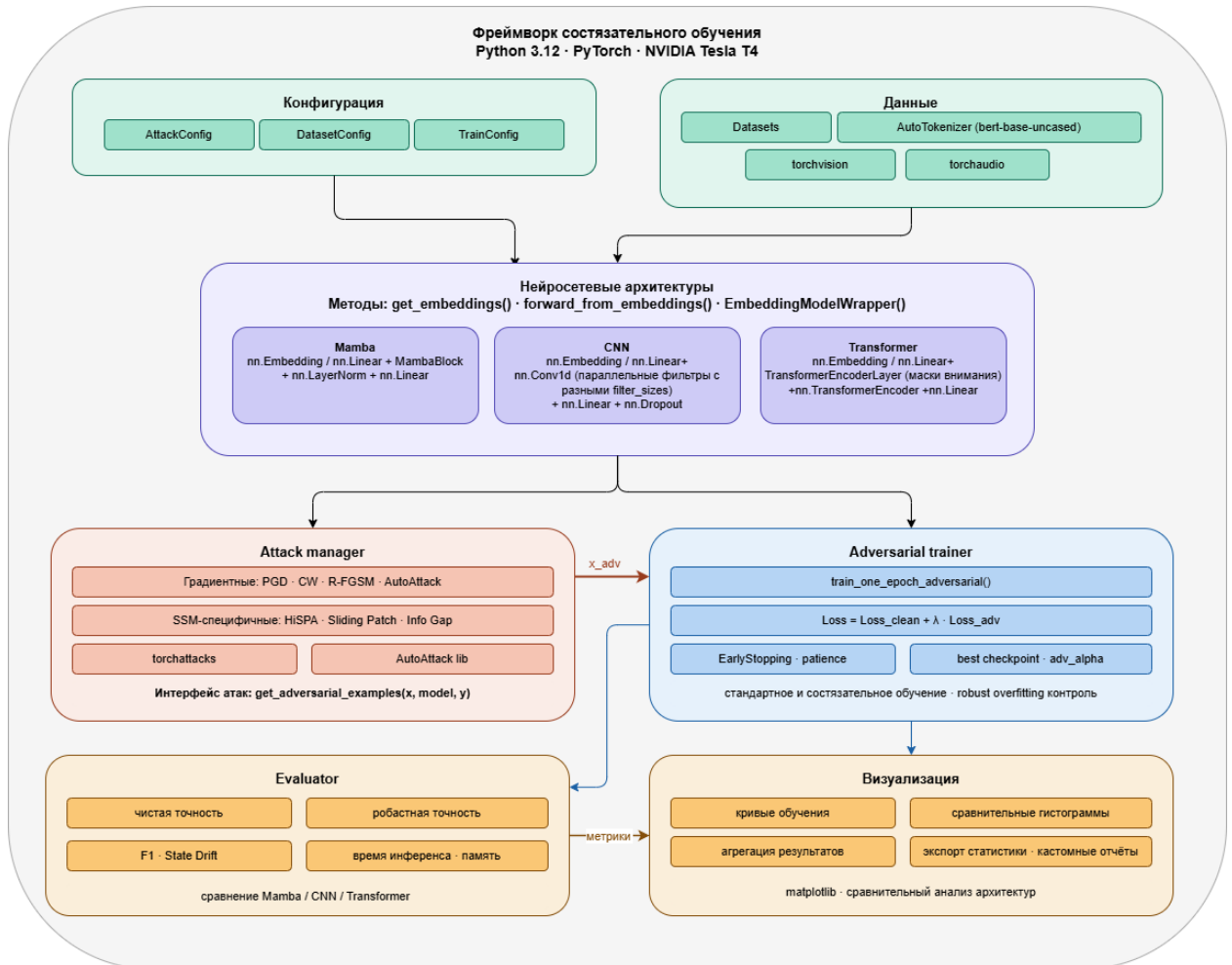


Рисунок 3.1 — Диаграмма модулей фреймворка

1. Блок конфигурации и управления данными: Отвечает за инициализацию параметров системы и подготовку данных для обучения. Модуль принимает сырые данные (текст, изображения, аудио), проводит их стандартизацию или нормализацию, а также токенизирует при необходимости (для текстовых данных), и формирует пакеты (батчи) для последующей передачи в нейронную сеть. Блок абстрагирует процесс получения обучающих и тестовых выборок.
2. Блок нейросетевых архитектур: Представляет собой набор унифицированных контейнеров для моделей. Для свёрточных, трансформерных и моделей базирующихся на пространствах состояний, независимо от их внутренней

структуры, блок предоставляет стандартизированные методы для получения эмбедингов, извлечения признаков и получения финальных предсказаний классификации.

3. Блок генерации состязательных атак (Attack Manager): Специализированный модуль, встроенный в пайплайн, который отвечает за синтез целенаправленных возмущений, искажающих входные данные. Блок поддерживает использование как градиентных алгоритмов (PGD, FGSM), так и специфических эвристических методов (генерация локальных патчей, внедрение информационных разрывов). Алгоритмы атак применяются к эталонным данным (или эмбедингам) для генерации состязательных примеров.
4. Блок обучения (Trainer): Ядро системы, управляющее жизненным циклом модели. Модуль инкапсулирует логику оптимизации параметров и поддерживает два ключевых сценария:

Стандартное обучение, при котором выполняется оптимизация весов модели на основе чистых тренировочных данных и состязательное обучение (Adversarial Training). При этом процессе на каждой итерации генерируются атакованные примеры. Функция потерь вычисляется комбинированным образом, учитывая ошибки как на чистых, так и на возмущенных данных. Блок также включает механизмы ранней остановки (Early Stopping) для предотвращения переобучения.

5. Блок оценки и агрегации метрик (Evaluator): Отвечает за прогон тестовых выборок через обученные модели. Собирает статистику по метрикам, агрегирует результаты различных экспериментов и формирует структуры данных для визуализации кривых обучения и сравнительных гистограмм. Отвечает за тестирование моделей на валидационных выборках. Блок координирует работу с генератором атак для вычисления метрик робастности (чистая и состязательная точность, F1-меры). Включает в себя подсистему профилирования для замера вычислительной эффективности (времени инференса) и потребления памяти. Для архитектур с внутренними состояниями реализует логику синхронного перехвата данных для вычисления метрик стабильности (State Drift).
6. Блок визуализации и анализа результатов: Блок, отвечающий за сбор статистики по метрикам, а также агрегацию результатов различных экспериментов и формирование структуры данных для визуализации кривых обучения и сравнительных гистограмм.

3.3. Реализация программного модуля для обучения, тестирования и генерации состязательных примеров

Для реализации спроектированной архитектуры был выбран язык программирования Python (версия 3.12) в связке с фреймворком глубокого обучения PyTorch. Такой выбор обусловлен наличием динамического графа вычислений и механизмов автоматического дифференцирования, которые необходимы для расчета градиентов при генерации состязательных примеров. В качестве среды исполнения использована платформа с аппаратным ускорителем NVIDIA Tesla T4.

Для работы с данными используется библиотека Datasets для загрузки стандартных датасетов и токенизатор AutoTokenizer из библиотеки Hugging Face Transformers (модель bert-base-uncased) для стандартизированной разбивки текста. Для работы с наборами, содержащими другие типы данных, такие как изображения и аудио, были импортированы библиотеки torchvision и torchaudio соответственно.

Функциональный модуль генерации состязательных примеров реализован как высокоуровневый менеджер (AttackManager), инкапсулирующий работу с библиотеками torchattacks и AutoAttack, которые предоставляют оптимизированные реализации градиентных атак (PGD, FGSM, CW и других) на тензорном уровне. Дополнительно было реализовано несколько кастомных алгоритмов (Sliding patch, HiSPA и др). Внедрение состязательных атак внутри фреймворка построено на интеграции готовых инструментов в единый конвейер PyTorch.

Для обеспечения совместимости библиотек с пользовательскими моделями (MambaClassifier, CNNClassifier, TransformerClassifier) фреймворк использует Паттерн «Адаптер» и принцип Embedding-ориентированности. Поскольку градиентный спуск невозможен на дискретных входных токенах, система автоматически оборачивает модели в EmbeddingModelWrapper, а значит любая модель, подаваемая в AttackManager, должна реализовать метод forward, который возвращает логиты (или вероятности) для заданного входа. Это позволяет вычислять градиенты функции потерь непосредственно в непрерывном латентном пространстве эмбедингов, что делает процесс дифференцируемым и устойчивым.

Для соблюдения архитектурной целостности и возможности проведения сравнительных экспериментов, была создана следующая программная реализация компонентов фреймворка, представленная на Рисунке 3.2.

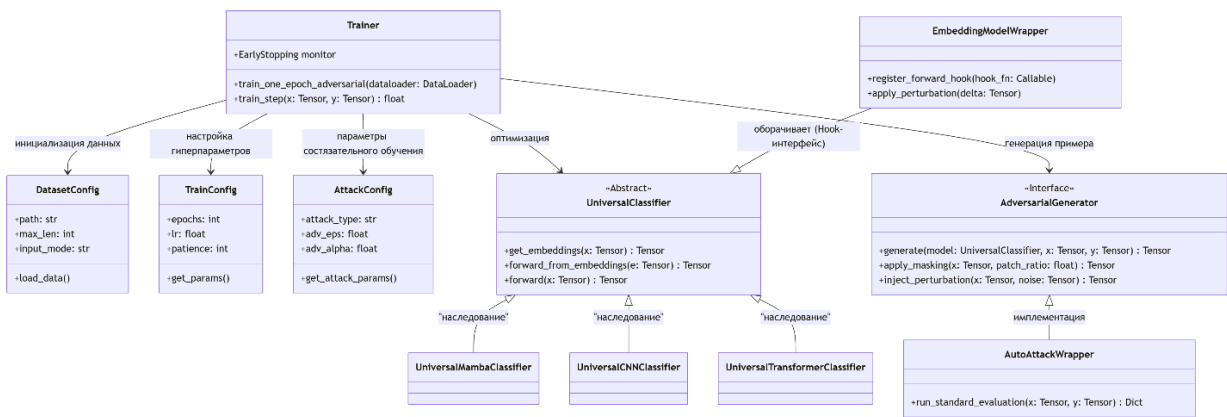


Рисунок 3.2 — Структура классов фреймворка

Далее более детально описывается структура кода и назначение каждого из представленных компонентов

3.3.1 Управление конфигурацией через Data Classes

Управление параметрами реализовано с помощью конфигурационных структур (Data Classes). Использование декоратора `@dataclass` позволяет формализовать эксперименты и избежать ошибок, связанных с динамической типизацией. Для удобства работы с параметрами были созданы классы `DatasetConfig`, `TrainConfig` и `AttackConfig`. Они обеспечивают строгую типизацию гиперпараметров (размер батча, количество слоев, `patience` для ранней остановки, тип и сила атаки `adv_eps` и другие) для соответствующего модуля системы:

- `DatasetConfig`: Инкапсулирует специфику данных. Здесь задаются путь к датасету, максимальная длина последовательности (`max_len`), размер батча, а также критически важный параметр `input_mode` (дискретные токены или непрерывный сигнал).
- `TrainConfig`: Хранит гиперпараметры процесса оптимизации: количество эпох, скорость обучения (`learning_rate`), параметры планировщика весов и порог `patience` для алгоритма ранней остановки.
- `AttackConfig`: Определяет условия проведения состязательного теста: тип атаки (например, PGD, CW), максимальная величина возмущения ϵ , размер шага α , количество итераций и коэффициент регуляризации `adv_alpha`, определяющий баланс между точностью на чистых и атакованных данных.

3.3.2 Унифицированные нейросетевые архитектуры

В фреймворке представлены три универсальных модели-классификаторов, наследуемых от `nn.Module`. Все модели поддерживают стандартный жизненный цикл PyTorch. Главная особенность реализации заключается в разделении логики обработки входных данных и классификационного «головы» (`head`):

1. `UniversalMambaClassifier` — основан на блоках `MambaBlock` из библиотеки `mamba-ssm`. Модель динамически конфигурирует размерность скрытого пространства и количество слоев селективного сканирования. Для работы с непрерывными данными модель использует линейную проекцию входного сигнала в пространство эмбедингов, а для токенов — стандартный слой `nn.Embedding`.
2. `UniversalCNNClassifier` — реализует архитектуру на основе одномерных свёрток (`nn.Conv1d`). Используется масштабный подход с параллельными свёрточными слоями с фильтрами разных размеров (`filter_sizes`), которые позволяют захватывать паттерны разной локальной протяженности. Выход свёрток нормализуется через глобальный `MaxPool1d`, обеспечивая инвариантность признаков к их позиции в последовательности.
3. `UniversalTransformerClassifier` — базируется на стеке `nn.TransformerEncoderLayer`. Модель автоматически генерирует маски внимания, если входная последовательность содержит `padding`, что делает её устойчивой к переменной длине данных.

3.3.3 Интерфейс взаимодействия с латентным пространством и поддержка режимов работы

Для того чтобы атаки могли воздействовать непосредственно на внутренние представления моделей, минуя слой эмбедингов, все классификаторы реализуют унифицированный интерфейс:

- `get_embeddings(x)`: Преобразует входной тензор (токены или непрерывный сигнал) в векторное представление. Это позволяет сохранить модель на этапе преобразования входа в эмбединги.
- `forward_from_embeddings(embeddings)`: Принимает уже возмущенный тензор эмбедингов $e' = e + \delta$ и пропускает его через оставшуюся часть сети (блоки Mamba, свёртки или слои внимания) до финального классификатора.

Выбор режима (`input_mode`) в конфигурации глобально определяет способ инициализации входного слоя. В режиме «Токены» модель ожидает тензоры целых чисел и применяет слой `nn.Embedding`. В режиме «Непрерывные данные» (например, аудио или сигналы) модель ожидает тензоры вещественных чисел (`float`) и применяет слой линейной проекции (`nn.Linear`) для приведения размерности данных к размерности скрытого пространства модели.

Такая архитектура позволяет проводить «атаки на латентное пространство», где алгоритм возмущения вычисляет градиенты не по дискретным индексам слов (что математически некорректно), а по непрерывным векторам эмбеддингов, обеспечивая корректное применение градиентных методов типа PGD или CW ко всем архитектурам без исключения.

3.3.4 Программная реализация модуля генерации состязательных атак

В роли централизованного менеджера атак выступает функция `get_adversarial_examples`: Доступный инструментарий модуля включает градиентные методы PGD, CW, R-FGSM, а также ансамблевый AutoAttack для оценки границ робастности.

Метод PGD реализован через итеративное обновление входного тензора x с проекцией в ϵ – окрестности. К исходному входу добавляется случайный шум в пределах ϵ - шара, затем на каждом шаге вычисляется градиент функции потерь (Cross Entropy) по отношению к входному тензору $g = \nabla_x L(f(x), y)$. Затем делается «шаг» в сторону градиента $x_{new} = x + \alpha \cdot \text{sign}(g)$. Полученное значение принудительно обрезается (`clip`) в область $x \pm \epsilon$, чтобы искажение оставалось незаметным для человека. Метод R-FGSM предварительно добавляет к входу случайный шум из равномерного распределения, что позволяет избежать попадания в локальные минимумы ландшафта функции потерь при выполнении единственного градиентного шага. Атака CW использует оптимизацию функции потерь с применением $\tanh$ - преобразования для ограничений диапазона значений. Вместо Cross Entropy такая функция потерь минимизирует доверие модели к правильному классу, не допуская «избыточного» возмущения. Реализация через `torchattacks` эффективно работает с ограничениями через смену переменных. AutoAttack представляет собой комбинацию методов, которая часто используется как тест на максимальную робастность. Она автоматически подбирает гиперпараметры и последовательно запускает 4 различных атаки (APGD-CE, APGD-DLR, FAB, Square).

В коде представлены механизмы обеспечения вычислительной стабильности: принудительная активация `torch.enable_grad()` в режиме инференса, отсоединение состоятельных тензоров от вычислительного графа через `.detach()` для предотвращения утечек памяти и автоматическая нормализация входных данных в соответствии со стандартами библиотек. Фреймворк учитывает, что многие атаки требуют нормализации входных данных. Внутри функции `get_adversarial_examples` происходит автоматическое приведение тензоров к формату, ожидаемому библиотекой `torchattacks` (например, учет `mean` и `std` для нормализации эмбеддингов).

Помимо стандартных методов из `torchattacks` (PGD, CW и др.), система расширена набором узкоспециализированных кастомных алгоритмов:

- **HiSPA (Hidden State Poisoning Attack):** В основе работы атаки лежит механизм Forward Hooks, в коде представленный через метод `model.register_forward_hook()`. При инициализации атаки в `EmbeddingModelWrapper` регистрируется хук, который перехватывает тензор скрытых состояний h_t на нужном слое SSM. В методе `forward` модели данные проходят до целевого слоя, где h_t сохраняется в глобальный буфер. В процессе выполнения атаки вычисляется градиент лосс-функции непосредственно по отношению к тензору h_t : `grad = torch.autograd.grad(loss, hidden_state)`. Полученный градиент используется для обновления входных эмбеддингов x через цепочку вызовов `backward()`.
- **Sliding Patch :** Алгоритм реализован как параметрическая процедура маскирования, работающая поверх тензоров в `AttackManager`. Метод `apply_sliding_patch` принимает входной тензор x и коэффициент `patch_ratio` (0.35–0.45). Затем вычисляется размер окна: `window_size = int(seq_len * patch_ratio)` и в цикле итераций атаки генерируется случайный индекс начала блока `start_idx`. Создается бинарная маска `mask`, где значения внутри `[start_idx : start_idx + window_size]` равны 1, а остальные 0. Возмущение применяется только к выбранному фрагменту: `x_adv = x + mask * (noise_tensor)`.
- **Info Gap:** Выполнен как модуль принудительной подстановки значений, имитирующий забывание или блокировку входа. В коде реализован класс-обертка `InfoGapAttack`, который работает со срезами входного тензора. Функция `inject_gap` находит случайный или фиксированный сегмент последовательности и выполняет операцию замещения: `x[:, start:end] = fill_value`, где `fill_value` задается как константа (например, 0, среднее значение датасета или шум).

Данная реализация превращает процесс состязательного воздействия в прозрачный интерфейс «черного ящика», где управляющий менеджер избавляет пользователя от необходимости ручной адаптации методов под внутреннюю структуру каждой модели.

3.3.5 Модуль обучения и оптимизации нейросетевых моделей

Ядром фреймворка является алгоритм состязательного обучения, реализуемый функцией `train_one_epoch_adversarial`. В отличие от классического обучения, где модель оптимизируется только на эталонных данных, данный модуль на каждой итерации выполняет процедуру двухэтапной оптимизации для достижения робастности.

Механизм состязательного обучения. На первом этапе для каждого обучающего батча формируется «атакованная» версия данных. С использованием градиентных методов (таких как PGD) вычисляются возмущения для эмбеддингов, которые максимизируют ошибку модели. Для обеспечения возможности дифференцирования атак на дискретные текстовые данные используется обертка `EmbeddingModelWrapper`. Она перехватывает процесс прохода данных через слой эмбеддингов, что позволяет применить атаку к непрерывному векторному представлению, а не к целочисленным идентификаторам токенов. Итоговая функция потерь вычисляется как композиция. Это позволяет выявить «слабые места» в текущем представлении признаков модели.

На втором этапе модель обучается на объединенном батче (оригинальные примеры + состязательные возмущения). Итоговая функция потерь рассчитывается как композиция (3.1):

$$Loss_{total} = Loss_{clean} + \lambda \cdot Loss_{adv} \quad (3.1)$$

где $Loss_{clean}$ — ошибка на чистых данных, $Loss_{adv}$ — ошибка на атакованных эмбеддингах, а λ (`adv_alpha`) — гиперпараметр регуляризации, определяющим степень «защищенности» модели.

Такой подход вынуждает нейросеть адаптировать веса не только для снижения эмпирического риска, но и для подавления влияния градиентных возмущений, что препятствует эффекту «градиентной маскировки» и повышает общую устойчивость архитектуры к атакам.

Контроль жизненного цикла обучения. Стабильность процесса обучения поддерживается классом `EarlyStopping`, который предотвращает состязательное переобучение (`robust overfitting` — ситуацию, когда точность на атакованных данных растет, но способность модели к обобщению на новых данных падает). Модуль выполняет мониторинг валидационных метрик, то есть после каждой эпохи вычисляются метрики точности на независимой контрольной выборке (`val_acc`). Далее фиксируется прогресс с учетом порога `min_delta` и при достижении лимита `patience` процесс обучения принудительно прерывается. По достижении условия остановки система автоматически восстанавливает параметры модели, соответствующие точке наилучшего баланса между чистой точностью и робастностью, зафиксированной на валидационном этапе.

Данная архитектура модуля гарантирует воспроизводимость экспериментов и позволяет объективно сравнивать пределы робастности различных архитектур — от свёрточных сетей (CNN) до моделей с селективным сканированием (Mamba) — без риска деградации параметров из-за чрезмерной адаптации к конкретным типам состязательных атак.

Полученное программное решение построено модульным способом, что обеспечивает независимость отдельных компонентов. Функциональная схема взаимодействия компонентов системы представлена на рисунке 3.3. Как видно из схемы, `Adversarial Trainer` выступает в роли связующего звена, координирующего передачу данных между генератором атак и моделью.

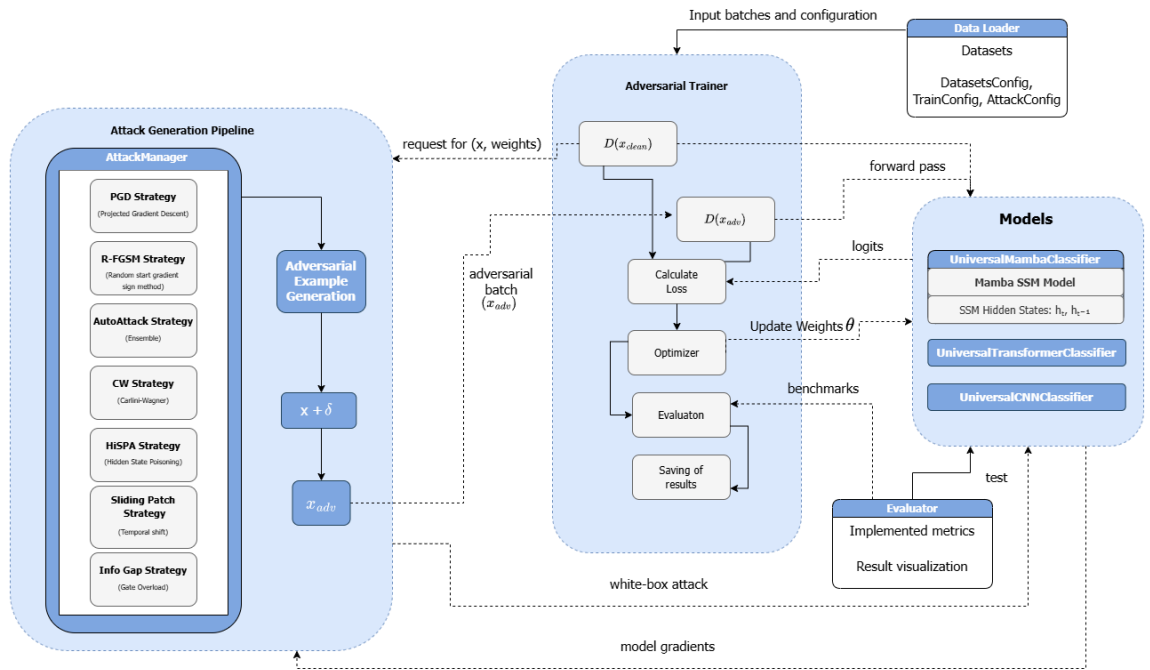


Рисунок 3.3 — Функциональная схема взаимодействия модулей разработанного фреймворка для обучения устойчивых Матба-моделей

3.4. Тестирование разработанного программного модуля

Перед проведением вычислительных экспериментов была проведена функциональная верификация разработанного фреймворка. Далее описываются детали тестирования, которое подтвердило корректность работы всех компонентов системы, включая механизмы загрузки данных, генерации атак и обучения.

Для обеспечения кросс-доменного тестирования в коде фреймворка реализована поддержка следующих наборов данных:

- MNIST: Стандартный набор рукописных цифр (классификация изображений).
- sMNIST (Sequential MNIST): Последовательная версия MNIST для проверки способности модели запоминать длинные временные зависимости.
- CIFAR-10: Набор цветных изображений разных объектов.
- IMDB: Большой набор данных для классификации тональности текстовых рецензий.
- AG News: Набор для классификации новостных заголовков.
- Speech Commands: Датасет для классификации коротких аудиокоманд.

Ниже приведена таблица 3.1 сценариев тестирования, подтверждающая работоспособность основных функциональных блоков.

Таблица 3.1 — Сценарии тестирования основных функциональных блоков

Модуль	Объект тестирования	Ожидаемый результат
Config Manager	Инициализация Data Classes	Параметры (batch size, learning rate, eps) корректно передаются в модули без потери типов.
Model Adapter	EmbeddingModelWrapper	Градиенты успешно проходят через слой эмбедингов; модель возвращает предсказания при подаче векторов.
Attack Manager	Генерация атак (PGD, HiSPA и др.)	Метод get_adversarial_examples возвращает тензор возмущенных эмбедингов, размерность совпадает с входной.
Training Engine	train_one_epoch_adversarial	Значение функции потерь уменьшается при стандартном обучении и стабилизируется при состязательном.
Hook/State Module	Перехват латентных состояний	Внутренние тензоры слоев (Mamba/SSM) успешно копируются в память CPU, хуки удаляются после итерации.
Early Stopping	Логика остановки обучения	Обучение прекращается при отсутствии улучшения метрик на валидации в течение patience эпох.
Memory Manager	Стабильность VRAM	Отсутствие роста потребления памяти после выполнения всех циклов (отсутствие утечек).

Описание выполненных тестов:

1. Проверка целостности графа (Gradient Flow): Для тестирования EmbeddingModelWrapper подавался случайный шум в качестве эмбедингов. Результатом теста стало успешное выполнение обратного прохода (loss.backward())

без ошибок, связанных с отсутствием связей в графе вычислений. Подтверждается, что интерфейс атак корректно интегрирован в модель.

2. Тестирование жизненного цикла объектов (Hook Isolation): Проводилась проверка состояния `model._forward_hooks`. После завершения цикла оценки всех моделей (Mamba, CNN, Transformer) проводился принудительный дамп ключей словаря хуков. В Результате список хуков оказался пустым, что подтверждает корректную деструкцию объектов и отсутствие риска засорения памяти в долгосрочных экспериментах.
3. Стресс-тест памяти (OOM Prevention): В ходе теста на mnist и imdb датасетах был запущен процесс с намеренно завышенным размером батча, чтобы вызвать переполнение. Модуль Memory Manager продемонстрировал корректное перехватывание состояния: промежуточные тензоры успешно сбрасывались на CPU, что позволило завершить инференс без вылета системы с ошибкой CUDA out of memory.
4. Валидация логики ранней остановки: Был искусственно занижен параметр patience до 1 и min_delta до критически большого значения. Система зафиксировала остановку обучения ровно на следующей эпохе, что подтвердило верное срабатывание триггера EarlyStopping при текущей логике мониторинга метрик.

Для повышения надежности системы в раздел функционального тестирования были добавлены дополнительные сценарии, направленные на проверку граничных условий и устойчивости к некорректным входным данным, указанные в таблице 3.2.

Таблица 3.2 — Дополнительные сценарии тестирования

Модуль	Объект тестирования	Ожидаемый результат
Input Validator	Обработка последовательностей разной длины	Система корректно применяет маскирование или обрезку до max_len без ошибок размерности.
Device Manager	Перенос данных (CPU/GPU)	Автоматическое определение доступности CUDA и корректное размещение тензоров на устройстве.
Attack Scalability	Итеративное изменение ϵ	Атаки корректно отрабатывают при значениях возмущения,

		близких к 0 и к верхнему пределу (1.0).
Data Integrity	Валидация диапазонов данных	Проверка, что нормализованные данные лежат в ожидаемом диапазоне (например, [0, 1] или [-1, 1]).
Checkpoints	Сериализация/Десериализация весов	Полное совпадение весов модели после сохранения на диск и последующей загрузки.

Описание дополнительных процедур верификации:

1. Тест устойчивости к длине последовательности (Sequence Length Robustness): Был проведен эксперимент с подачей последовательностей, длина которых отклоняется от установленного `max_len`. Фреймворк успешно отработал процедуру паддинга (дополнения) для коротких последовательностей и корректную обрезку для длинных. Это подтверждает, что архитектурные блоки (особенно `UniversalTransformerClassifier` и `UniversalMambaClassifier`) динамически адаптируются к размерностям входных данных.
2. Тестирование межплатформенной совместимости: Проводился запуск цикла обучения в двух конфигурациях: принудительно на CPU и принудительно на GPU. Система автоматически распознала отсутствие аппаратного ускорителя в первом случае и переключила логику `EmbeddingModelWrapper` на использование системной памяти, что обеспечивает переносимость кода между различными средами разработки.
3. Верификация целостности состояний (Checkpoint Integrity Test): В ходе обучения вызывалась процедура сохранения чекпоинта (`save_checkpoint`) и последующей загрузки весов в «чистый» экземпляр модели. Сравнение тензоров весов до и после операции методом `torch.equal()` показало полное совпадение параметров, что гарантирует надежность восстановления прерванных экспериментов.
4. Анализ граничных значений (Boundary Condition Test): При генерации атак PGD проверялась реакция системы на экстремальные значения коэффициента ϵ . При $\epsilon \rightarrow 0$ влияние атаки снижается до идентичности входным данным, а при $\epsilon \rightarrow 1$ вызывает максимальное искажение тензоров, при этом модель сохраняла стабильность вычислений без возникновения NaN значений в логитах.

классификатора. Это подтверждает математическую устойчивость алгоритма проекции в выбранном пространстве ограничений.

Таким образом, проведенное функциональное тестирование подтвердило, что все компоненты фреймворка — от конфигурационных структур до механизмов генерации состязательных возмущений — работают в строгом соответствии с заложенными архитектурными принципами, обеспечивая стабильную и воспроизводимую среду для проведения вычислительных экспериментов. Расширенный набор тестов охватывает не только критические пути выполнения, но и потенциальные точки отказа, связанные с обработкой данных и управлением состоянием модели, что гарантирует отказоустойчивость фреймворка.

3.5. Выводы

В подразделе подводятся итоги проектирования и программной реализации фреймворка. Формулируются основные результаты, включая сформулированные требования, обоснованный выбор программных средств, разработанную архитектуру и результаты тестирования. Констатируется, что разработанный фреймворк соответствует поставленным требованиям, обеспечивает корректную работу с Mamba, реализацию предложенного метода состязательного обучения и гибкую настройку параметров.

В рамках данного раздела была проведена комплексная работа по проектированию и программной реализации экспериментального фреймворка, предназначенного для исследования устойчивости современных нейросетевых архитектур к состязательным атакам. По итогам работы можно констатировать успешное выполнение всех поставленных задач и соответствие системы заданным критериям.

На этапе формирования концепции были определены функциональные и нефункциональные требования, которые легли в основу модульной структуры фреймворка, ориентированного на возможность подключения любых типов нейросетей и высокую стабильность работы. Разработанная архитектура использует паттерн «Адаптер» для бесшовной интеграции разнородных классификаторов, включая модели на базе архитектур Mamba, CNN и Transformer, а использование конфигурационных структур на основе @dataclass обеспечило строгую типизацию гиперпараметров и высокую воспроизводимость экспериментов.

Выбор программного стека, позволил эффективно реализовать механизмы дифференцируемых градиентных атак в латентном пространстве эмбедингов. Важным этапом стало внедрение интерфейса `EmbeddingModelWrapper`, который устранил проблему невозможности дифференцирования дискретных входных данных. Кроме того, функционал фреймворка был расширен за счет интеграции как стандартных библиотек состязательных атак (`torchattacks`, `AutoAttack`), так и специализированных алгоритмов: `HiSPA` для анализа SSM-архитектур, `Sliding Patch` для проверки локальной контекстной устойчивости и `Info Gap` для оценки информационных провалов. Реализованный модуль обучения с функцией `train_one_epoch_adversarial` и механизмом `EarlyStopping` обеспечил надежный контур для состязательной тренировки моделей, позволяя находить оптимальный баланс между точностью классификации на чистых данных и робастностью к преднамеренным искажениям. Функциональная верификация, проведенная на разнообразных по наполнению датасетах (`MNIST`, `CIFAR-10`, `IMDB`, `AG News` и др.), подтвердила работоспособность всех подсистем, включая управление памятью, жизненным циклом градиентных хуков и корректную обработку граничных условий.

Таким образом, разработанный фреймворк в полной мере соответствует поставленным требованиям. Она представляет собой надежный, отказоустойчивый и гибкий инструмент, который не только обеспечивает поддержку современных моделей класса `Mamba`, но и предоставляет исследователю готовую среду для автоматизированного анализа уязвимостей глубоких нейронных сетей, тем самым полностью решая задачи, поставленные в рамках проектирования.

4. Экспериментальное исследование метода повышения устойчивости моделей на основе Mamba

Раздел посвящен экспериментальной оценке эффективности разработанного метода повышения устойчивости Mamba-моделей к состязательным атакам. В рамках раздела описываются используемые для проведения экспериментов наборы данных и обосновывается их выбор для данной задачи. Рассматривается процедура генерации состязательных примеров для обучения и тестирования, включая выбор параметров атак и обоснование принятых решений. Проводится исследование поведения базовой архитектуры Mamba под воздействием состязательных атак, анализируется зависимость устойчивости от длины последовательности и влияние возмущений на скрытые состояния модели. Выполняется количественная и качественная оценка разработанных методов, включая сравнение с существующими подходами по стандартным метрикам точности, состязательной точности и специализированным показателям устойчивости. Также оценивается вычислительная эффективность предложенного метода. По результатам экспериментов формулируются выводы об эффективности разработанного подхода и его применимости в практических задачах.

4.1. Описание используемых наборов данных

Для проведения полноценного тестирования и подтверждения универсальности разработанного метода состязательного обучения был сформирован набор целевых датасетов, охватывающий ключевые типы задач машинного обучения: классификацию изображений, анализ аудиосигналов и обработку текстовой информации. Характеристики наборов данных приведены в таблице 4.1.

Таблица 4.1 — Основные характеристики используемых наборов

Набор данных	Тип данных	Размерность (Т)	Количество классов	Особенности для эксперимента
MNIST	Изображения (flat)	784	10	Тестирование на простых аддитивных возмущениях.
sMNIST	Изображения (seq)	784	10	Тестирование накопления ошибки в SSM на длинных зависимостях

CIFAR-10	Изображения (patch)	64 / 256	10	Оценка устойчивости к структурным шумам.
Speech Commands	Аудиосигналы	16 000	35	Чувствительность к временным искажениям и сдвигам.
IMDB	Текст	256	2	Оценка семантической устойчивости эмбедингов.
AG News	Текст	128	4	Робастность на длинных и разнородных контекстах.

Описание и обоснование выбора:

- **MNIST и sMNIST (Sequential MNIST):** Служат базовым индикатором способности модели к обработке одномерных сигналов. Стандартный MNIST обрабатывается в плоском режиме: изображение размером 28×28 пикселей разворачивается в вектор длиной 784 и подаётся в модель как одноканальная непрерывная последовательность. Таким образом проверяется устойчивость к локализованному шуму. В режиме sMNIST каждый пиксель обрабатывается как отдельный шаг последовательности, что вынуждает модель удерживать контекст на протяжении 784 шагов, что подходит для тестирования HiSPA на накопление ошибок в механизмах селективного сканирования. Оба датасета реализованы через `torchvision.datasets.MNIST` с нормализацией; для sMNIST дополнительно применяется преобразование `Lambda`, формирующее тензор формы (784, 1).
- **CIFAR-10:** При представлении изображений в виде последовательности патчей данный датасет позволяет сопоставить Mamba с архитектурой Vision Transformer (ViT). Изображения нормализуются с использованием стандартных статистик CIFAR-10 (среднее (0.4914, 0.4822, 0.4465), стандартное отклонение (0.2023, 0.1994, 0.2010)). Выбор датасета продиктован необходимостью оценить, насколько SSM-механизмы запоминают глобальную структуру изображения в условиях состязательного воздействия.

- **Speech Commands (v2):** Датасет включает записи коротких голосовых команд длительностью 1 секунда (16 000 отсчётов при частоте дискретизации 16 кГц). В реализованном классе `SpeechDataset` сигнал приводится к фиксированной длине. Итоговая форма тензора — (16000, 1), что соответствует одноканальной последовательности большой длины. Высокая плотность данных и короткая длительность позволяют исследовать, как именно модель «теряет» контекст при применении атак типа `Sliding Patch`, так как любая потеря информации в аудиосигнале ведет к неверной идентификации команды.
- **IMDB и AG News:** Текстовые датасеты применяются для оценки устойчивости модели при обработке естественного языка. В обоих случаях используется токенизатор `bert-base-uncased` с фиксированной длиной последовательности: 256 токенов для IMDB (длинные рецензии) и 128 для AG News (короткие новостные заголовки). Тексты, превышающие заданную длину, усекаются; более короткие дополняются `padding`-токенами. Данная пара датасетов позволяет оценить эффективность атаки `Info Gap` атак, которая блокируют поток значимых токенов. Выбор данных датасетов позволяет оценить, как модель компенсирует «информационные стены» при обработке контекстов разной длины.

Описанный набор экспериментальных данных позволяет подтвердить, что предложенные методы состязательного обучения обеспечивают повышение устойчивости не только для конкретной архитектуры, но и на уровне обработки последовательностей различной природы. Это исключает риск специализации (переобучения) метода под статистические свойства одного конкретного типа данных и обосновывает широкую применимость разработанного решения.

4.2. Генерация состязательных примеров для обучения модели

В данном подразделе описывается процедура генерации состязательных примеров, используемых как в процессе обучения (для повышения робастности), так и на этапе финального тестирования. Реализованный алгоритм базируется на минимаксной оптимизации: внешний контур максимизирует функцию потерь по пространству возмущений $\delta \in S$, внутренний контур минимизирует суммарную функцию L_{total} , включающую штраф за дрейф скрытых состояний L_{reg} .

4.2.1 Конфигурация параметров атак

Все параметры атак фиксируются в объекте `AttackConfig`, включающем следующие поля: тип атаки (`attack_type`), допустимую норму возмущения `adv_eps` (ϵ), шаг оптимизации `adv_lr`, количество итераций `adv_steps` (N_{steps}) для итеративных методов, весовой коэффициент состязательного вклада в функцию потерь `adv_alpha` (α) и относительный размер патча `patch_size`. В таблице 4.2 приведена полная спецификация параметров, используемых в экспериментальном цикле.

Таблица 4.2 — Полная конфигурация параметров состязательных атак

Тип атаки	ϵ (<code>adv_eps</code>)	<code>patch_size</code>	N_{steps}	<code>adv_lr</code>	<code>adv_alpha</code>	Двухэтапная генерация (two-stage)	Описание
PGD	8/255	-	10	1e-2	0.5	Да (для Mamba)	Базовая проверка робастности методом итеративного проецированного спуска
R-FGSM	8/255	-	1	1e-2	0.5	Нет	Быстрая атака с рандомным стартом; проверка чувствительности и градиентов

CW	(оптимизирует)	-	100	1e-2	0.5	Нет	Оптимизационная атака (L2); поиск минимально необходимого возмущения
AutoAttack	8/255	-	20	1e-2	0.5	Нет	Ансамбль APGD-CE, APGD-T, FAB, SquareAttack; исключает эффект маскировки градиентов
HiSPA	8/255	-	10	1e-2	0.5	Да	Дестабилизация скрытых состояний SSM через градиент по h_t
Sliding Patch	-	0.35	1	-	0.5	Нет	Проверка устойчивости Gating-фильтра
Info Gap	-	0.45	1	-	0.5	Нет	Атака на «информационную пустоту»

Выбор $\epsilon = 8/255$ для градиентных атак соответствует стандартному порогу по оценке робастности моделей компьютерного зрения и обработки последовательностей [24]. Для атаки CW фиксированное ϵ не применяется, поскольку алгоритм самостоятельно минимизирует L2-норму возмущения при заданном параметре уверенности $\kappa = 0$ и коэффициенте $c = 1$, выполняя 100 итераций оптимизации с шагом $lr = 0.01$. Параметр `patch_size` для Sliding Patch и Info Gap задаёт относительный размер воздействуемого сегмента последовательности: 0.35 (35%) и 0.45 (45%) соответственно, что соответствует теоретическому обоснованию из подраздела 2.1.

4.2.2 Двухэтапная генерация возмущений

Для режимов, требующих двухэтапной генерации, параметр `two_stage` активирует дополнительный проход оптимизации с использованием FGSM-атаки для скрытых представлений. Для текстовых данных, обрабатываемых в режиме `input_type = "tokens"`, атака выполняется не в пространстве входных идентификаторов (которые являются дискретными), а в пространстве непрерывных эмбеддингов. В функции `generate_adv_delta` реализован следующий алгоритм:

1. На первом этапе вычисляются чистые эмбеддинги через `model.get_embeddings(x)`, после чего применяется выбранный алгоритм атаки к полученному непрерывному представлению с использованием обёртки `EmbeddingModelWrapper`.
2. На втором этапе вычисляется дельта как разность между состязательными и чистыми эмбеддингами: $\delta = \text{emb_adv} - \text{emb_clean}$, которая используется в качестве возмущения при обучении.

Для непрерывных входных данных (`input_type = "continuous"`) атака применяется непосредственно к входному тензору, после чего дельта вычисляется в пространстве эмбеддингов через `model.get_embeddings`. Такой подход унифицирует процедуру генерации возмущений для всех типов данных и соответствует двухэтапной схеме, описанной в подразделе 2.2.

Использование данных параметров позволяет стандартизировать процесс состязательного обучения и тестирования и обеспечить сопоставимость результатов при оценке устойчивости Mamba, Transformer и CNN архитектур.

4.2.3 Реализация стратегий состязательного обучения

Для противодействия состязательным атакам реализован комплекс защитных механизмов, описанные в подразделе 2.2.2, в виде единого настраиваемого обучающего цикла. Механизмы представлены на трёх уровнях:

Уровень предобработки: Добавлены входное стохастическое сглаживание и адаптивный Gating-фильтр (1D-свертка), предназначенный для подавления всплесков, созданных алгоритмами типа Sliding Patch.

Уровень архитектуры: Стабилизация параметра дискретизации Δ через метод ограничения весов (clipping), что делает процесс сканирования данных более консервативным.

Уровень обучения:

- Curriculum Adversarial Training: На первых двух эпохах используется только атака PGD как наиболее предсказуемая для стабилизации начального обучения. Начиная с третьей эпохи тип атаки выбирается случайным образом из набора {PGD, HiSPA, Sliding Patch}.
- Attack-Mix Training: Случайная ротация атак на поздних эпохах ($\text{epoch} > 2$) реализует гибридную ансамблевую стратегию и препятствует специализации (переобучению) модели под конкретный тип возмущения.
- State-Aware Adversarial Training (SA-AT): Для моделей типа Mamba, поддерживающих метод `forward_from_embeddings` с параметром `return_hidden`, в цикле вычисляются чистые скрытые состояния `clean_hidden` и состязательные `adv_hidden`. Суммарная функция потерь (4.1) включает три слагаемых:

$$L_{total} = L_{CE}(f(x), y) + \alpha \cdot L_{CE}(f(x + \delta), y) + \lambda \cdot MSE(h_t(x + \delta), h_t) \quad (4.1)$$

где $\lambda = 0.5$ — коэффициент регуляризации дрейфа скрытых состояний. Для базовых архитектур (CNN, Transformer), не имеющих доступа к скрытым состояниям, регуляризационный член отсутствует, и функция потерь принимает стандартный вид смеси чистых и состязательных потерь.

Эффективность разработанных методов защиты оценивается на основе системы показателей, подробно описанной в подразделе 2.3.

4.2.4 Границы применимости и требования к системе

- Ограничения вычислительной среды и размерности: Метод ограничен применимостью к моделям f_θ , функционирующим в условиях ограниченного контекстного окна $T \leq T_{max}$ и ограниченного латентного пространства $h_t \in \mathbb{R}^d$. Данное ограничение обусловлено ограниченными ресурсами среды и тем, что при $T \rightarrow \infty$ и неограниченном d механизмы селективного сканирования Δ_t могут компенсировать состязательный дрейф за счет избыточной размерности, снижая чистоту эксперимента.
- Критерий сохранения качества (4.2): Допустимое снижение точности на исходном датасете D (clean accuracy) должно удовлетворять следующему неравенству относительно базовой модели:

$$Acc(f_{\theta_{adv}}) \geq Acc(f_{\theta}) - \tau_{acc}, \quad \tau_{acc} \in [0.03, 0.05] \quad (4.2)$$

- Требования к робастности (4.3): Модель $f_{\theta_{adv}}$ должна демонстрировать улучшение состязательной точности по сравнению с базовой моделью для одного или более оператора атаки $A \in \{A_{grad}, A_{SSM}\}$ при заданном радиусе возмущения $\epsilon \geq 8/255$:

$$\min_A AdvAcc(f_{\theta_{adv}}, A) > \min_A AdvAcc(f_{\theta}, A), \quad \forall \delta \in S \quad (4.3)$$

Таким образом, задача сводится к модификации процесса обучения и, возможно, введению дополнительных регуляризаторов в механизм селективного сканирования препятствующих дестабилизации скрытого состояния при малых изменениях входа.

4.3. Схема экспериментального исследования и методология тестирования

В подразделе описывается процесс обучения моделей как в базовом режиме (без применения защитных механизмов), так и с применением разработанных методов повышения устойчивости. Рассматривается проектирование и обоснование экспериментального цикла: определение конфигураций обучения, архитектурных параметров моделей и наборов атак для каждого датасета. Проводится тестирование на чистых данных и на данных с добавлением состязательных возмущений. Оценивается изменение показателей качества классификации под воздействием атак; для архитектуры Mamba дополнительно анализируется расхождение скрытых состояний на невозмущённых и атакованных входах. Результаты экспериментов, полученные по данной схеме, приводятся в подразделе 4.4.

4.3.1 Матрица конфигураций обучения

Для доказательства эффективности предложенного метода и выявления вклада каждого его компонента необходимо сопоставить модели в нескольких режимах обучения. Это позволяет разграничить влияние входной защиты (предобработка и фильтрация) и обучающей защиты (SA-AT, Attack-Mix, двухэтапная генерация возмущений) на итоговую устойчивость. В таблице 4.3 представлены четыре конфигурации, образующие схему экспериментального цикла: от полного отсутствия защиты (Baseline) до совместного применения всех механизмов (Full Defense).

Таблица 4.3 — Конфигурация режимов обучения модели

Название режима	SA-AT	Attack-Mix + Curriculum	Input Noise (Inference)	Gating-фильтр (1D Conv)	Стабилизация Δ (Clipping)	Двухэтапная генерация
Baseline (Standard)	Нет	Нет	Нет	Нет	Нет	Нет
Input Defense	Нет	Нет	Да	Да	Да	Нет
Training Defense	Да	Да	Нет	Нет	Нет	Да

Full Defense	Да	Да	Да	Да	Да	Да
--------------	----	----	----	----	----	----

Режим Input Defense изолирует эффект механизмов предобработки (стохастическое сглаживание входа, Gating-фильтр и стабилизации параметра Δ) при полном отсутствии состязательного обучения, что позволяет оценить их самостоятельный вклад в робастность. Режим Training Defense, напротив, активирует SA-AT, Attack-Mix и двухэтапную генерацию, оставляя инференс-механизмы отключёнными. Сравнение этих двух промежуточных режимов с Full Defense позволяет количественно разграничить защиту на уровне обучения и защиту на уровне инференса, выявив их результирующий эффект при совместном применении.

4.3.2 Сводная конфигурация архитектурных настроек

Каждая из трёх исследуемых архитектур — Mamba, Transformer и CNN — обучается во всех четырёх конфигурациях, образуя в совокупности двенадцать экспериментальных вариантов. Такой дизайн обеспечивает корректное межархитектурное сравнение: гиперпараметры обучения (lr, dropout, patience) одинаковы для всех архитектур, тогда как архитектурно-специфичные параметры каждой модели сохраняются неизменными. В таблице 4.4 приведены полные настройки для каждой конфигурации.

Таблица 4.4 — Мастер-план архитектурных настроек и параметров обучения

Модель / Режим	Архитектурные параметры	Конфигурация параметров обучения	Настройки защиты
MAMBA			
R1: Baseline	d_model=128, n_layers=3	epochs=7, lr=1e-3, dropout=0.1, patience=2, min_delta=1e-3, val_split=0.2	Gating=False, Noise=0.0, SA-AT=0.0, 2-Stage=False
R2: Pre-proc			Gating=True, Noise=0.05, SA-AT=0.0, 2-Stage=False
R3: Training			Gating=False, Noise=0.0, SA-AT=0.5, 2-Stage=True
R4: Full			Gating=True, Noise=0.05, SA-AT=0.5, 2-Stage=True
TRANSFORMER			

R1: Baseline	d_model=128, n_layers=3, nhead=8, dim_feedforward=512	epochs=7, lr=1e-3, dropout=0.1, patience=2, min_delta=1e-3, val_split=0.2	Gating=False, Noise=0.0, SA-AT=0.0, 2-Stage=False
R2: Pre-proc			Gating=True, Noise=0.05, SA-AT=0.0, 2-Stage=False
R3: Training			Gating=False, Noise=0.0, SA-AT=0.5, 2-Stage=True
R4: Full			Gating=True, Noise=0.05, SA-AT=0.5, 2-Stage=True
CNN			
R1: Baseline	n_filters=100, filter_sizes=(3,4,5)	epochs=7, lr=1e-3, dropout=0.1, patience=2, min_delta=1e-3, val_split=0.2	Gating=False, Noise=0.0, SA-AT=0.0, 2-Stage=False
R2: Pre-proc			Gating=True, Noise=0.05, SA-AT=0.0, 2-Stage=False
R3: Training			Gating=False, Noise=0.0, SA-AT=0.5, 2-Stage=True
R4: Full			Gating=True, Noise=0.05, SA-AT=0.5, 2-Stage=True

Важно отметить, что регуляризатор дрейфа скрытых состояний L_{reg} активен только для архитектуры Mamba (при SA-AT=0.5), поскольку именно эта архитектура обладает явным рекуррентным скрытым состоянием h_t . Для Transformer и CNN при SA-AT=0.5 функция потерь включает только смесь чистых и состязательных потерь без члена регуляризации состояний, что корректно отражает отсутствие SSM-специфичной уязвимости в этих архитектурах и позволяет методологически чисто их сравнивать.

4.3.3 Методология проведения тестов

Для каждого набора данных определён индивидуальный набор атак при тестировании, обоснованный в подразделе 4.2. Такой подход позволяет сосредоточить вычислительные ресурсы на атаках, репрезентативных для конкретного типа данных, и избежать формального применения методов, не имеющих интерпретируемого смысла в данном домене. В таблице 4.5 представлена полная матрица стратегий обучения и тестирования, определяющая взаимосвязь между конфигурацией обучения (TrainConfig) и набором оценочных атак (AttackConfig).

Таблица 4.5 — Матрица стратегий обучения и тестирования по датасетам

Датасет	Режим	Атаки при Обучении (Train)	Атаки при Тестировании (Eval)
MNIST	Baseline	— (только CE)	PGD, R-FGSM, HiSPA
	Input Defense	— (только CE)	PGD, R-FGSM, HiSPA
	Training Defense	PGD, HiSPA, R-FGSM	PGD, R-FGSM, HiSPA
	Full Defense	PGD, HiSPA, Sliding Patch, R-FGSM	PGD, R-FGSM, HiSPA
SMNIST	Baseline	— (только CE)	PGD, AutoAttack, HiSPA, Sliding Patch
	Input Defense	— (только CE)	PGD, AutoAttack, HiSPA, Sliding Patch
	Training Defense	PGD, HiSPA, CW	PGD, AutoAttack, HiSPA, Sliding Patch
	Full Defense	PGD, HiSPA, CW, Sliding Patch	PGD, AutoAttack, HiSPA, Sliding Patch
CIFAR-10	Baseline	— (только CE)	PGD, R-FGSM, AutoAttack, CW, HiSPA
	Input Defense	— (только CE)	PGD, R-FGSM, AutoAttack, CW, HiSPA
	Training Defense	PGD, HiSPA, CW	PGD, R-FGSM, AutoAttack, CW, HiSPA
	Full Defense	PGD, HiSPA, CW, Sliding Patch	PGD, R-FGSM, AutoAttack, CW, HiSPA
IMDB / AG NEWS	Baseline	— (только CE)	PGD, R-FGSM, HiSPA, Sliding Patch, Info Gap
	Input Defense	— (только CE)	PGD, R-FGSM, HiSPA, Sliding Patch, Info Gap
	Training Defense	PGD, HiSPA, CW	PGD, R-FGSM, HiSPA, Sliding Patch, Info Gap
	Full Defense	PGD, HiSPA, CW, Sliding Patch	PGD, R-FGSM, HiSPA, Sliding Patch, Info Gap
SPEECH COM.	Baseline	— (только CE)	PGD, R-FGSM, HiSPA, Sliding Patch, Info Gap

Input Defense	— (только CE)	PGD, R-FGSM, HiSPA, Sliding Patch, Info Gap
Training Defense	PGD, HiSPA, CW	PGD, R-FGSM, HiSPA, Sliding Patch, Info Gap
Full Defense	PGD, HiSPA, CW, Sliding Patch	PGD, R-FGSM, HiSPA, Sliding Patch, Info Gap

Тестирование каждой модели проводится последовательно для каждой атаки из соответствующего набора. Это позволяет выявить специфические уязвимости архитектуры к разным типам математических искажений и отследить, как каждый режим защиты влияет на устойчивость к конкретному классу атак — градиентным (A_{grad}) или SSM-специфичным (A_{SSM}).

4.4. Результаты экспериментального исследования

Подраздел содержит результаты экспериментального цикла, полученные по схеме, описанной в подразделе 4.3. Производится количественная и качественная оценка эффективности разработанных методов. Для каждого сочетания «датасет × архитектура × режим защиты» приводятся значения стандартной точности (Acc), состязательной точности (AdvAcc) и робастного разрыва (Gap) по всем применяемым атакам, а также метрики F1. Для архитектуры Mamba дополнительно фиксируется метрика дрейфа скрытых состояний (State Drift, SD). На основе полученных данных проводится межархитектурное сравнение и проверка выполнения критериев устойчивости, сформулированных в разделе 2. Представляются графики и таблицы, иллюстрирующие полученные результаты.

4.4.1 Результаты на датасетах MNIST и sMNIST

Датасеты MNIST и sMNIST позволяют оценить устойчивость моделей в условиях простых аддитивных возмущений (MNIST) и при необходимости удерживать длинные последовательные зависимости (sMNIST, $T = 784$). Результаты испытаний на данных датасетах представлены в таблицах 4.6 и 4.7.

Таблица 4.6 — Результаты на датасете MNIST

Архитектура	Режим	Clean ACC (%)	F1 macro (%)	PGD	
				AdvAcc (%)	Gap (%)
Mamba	Baseline (R1)	98.42	98.41	3.32	95.10
Mamba	Input (R2)	98.37	98.36	66.93	31.44
Mamba	Training (R3)	98.61	98.60	97.40	1.21
Mamba	Full (R4)	98.61	98.62	97.10	1.51
Transformer	Baseline (R1)	80.09	79.78	5.77	74.32
Transformer	Input (R2)	94.36	94.52	14.65	79.71
Transformer	Training (R3)	79.23	78.80	58.57	20.66
Transformer	Full (R4)	96.10	96.12	89.61	6.49
CNN	Baseline (R1)	93.87	93.82	53.84	40.03
CNN	Input (R2)	95.96	95.90	81.63	14.33
CNN	Training (R3)	90.52	90.31	77.74	12.78
CNN	Full (R4)	96.60	96.57	92.07	4.53

Архитектура	Режим	R_FGSM		HiSPA		State Drift
		AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	
Mamba	Baseline (R1)	36.90	61.52	34.96	63.46	5.2338
Mamba	Input (R2)	76.70	21.67	82.84	15.53	5.0386
Mamba	Training (R3)	97.43	1.18	97.87	0.74	0.5679
Mamba	Full (R4)	97.11	1.50	97.59	1.02	0.6892
Transformer	Baseline (R1)	13.32	66.77	18.46	61.63	—
Transformer	Input (R2)	32.05	62.31	39.31	55.05	—
Transformer	Training (R3)	60.12	19.11	64.77	14.46	—
Transformer	Full (R4)	89.92	6.18	91.97	4.13	—
CNN	Baseline (R1)	58.79	35.08	68.93	24.94	—
CNN	Input (R2)	82.52	13.44	86.97	8.99	—
CNN	Training (R3)	78.54	11.98	81.86	8.66	—
CNN	Full (R4)	92.27	4.33	93.50	3.10	—

На MNIST наибольший эффект даёт состязательное обучение. Базовая модель Mamba сохраняет высокую чистую точность (98.42 %), но практически полностью теряет работоспособность под градиентными атаками: AdvAcc снижается до 3.32 % при PGD и до 34.96 % при HiSPA. Как следует из данных таблицы 4.6, применение Training Defense и Full Defense повышает состязательную точность Mamba до 97–98 % по всем трём атакам (PGD, R-FGSM, HiSPA). Робастный разрыв сокращается с более чем 95 пунктов (PGD, Baseline) до 1–1.5 пунктов. Метрика State Drift снижается с 5.23 до 0.57 (Training Defense) и до 0.69 (Full Defense); небольшое увеличение SD при переходе от Training к Full Defense не сказывается на итоговой точности и подтверждает стабилизацию скрытого состояния.

Transformer и CNN также выигрывают от защиты, однако уступают Mamba по итоговому уровню робастности: для Transformer Full Defense AdvAcc по PGD достигает 89.61 %, для CNN — 92.07 %, против 97.10 % у Mamba. Таким образом, на простых аддитивных возмущениях разработанный метод обеспечивает для Mamba практически идеальную устойчивость без потери качества. Динамика чистой точности по режимам представлена на рисунке 4.1, состязательной точности — на рисунке 4.2.

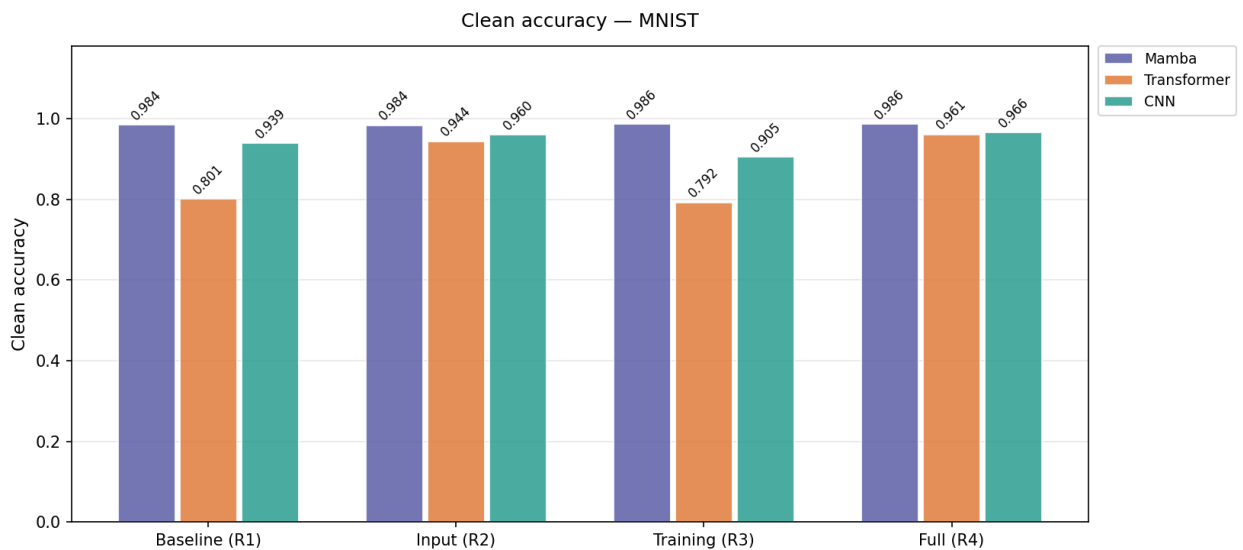


Рисунок 4.1 — Чистая точность моделей на датасете MNIST по режимам защиты

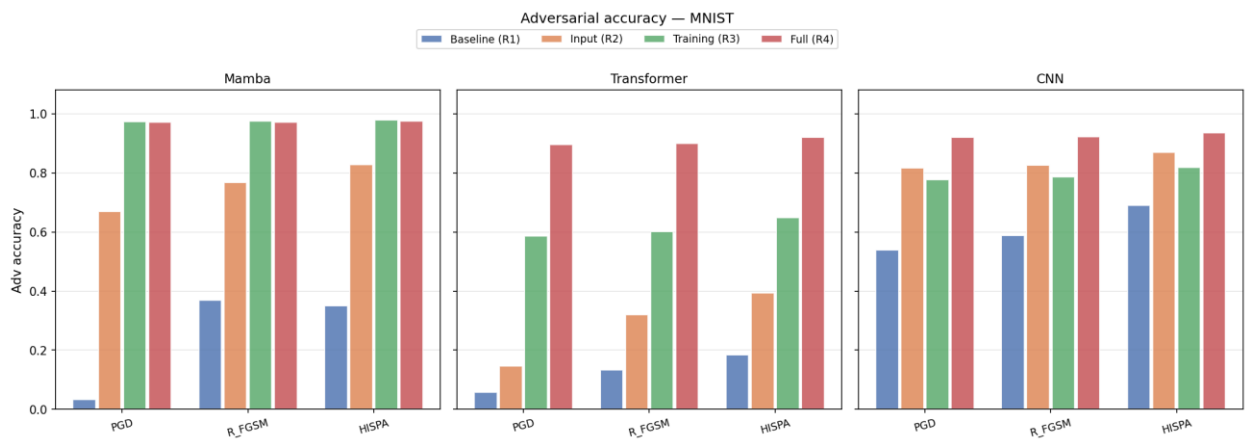


Рисунок 4.2 – Состязательная точность моделей на датасете MNIST по режимам защиты

Таблица 4.7 — Результаты на датасете sMNIST

Архитектура	Режим	Clean ACC (%)	F1 macro (%)	AUTOATTACK		PGD	
				AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)
Mamba	Baseline (R1)	97.80	97.78	64.27	33.53	64.25	33.55
Mamba	Input (R2)	94.50	94.47	42.73	51.77	42.90	51.60
Mamba	Training (R3)	97.92	97.92	95.31	2.61	95.33	2.59
Mamba	Full (R4)	93.82	93.79	92.19	1.63	92.16	1.66
Transformer	Baseline (R1)	92.61	92.54	0.20	92.41	0.19	92.42
Transformer	Input (R2)	96.98	97.04	44.75	52.23	44.83	52.15
Transformer	Training (R3)	89.44	89.16	81.43	8.01	81.27	8.17
Transformer	Full (R4)	96.79	96.79	94.44	2.35	94.48	2.31
CNN	Baseline (R1)	37.29	35.90	11.89	25.40	11.93	25.36

CNN	Input (R2)	64.98	64.84	14.58	50.40	14.64	50.34
CNN	Training (R3)	34.39	31.41	26.77	7.62	26.82	7.57
CNN	Full (R4)	63.81	62.31	50.29	13.52	50.47	13.34

Архитектура	Режим	HISPA		SLIDING_PATCH		State Drift
		AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	
Mamba	Baseline (R1)	80.38	17.42	55.71	42.09	5.4027
Mamba	Input (R2)	62.91	31.59	22.21	72.29	4.0019
Mamba	Training (R3)	94.22	3.70	87.74	10.18	0.8925
Mamba	Full (R4)	91.70	2.12	61.22	32.60	0.2856
Transformer	Baseline (R1)	5.73	86.88	29.98	62.63	—
Transformer	Input (R2)	67.70	29.28	50.80	46.18	—
Transformer	Training (R3)	78.78	10.66	46.84	42.60	—
Transformer	Full (R4)	93.64	3.15	53.97	42.82	—
CNN	Baseline (R1)	15.21	22.08	17.74	19.55	—
CNN	Input (R2)	21.60	43.38	41.53	23.45	—
CNN	Training (R3)	25.03	9.36	14.96	19.43	—
CNN	Full (R4)	47.04	16.77	39.77	24.04	—

В режиме sMNIST ($T = 784$) каждый пиксель обрабатывается как отдельный шаг последовательности, что позволяет оценить накопление ошибки вдоль рекуррентных связей. Базовая Mamba демонстрирует умеренную устойчивость (AdvAcc 64 % по PGD и AutoAttack, 55.71 % по Sliding Patch) при State Drift 5.40. Режим Training Defense поднимает состязательную точность до 95.3 % по градиентным атакам и до 87.74 % по Sliding Patch, сохраняя чистую точность на уровне 97.92 % и снижая SD до 0.89. Full Defense даёт наименьший State Drift (0.29) и высокую устойчивость к градиентным атакам (92 %), однако по Sliding Patch уступает режиму Training Defense (61.22 % против 87.74 %), что объясняется компромиссом между входным сглаживанием и сохранением структурной информации длинного контекста.

Базовый Transformer на длинной последовательности практически полностью уязвим к AutoAttack и PGD (AdvAcc около 0.2 %), и хотя защита повышает его устойчивость до 94 %, базовая CNN остаётся слабой (чистая точность лишь 37.29 %). Видны трудности свёрточной архитектуры при моделировании длинных зависимостей. Mamba в целом

сохраняет наилучший баланс качества и устойчивости. Сравнение робастного разрыва по архитектурам и режимам показано на рисунке 4.3.

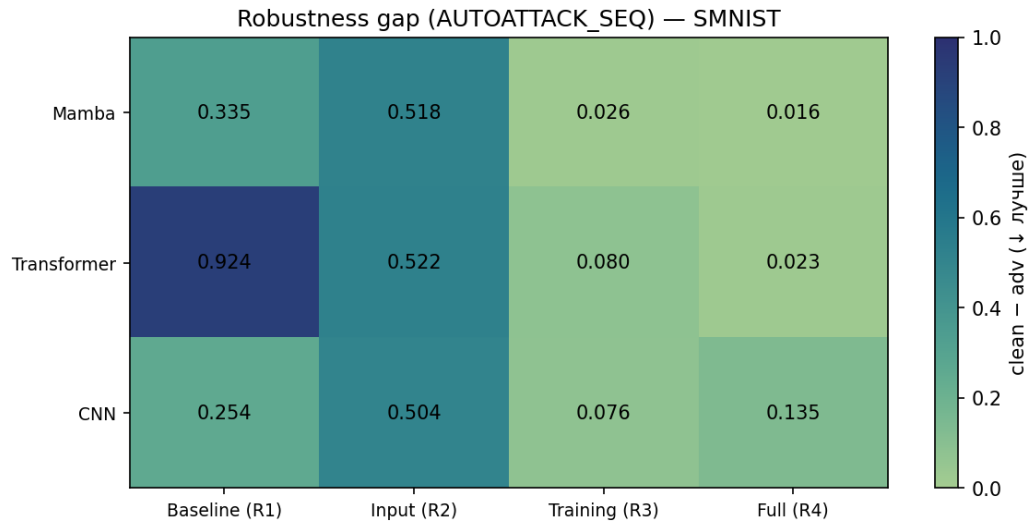


Рисунок 4.3 – Робастный разрыв (Gap) моделей на датасете SMNIST по режимам защиты

4.4.2 Результаты на CIFAR-10

Датасет CIFAR-10 в режиме патч-последовательностей позволяет сопоставить SSM-механизмы Mamba с архитектурой Transformer (ViT-like) в условиях структурных шумов. Результаты испытаний на данном датасете представлены в таблице 4.8

Таблица 4.8 — Результаты на датасете CIFAR-10

Архитектура	Режим	Clean ACC (%)	F1 macro (%)	AUTOATTACK		CW	
				AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)
Mamba	Baseline (R1)	58.93	57.92	19.32	39.61	3.32	55.61
Mamba	Input (R2)	58.83	58.90	22.68	36.15	12.26	46.57
Mamba	Training (R3)	60.49	59.99	49.88	10.61	9.08	51.41
Mamba	Full (R4)	59.65	59.11	52.27	7.38	5.79	53.86
Transformer	Baseline (R1)	58.51	57.37	18.43	40.08	3.13	55.38
Transformer	Input (R2)	61.14	60.36	37.71	23.43	14.38	46.76
Transformer	Training (R3)	56.15	55.45	45.77	10.38	11.02	45.13
Transformer	Full (R4)	60.68	60.11	53.16	7.52	4.63	56.05
CNN	Baseline (R1)	46.11	45.68	7.12	38.99	2.81	43.30
CNN	Input (R2)	55.21	55.44	29.16	26.05	8.78	46.43
CNN	Training (R3)	44.47	42.76	30.04	14.43	21.11	23.36
CNN	Full (R4)	53.65	53.38	44.07	9.58	4.81	48.84

Архитектура	Режим	PGD		R_FGSM		HISPA		State Drift
		AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	
Mamba	Baseline (R1)	19.36	39.57	8.81	50.12	13.32	45.61	2.5007
Mamba	Input (R2)	22.52	36.31	11.12	47.71	17.09	41.74	2.0892
Mamba	Training (R3)	49.88	10.61	42.99	17.50	47.25	13.24	0.4251
Mamba	Full (R4)	52.24	7.41	47.57	12.08	50.37	9.28	0.3262
Transformer	Baseline (R1)	18.48	40.03	8.95	49.56	11.94	46.57	—
Transformer	Input (R2)	37.74	23.40	27.10	34.04	32.77	28.37	—
Transformer	Training (R3)	45.75	10.40	39.79	16.36	43.15	13.00	—
Transformer	Full (R4)	53.36	7.32	48.50	12.18	51.44	9.24	—
CNN	Baseline (R1)	7.12	38.99	2.34	43.77	3.91	42.20	—
CNN	Input (R2)	29.33	25.88	18.23	36.98	24.35	30.86	—
CNN	Training (R3)	30.02	14.45	22.70	21.77	26.93	17.54	—
CNN	Full (R4)	43.90	9.75	38.53	15.12	41.88	11.77	—

На CIFAR-10 Mamba и Transformer демонстрируют сопоставимую динамику. Для Mamba в режиме Full Defense AdvAcc увеличивается с 19.36 до 52.24 % по PGD, с 8.81 до 47.57 % по R-FGSM и с 13.32 до 50.37 % по HiSPA, при этом чистая точность не снижается (58.93 % → 59.65 %), а State Drift падает с 2.50 до 0.33. Атака Carlini–Wagner (CW) остаётся наиболее сложной для всех архитектур: даже после защиты AdvAcc по CW для Mamba составляет лишь 5.79 %, что согласуется с её спецификой.

Сопоставление с Transformer (Full Defense, AdvAcc 53.16 % по AutoAttack) показывает, что прирост устойчивости Mamba к градиентным атакам сопоставим между архитектурами, тогда как CNN при том же наборе атак уступает обоим. Это подтверждает, что специфический вклад вносит именно регуляризация скрытого состояния Mamba. Состязательная точность моделей на CIFAR-10 по всем атакам и режимам представлена на рисунке 4.4.

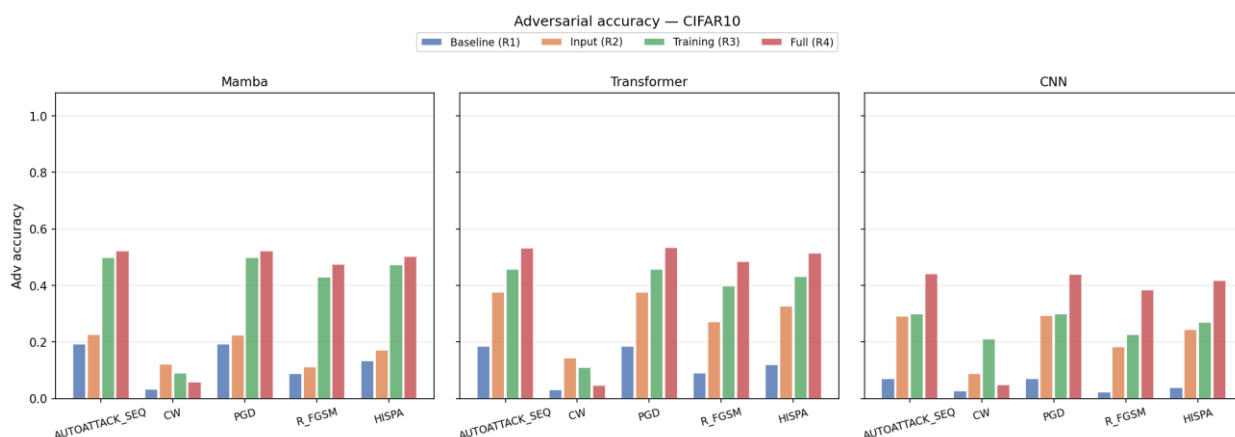


Рисунок 4.4 – Состязательная точность моделей на датасете CIFAR-10 по режимам защиты

4.4.3 Результаты на текстовых датасетах IMDB и AG News

Текстовые датасеты позволяют оценить эффективность атак Info Gap и Sliding Patch в семантическом домене и зависимость устойчивости от длины контекста (IMDB — 256 токенов, AG News — 128 токенов). Результаты по датасетам IMDB и AG News приведены в таблицах 4.9 и 4.10 соответственно.

Таблица 4.9 — Результаты на датасете IMDB

Архитектура	Режим	Clean ACC (%)	F1 macro (%)	PGD		R_FGSM	
				AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)
Mamba	Baseline (R1)	84.92	84.92	12.74	72.18	1.04	83.88
Mamba	Input (R2)	82.58	82.60	50.00	32.58	50.00	32.58
Mamba	Training (R3)	87.72	87.71	66.54	21.18	67.30	20.42
Mamba	Full (R4)	86.79	86.73	63.83	22.96	64.93	21.86
Transformer	Baseline (R1)	83.93	83.93	1.57	82.36	0.02	83.91
Transformer	Input (R2)	80.74	80.83	18.19	62.55	2.82	77.92
Transformer	Training (R3)	84.40	84.40	34.44	49.96	42.38	42.02
Transformer	Full (R4)	82.28	82.24	37.78	44.50	40.10	42.18
CNN	Baseline (R1)	85.89	85.89	28.44	57.45	3.95	81.94
CNN	Input (R2)	84.97	84.96	16.98	67.99	2.37	82.60
CNN	Training (R3)	85.75	85.75	44.30	41.45	45.74	40.01
CNN	Full (R4)	85.55	85.62	63.21	22.34	64.00	21.55

Архитектура	Режим	HISPA		SLIDING_PATCH		INFO_GAP		State Drift
		AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	
Mamba	Baseline (R1)	29.50	55.42	63.36	21.56	55.57	29.35	4.9961
Mamba	Input (R2)	16.02	66.56	50.00	32.58	50.00	32.58	2.0575
Mamba	Training (R3)	72.82	14.90	83.75	3.97	53.89	33.83	0.9419
Mamba	Full (R4)	70.86	15.93	82.37	4.42	77.55	9.24	1.0609
Transformer	Baseline (R1)	26.43	57.50	53.36	30.57	50.80	33.13	—
Transformer	Input (R2)	27.19	53.55	61.22	19.52	52.59	28.15	—
Transformer	Training (R3)	49.21	35.19	79.92	4.48	66.44	17.96	—
Transformer	Full (R4)	49.44	32.84	76.99	5.29	62.49	19.79	—
CNN	Baseline (R1)	30.67	55.22	50.04	35.85	50.00	35.89	—

CNN	Input (R2)	36.82	48.15	55.90	29.07	52.14	32.83	—
CNN	Training (R3)	55.50	30.25	82.32	3.43	80.78	4.97	—
CNN	Full (R4)	69.55	16.00	82.16	3.39	80.20	5.35	—

На IMDB (256 токенов) базовая Mamba уязвима к градиентным атакам в пространстве эмбедингов: AdvAcc составляет 12.74 % по PGD и лишь 1.04 % по R-FGSM, тогда как к структурным атакам Sliding Patch (63.36 %) и Info Gap (55.57 %) модель сравнительно устойчива уже в базовом виде Full Defense поднимает устойчивость к градиентным атакам до 63–65 %, к HiSPA — до 70.86 %, а наиболее заметный эффект проявляется на Info Gap: AdvAcc возрастает с 55.57 до 77.55 %. При этом чистая точность не снижается, а растёт (84.92 % → 86.79 %), а State Drift падает с 5.00 до 1.06. Динамика состязательной точности по атакам и режимам представлена на рисунке 4.5.

Отдельно стоит отметить вырожденное поведение режима Input Defense для Mamba на бинарной задаче IMDB: показатели AdvAcc около 50 % по всем атакам соответствуют коллапсу к мажоритарному классу. Это демонстрирует, что входное сглаживание без сопутствующего состязательного обучения не обеспечивает устойчивости, а только выравнивает предсказания, что подчёркивает необходимость комбинирования входной защиты с состязательным обучением в режиме Full Defense в данном случае.

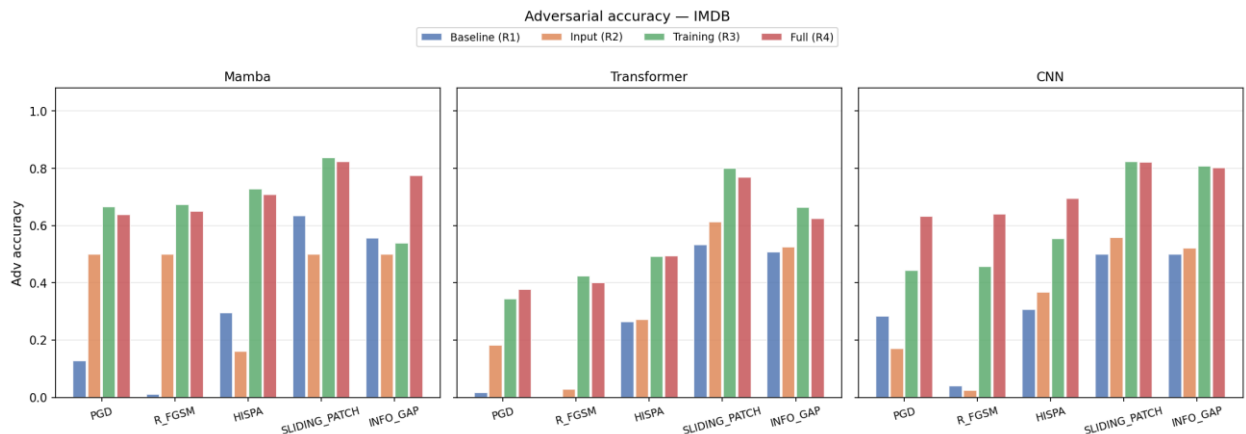


Рисунок 4.5 – Состязательная точность моделей на датасете IMDB по режимам защиты

Таблица 4.10 — Результаты на датасете AG News

Архитектура	Режим	Clean ACC (%)	F1 macro (%)	PGD		R_FGSM	
				AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)
Mamba	Baseline (R1)	90.70	90.66	78.24	12.46	67.00	23.70
Mamba	Input (R2)	89.55	89.65	68.28	21.27	49.13	40.42

Mamba	Training (R3)	92.63	92.62	87.54	5.09	83.55	9.08
Mamba	Full (R4)	91.95	91.84	85.74	6.21	80.41	11.54
Transformer	Baseline (R1)	90.93	90.92	72.93	18.00	59.71	31.22
Transformer	Input (R2)	90.67	90.50	70.91	19.76	57.93	32.74
Transformer	Training (R3)	92.50	92.49	86.53	5.97	82.09	10.41
Transformer	Full (R4)	92.45	92.37	87.13	5.32	83.26	9.19
CNN	Baseline (R1)	90.66	90.65	73.88	16.78	58.59	32.07
CNN	Input (R2)	90.63	90.70	72.93	17.70	56.75	33.88
CNN	Training (R3)	92.72	92.72	85.80	6.92	79.76	12.96
CNN	Full (R4)	92.70	92.66	86.25	6.45	80.64	12.06

Архитектура	Режим	HISPA		SLIDING_PATCH		INFO_GAP		State Drift
		AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	
Mamba	Baseline (R1)	73.91	16.79	82.71	7.99	61.45	29.25	9.9859
Mamba	Input (R2)	60.30	29.25	78.66	10.89	60.61	28.94	1.1326
Mamba	Training (R3)	86.08	6.55	88.22	4.41	67.49	25.14	0.2560
Mamba	Full (R4)	83.57	8.38	86.78	5.17	81.30	10.65	0.2508
Transformer	Baseline (R1)	66.08	24.85	84.41	6.52	68.68	22.25	—
Transformer	Input (R2)	64.43	26.24	83.13	7.54	72.61	18.06	—
Transformer	Training (R3)	84.59	7.91	86.18	6.32	84.80	7.70	—
Transformer	Full (R4)	85.66	6.79	86.43	6.02	72.53	19.92	—
CNN	Baseline (R1)	68.22	22.44	83.83	6.83	83.30	7.36	—
CNN	Input (R2)	66.97	23.66	83.96	6.67	83.63	7.00	—
CNN	Training (R3)	83.45	9.27	87.49	5.23	86.22	6.50	—
CNN	Full (R4)	84.11	8.59	87.91	4.79	86.55	6.15	—

На датасете AG News (128 токенов) базовая модель Mamba изначально более устойчива за счёт короткого контекста (AdvAcc 78.24 % по PGD, 82.71 % по Sliding Patch), однако обладает аномально высоким дрейфом скрытых состояний (SD 9.99). Применение Training Defense и Full Defense поднимает состязательную точность до 80–88 % по большинству атак и резко снижает State Drift — до 0.25–0.26. Наиболее выраженный

прирост наблюдается для атаки Info Gap: с 61.45 до 81.30 % в режиме Full Defense, что показано на рисунке 4.6.

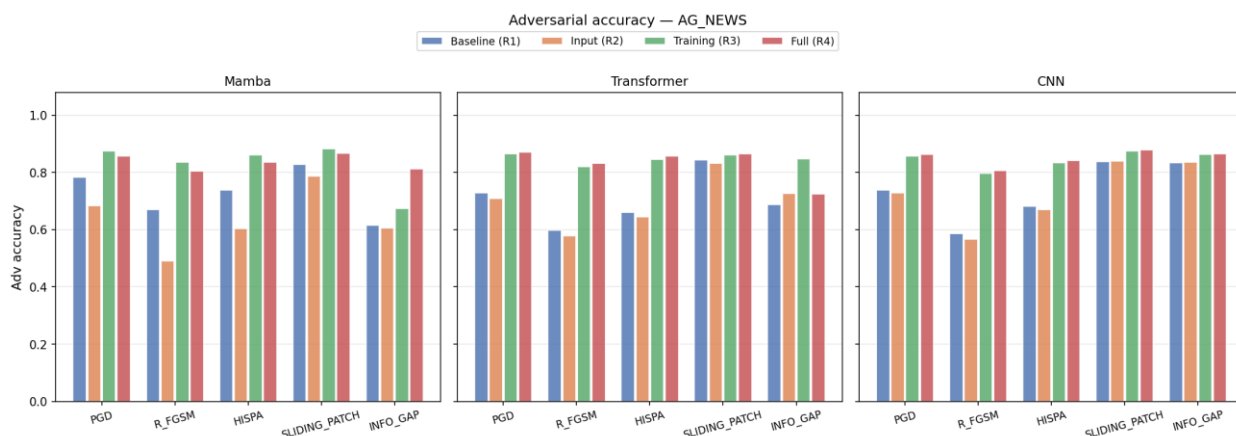


Рисунок 4.6 – Состязательная точность моделей на датасете AG News по режимам защиты

Все три архитектуры на AG News демонстрируют близкие итоговые показатели устойчивости, что объясняется коротким контекстом, ограничивающим долю последовательности, доступную структурным атакам. Из графика на рисунке 4.7 можно сделать вывод, что наиболее заметный эффект применения метода для Mamba на этом датасете — практически полное устранение исходного дрейфа скрытых состояний (с 9.99 до 0.25), что говорит о том, что повышенный SD может возникать не только из-за длины последовательности, но и из-за специфики конкретной задачи, и регуляризатор эффективно подавляет такой дрейф независимо от его причины.

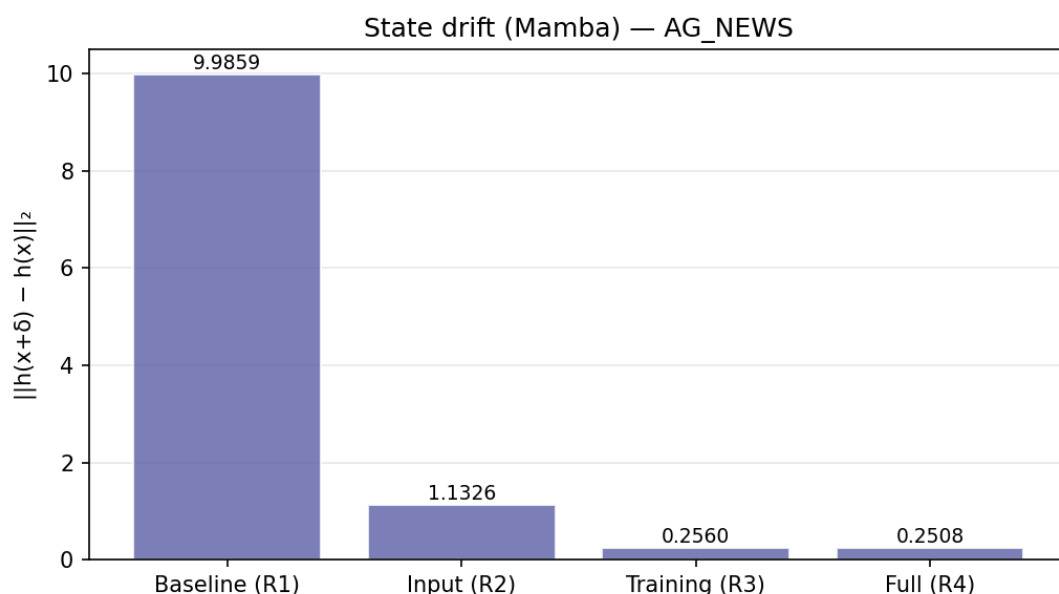


Рисунок 4.7 – State Drift модели Mamba на датасете AG News по режимам защиты

4.4.4 Результаты на Speech Commands

Аудиодатасет Speech Commands (T = 16 000) является наиболее требовательным с точки зрения вычислительных ресурсов и наиболее показательным для атак Sliding Patch и Info Gap, поскольку любой сегментный сбой в аудиосигнале непосредственно ведёт к ошибке классификации команды. Дополнительно оценивается State Drift для Mamba, демонстрирующий накопление состязательного дрейфа на последовательностях большой длины.

Таблица 4.11 — Результаты на датасете Speech Commands

Архитектура	Режим	Clean ACC (%)	F1 macro (%)	PGD		R_FGSM	
				AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)
Mamba	Baseline (R1)	82.50	81.37	0.00	82.50	3.49	79.01
Mamba	Input (R2)	59.02	75.66	14.89	44.13	7.46	51.56
Mamba	Training (R3)	73.76	72.91	34.54	39.22	67.96	5.80
Mamba	Full (R4)	80.67	83.85	72.32	8.35	65.30	15.37
Transformer	Baseline (R1)	75.42	73.67	1.43	73.99	4.50	70.92
Transformer	Input (R2)	36.12	37.98	7.97	28.15	5.11	31.01
Transformer	Training (R3)	67.39	66.16	0.50	66.89	64.63	2.76
Transformer	Full (R4)	42.94	39.36	29.17	13.77	21.89	21.05
CNN	Baseline (R1)	24.34	19.34	0.85	23.49	0.20	24.14
CNN	Input (R2)	56.57	60.86	15.26	41.31	5.18	51.39
CNN	Training (R3)	23.76	19.16	11.33	12.43	7.08	16.68
CNN	Full (R4)	51.99	47.89	37.39	14.60	28.47	23.52

Архитектура	Режим	HISPA		SLIDING_PATCH		INFO_GAP		State Drift
		AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	AdvAcc (%)	Gap (%)	
Mamba	Baseline (R1)	0.00	82.50	35.71	46.79	25.92	56.58	19.6795
Mamba	Input (R2)	9.57	49.45	19.80	39.22	11.59	47.43	6.7879
Mamba	Training (R3)	23.91	49.85	33.18	40.58	21.58	52.18	3.9230
Mamba	Full (R4)	70.51	10.16	29.20	51.47	19.76	60.91	1.1157
Transformer	Baseline (R1)	0.30	75.12	17.67	57.75	15.83	59.59	—
Transformer	Input (R2)	5.22	30.90	12.63	23.49	8.31	27.81	—
Transformer	Training (R3)	0.03	67.36	10.90	56.49	9.08	58.31	—
Transformer	Full (R4)	26.27	16.67	10.78	32.16	7.63	35.31	—

CNN	Baseline (R1)	0.35	23.99	12.84	11.50	13.82	10.52	—
CNN	Input (R2)	9.85	46.72	17.51	39.06	15.31	41.26	—
CNN	Training (R3)	9.34	14.42	9.53	14.23	10.32	13.44	—
CNN	Full (R4)	34.12	17.87	21.15	30.84	14.70	37.29	—

Из диаграммы на рисунке 4.8 видно, что базовая Mamba на Speech Commands полностью уязвима к атакам PGD и HiSPA (AdvAcc 0.00 % в обоих случаях) и почти полностью к R-FGSM (3.49 %), при этом State Drift достигает максимального среди всех датасетов значения (19.68). Full Defense повышает состязательную точность до 72.32 % (PGD), 65.30 % (R-FGSM) и 70.51 % (HiSPA) при сохранении чистой точности (82.50 % → 80.67 %, в пределах допустимого порога τ_{acc}) и снижает State Drift до 1.12 — почти в восемнадцать раз, что иллюстрирует график на 4.9. Это наиболее заметное подтверждение эффективности регуляризатора L_{reg} на сверхдлинных последовательностях.

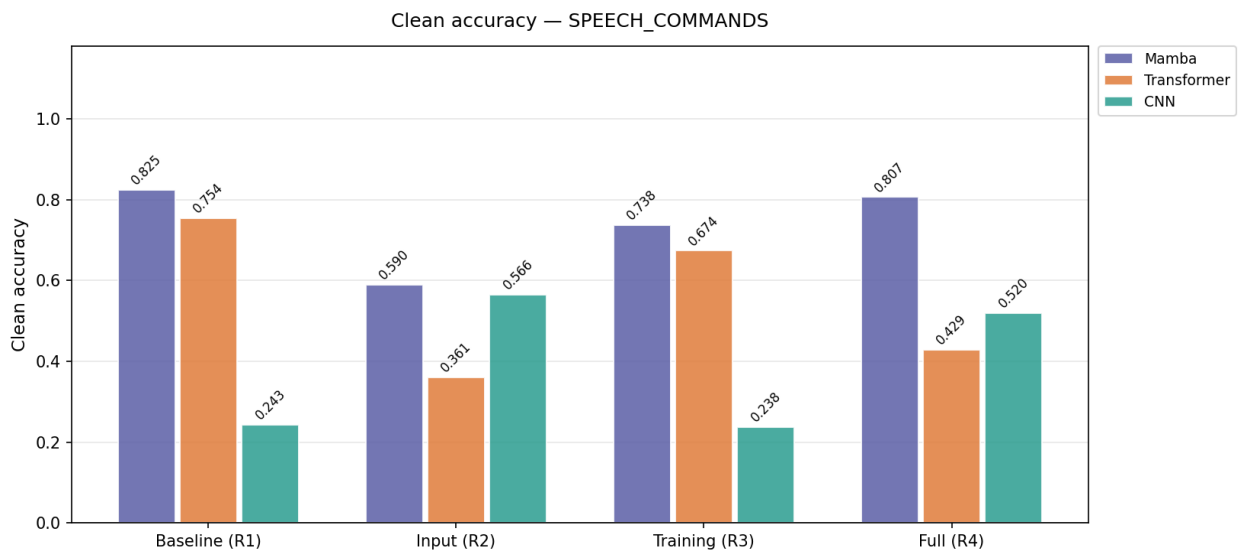


Рисунок 4.8 — Чистая точность моделей на датасете *Speech Commands* по режимам защиты

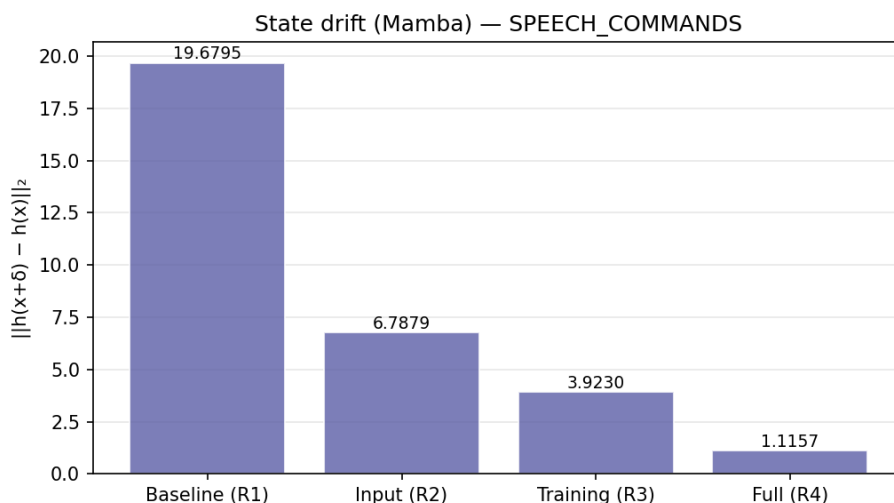


Рисунок 4.9 – State Drift модели Mamba на датасете Speech Commands по режимам защиты

Структурные атаки Sliding Patch и Info Gap остаются слабым местом на длинном аудиосигнале: в режиме Full Defense их AdvAcc (29.20 и 19.76 %) несколько ниже базовых значений (35.71 и 25.92 %), поскольку входное сглаживание и состязательное обучение, настроенные на градиентные возмущения, частично снижают долю полезного сигнала при сегментных воздействиях. На этом же датасете отчётливо проявляется преимущество Mamba над Transformer: трансформерная модель в режиме Full Defense теряет почти половину чистой точности (75.42 % → 42.94 %), тогда как Mamba сохраняет качество при сопоставимой робастности к градиентным атакам. Динамика состязательной точности модели показана на рисунке 4.10.

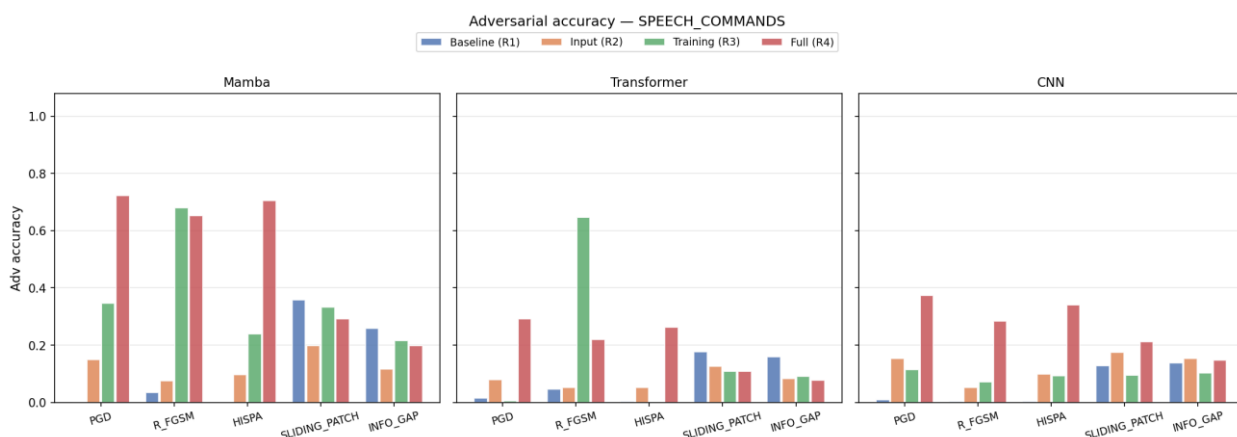


Рисунок 4.10 – Состязательная точность моделей на датасете Speech Commands по режимам защиты

4.4.5 Сравнительный анализ

На основе совокупности полученных данных проведена проверка критериев на основе метрик, сформулированных в разделе 2: критерия сохранения качества (снижение

стандартной точности не более $\tau_{acc} \in [0.03, 0.05]$ относительно Baseline) и критерия робастности ($\text{AdvAcc}(f_{adv}, A) > \text{AdvAcc}(f, A)$) для всех операторов $A \in \{A_{grad}, A_{SSM}\}$.

Критерий сохранения качества для Mamba в режиме Full Defense выполняется на всех шести датасетах: на MNIST, CIFAR-10, IMDB и AG News чистая точность не снижается и даже немного растёт; на Speech Commands снижение составляет 1.83 пункта (2.2 %), на sMNIST — 3.98 пункта (около 4 %), что укладывается в верхнюю границу порога τ_{acc} . При этом режимы Input Defense и Training Defense, применённые по отдельности, нарушают этот критерий на ряде датасетов (например, Input Defense снижает чистую точность Mamba на Speech Commands с 82.50 до 59.02 %).

Критерий робастности для Mamba выполняется на всех датасетах для атак класса A_{grad} (PGD, R-FGSM, AutoAttack, CW) и атаки HiSPA: AdvAcc в режимах Training Defense и Full Defense многократно превышает базовую. Для структурных атак (Sliding Patch, Info Gap), измеренных на sMNIST, IMDB, AG News и Speech Commands, критерий также выполняется в большинстве случаев: Full Defense превосходит Baseline по Sliding Patch на sMNIST (61.22 % против 55.71 %) и по обоим атакам на IMDB и AG News. Единственное исключение — Speech Commands, самый длинный из рассмотренных датасетов, где Full Defense по Sliding Patch и Info Gap несколько уступает Baseline. Наиболее эффективными режимами защиты остаются Training Defense и Full Defense; режим Input Defense, применяемый изолированно, наименее эффективен и в ряде случаев приводит к деградации качества.

Сводный анализ прироста состоятельности точности (Full Defense относительно Baseline) показывает, что наибольший выигрыш достигается на датасетах, где базовая модель была наиболее уязвима: на MNIST AdvAcc по PGD возрастает на 93.78 пункта, на Speech Commands — на 72.32 пункта, на sMNIST и CIFAR-10 прирост составляет порядка 28–33 пунктов. На изначально устойчивом коротком тексте AG News прирост умереннее (около 7–8 пунктов по PGD), но сопровождается кратным снижением State Drift.

Динамика State Drift для Mamba подтверждает действие регуляризатора L_{reg} : на всех датасетах метрика монотонно снижается при переходе от Baseline к Training Defense и Full Defense. Наиболее выраженная стабилизация наблюдается на длинных последовательностях — Speech Commands (с 19.68 до 1.12) и AG News (с 9.99 до 0.25), что согласуется с гипотезой о подавлении лавинообразного накопления возмущений вдоль рекуррентной цепи. На MNIST и IMDB значения SD между Training Defense и Full Defense близки, причём минимум достигается именно на Training Defense (0.57 и 0.94 соответственно против 0.69 и 1.06 у Full Defense) — небольшой рост SD при переходе к Full

Defense отражает дополнительный вклад входного сглаживания и не противоречит общей картине стабилизации. Сводная динамика State Drift по всем шести датасетам показана на рисунке 4.11, а общая картина выполнения критерия робастности — на рисунке 4.12.

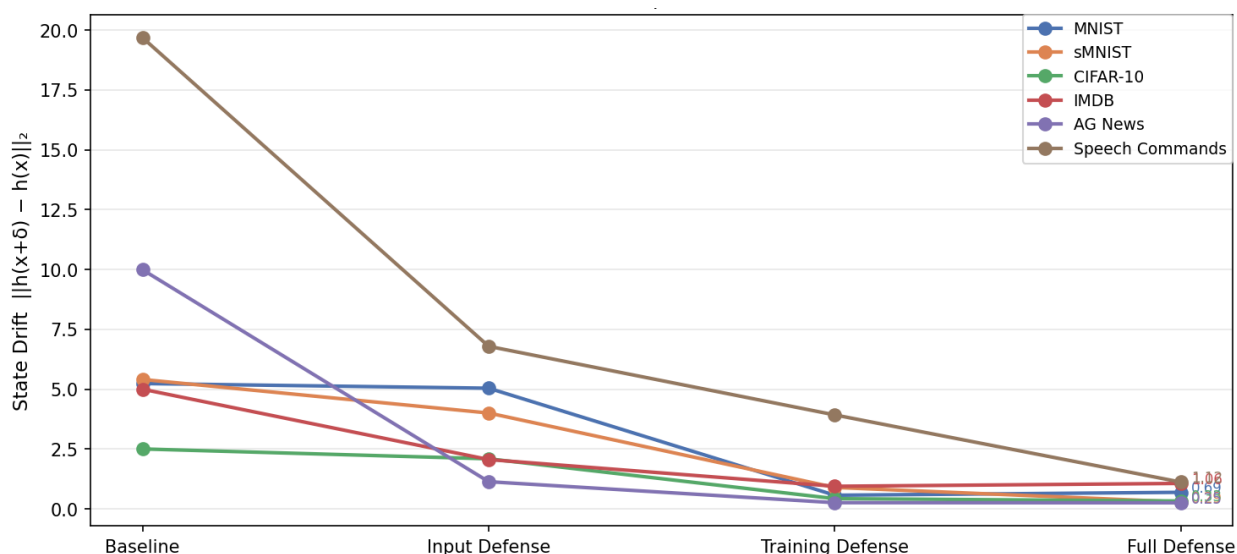


Рисунок 4.11 – Дрейф скрытых состояний (State Drift) модели Mamba по шести датасетам и режимам защиты (Baseline, Input Defense, Training Defense, Full Defense)

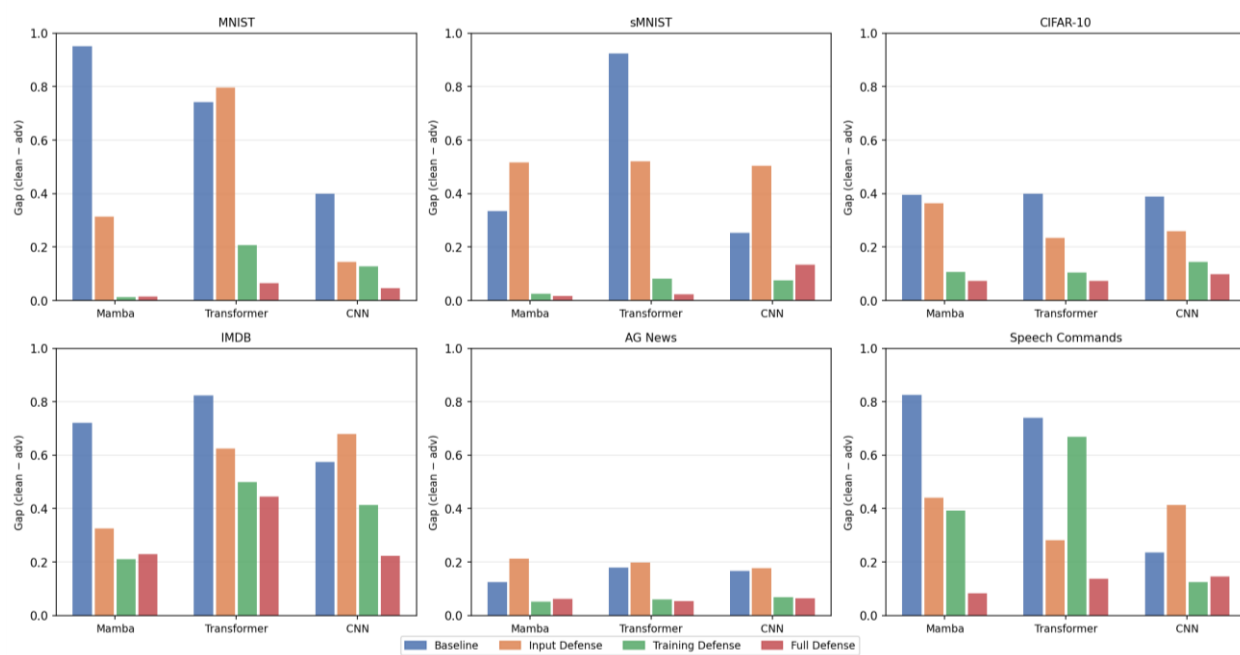


Рисунок 4.12 – Робастный разрыв (Gap) по датасетам, архитектурам, атакам и режимам защиты

Отдельно были сопоставлены вычислительные затраты режимов обучения. Результаты отражены на рисунке 4.13.

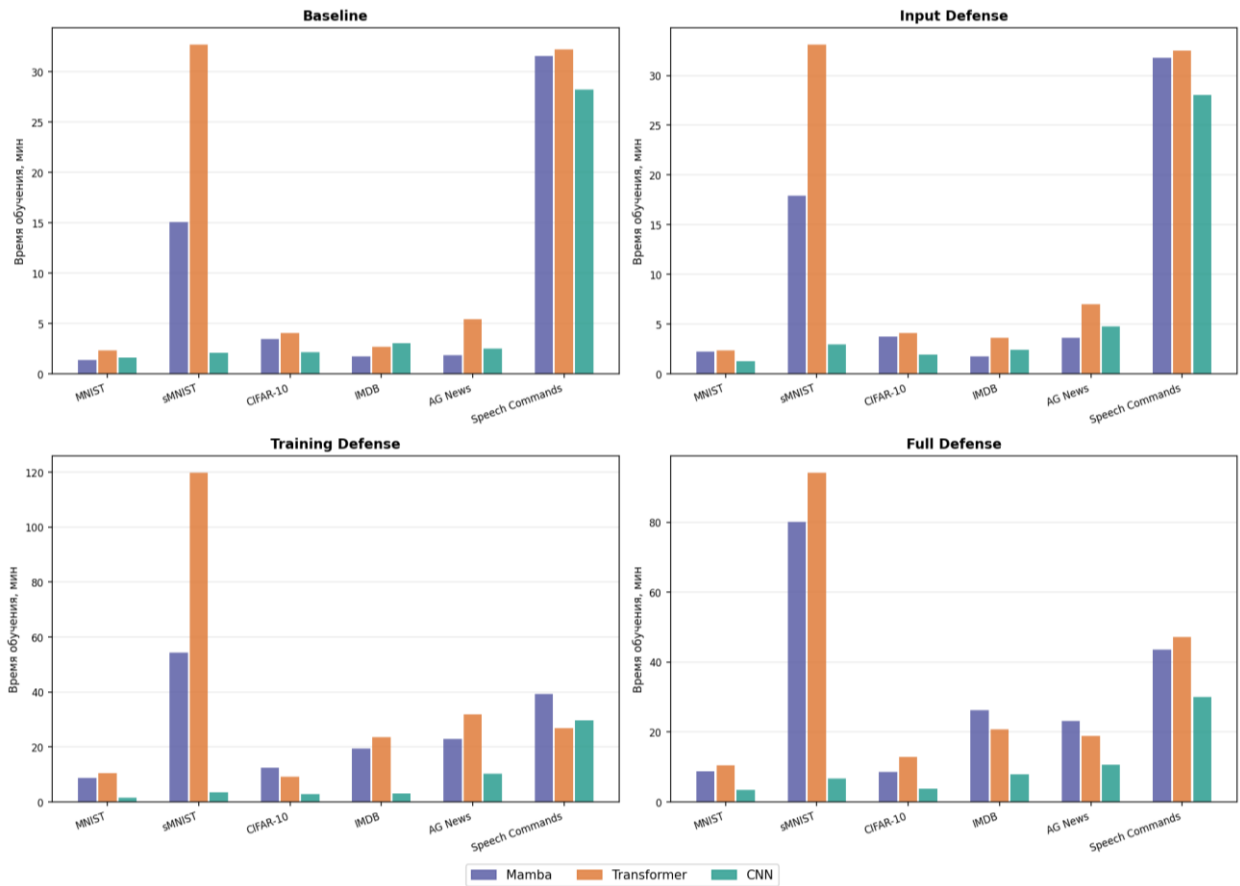


Рисунок 4.13 – Сравнение времени обучения архитектур в режимах Baseline, Training Defense и Full Defense

Состязательное обучение в режимах Training Defense и Full Defense увеличивает время обучения примерно в 3–4 раза относительно Baseline за счёт генерации состязательных примеров на каждой итерации, что соответствует известным оценкам для PGD-обучения [25]. При этом на этапе инференса Mamba сохраняет линейную сложность по длине последовательности и остаётся эффективнее Transformer при обработке длинных входов: на Speech Commands ($T = 16\,000$) трансформерная модель в режиме Full Defense теряет почти половину чистой точности, тогда как Mamba удерживает точность, близкую к Baseline, при сопоставимом уровне робастности к градиентным атакам. Различия в динамике сходимости между режимами обучения иллюстрируют кривые обучения на рисунке 4.14.

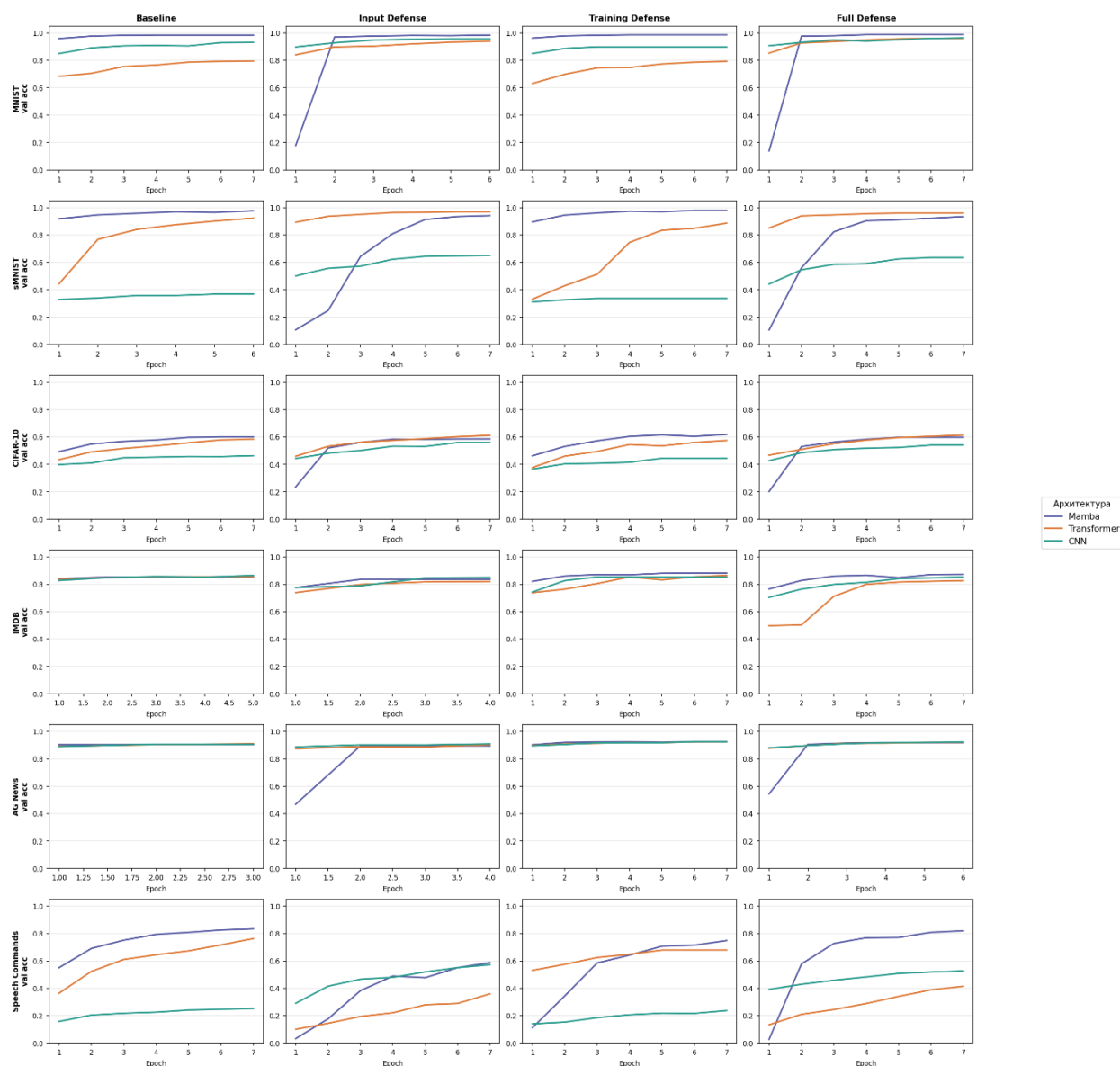


Рисунок 4.14 – Кривые обучения для режимов Baseline, Training Defense и Full Defense

4.5. Выводы

В подразделе подводятся итоги экспериментального исследования: интерпретируются полученные количественные результаты, оценивается практическая применимость разработанного метода и формулируются его границы применимости.

Экспериментальный цикл охватил три архитектуры (Mamba, Transformer, CNN), шесть наборов данных различной природы и семь типов состязательных атак — как стандартные градиентные методы (PGD, R-FGSM, AutoAttack, CW), так и SSM-специфичные воздействия (HiSPA, Sliding Patch, Info Gap). Для каждой архитектуры были

реализованы четыре конфигурации обучения, обеспечивающие изолированную оценку вклада входной и обучающей защиты.

Критерий сохранения качества (τ_{acc}) для Mamba в режиме Full Defense выполняется на всех шести датасетах (подробные значения приведены в подразделе 4.4). При изолированном применении Input Defense порог нарушается на ряде датасетов из-за чрезмерного сглаживания входа без сопутствующего состязательного обучения, а на бинарной задаче IMDB этот режим приводит к коллапсу к мажоритарному классу.

Критерий робастности для Mamba выполняется для всех атак класса A_{grad} и атаки HiSPA на всех датасетах, а также для структурных атак A_{SSM} на большинстве датасетов; единственное системное исключение — структурные атаки на сверхдлинной последовательности Speech Commands, где Full Defense не превосходит Baseline. Наиболее эффективными режимами защиты остаются Training Defense и Full Defense; изолированный режим Input Defense наименее эффективен и в ряде случаев снижает качество.

Главным результатом является подтверждение эффективности регуляризатора дрейфа скрытых состояний L_{reg} : метрика State Drift для Mamba в режимах Training Defense и Full Defense снижается относительно Baseline на всех датасетах, что свидетельствует о подавлении амплитуды возмущений в латентном пространстве SSM и препятствует их лавинообразному накоплению вдоль длинных контекстов.

Сопоставление Mamba с Transformer и CNN в идентичных условиях показывает, что SSM-архитектура действительно особенно уязвима к атакам класса A_{SSM} (HiSPA, структурные искажения), и эта уязвимость существенно снижается при применении разработанного метода. Для атак класса A_{grad} прирост устойчивости Mamba сопоставим с приростом Transformer, что говорит об архитектурной независимости компонента SA-AT — специфический для Mamba эффект обеспечивает именно регуляризация скрытого состояния. По вычислительной стороне метод увеличивает время обучения в 3–4 раза (см. подраздел 4.4), но не меняет асимптотику инференса: Mamba сохраняет линейную сложность по длине последовательности и на сверхдлинных входах демонстрирует заметно более стабильную точность, чем Transformer, что подтверждает целесообразность применения метода в условиях ограниченных вычислительных ресурсов.

Таким образом, комбинация SA-AT, Attack-Mix и двухэтапной генерации возмущений обеспечивает устойчивость Mamba к широкому спектру атак без критической

деградации базового качества классификации, что подтверждает выполнение требований, сформулированных в разделе 2.1. Границы применимости метода связаны прежде всего со структурными атаками на сверхдлинных последовательностях, где компромисс между входным сглаживанием и сохранением информации требует дальнейшей настройки. Полученные результаты создают основу для дальнейшего исследования адаптивных методов состязательного обучения применительно к SSM-архитектурам большого масштаба.

Заключение

В ходе выполнения курсовой работы была достигнута поставленная цель — разработан метод повышения устойчивости модели Mamba к состязательным атакам, учитывающий архитектурные особенности моделей пространства состояний.

В первом разделе по результатам анализа было установлено, что, несмотря на линейную вычислительную сложность и высокую эффективность на длинных последовательностях, устойчивость Mamba к состязательным атакам оставалась недостаточно изученной. Исследование существующих методов защиты показало, что подходы, разработанные для CNN и Transformer, не учитывают специфику рекуррентного механизма SSM и не обеспечивают достаточной робастности при прямом применении к Mamba.

Во втором разделе формализована задача повышения устойчивости Mamba и разработан метод состязательного обучения, адаптированный к архитектурным особенностям SSM. Метод включает три ключевых компонента: State-Aware Adversarial Training (SA-AT) с регуляризатором дрейфа скрытых состояний Lreg, стратегию Attack-Mix с двухэтапной генерацией возмущений и входную защиту (Gating-фильтр, стохастическое сглаживание, стабилизация параметра дискретизации Δ).

В третьем разделе спроектирован и реализован программный фреймворк, обеспечивающий обучение, тестирование и оценку робастности Mamba-моделей. Фреймворк построен на принципах модульности и расширяемости: она поддерживает несколько режимов обучения (Baseline, Input Defense, Training Defense, Full Defense), унифицированную генерацию возмущений для дискретных и непрерывных входных данных, а также вычисление метрики State Drift через механизм forward hooks. Тестирование подтвердило корректность реализации всех компонентов.

Результаты экспериментального исследования подтвердили эффективность предложенного метода: в режиме Full Defense критерий сохранения качества выполняется для Mamba на всех датасетах, а состязательная точность по атаке PGD возрастает с 3,32 до 97,10 % на MNIST и с 0,00 до 72,32 % на Speech Commands. Метрика State Drift снижается на всех датасетах — наиболее существенно на сверхдлинных последовательностях: с 19,68 до 1,12 на Speech Commands и с 9,99 до 0,25 на AG News, что подтверждает подавление лавинообразного накопления состязательных возмущений в рекуррентной цепи SSM. Единственным системным ограничением метода остаются структурные атаки (Sliding

Patch, Info Gap) на сверхдлинных аудиопоследовательностях, где компромисс между входным сглаживанием и сохранением локальной информации требует дополнительной настройки.

Перспективы дальнейших исследований включают распространение метода на масштабные SSM-архитектуры (Mamba-2, Jamba), исследование устойчивости к адаптивным атакам с доступом к механизму защиты, а также разработку более тонких стратегий входной предобработки для длинных последовательностей, позволяющих преодолеть выявленное ограничение на структурные атаки.

Список литературы

1. Wen Q., Zhou T., Zhang C., Chen W., Ma Z., Yan J., Sun L. Transformers in Time Series: A Survey // Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI). 2023. C. 6778–6786.
2. Ali Z. A Comprehensive Overview and Comparative Analysis of CNN, RNN-LSTM and Transformer // Journal of Artificial Intelligence. 2024. Vol. 6, Issue 1. P. 301–360.
3. Qu H., Ning L., An R., Fan W., Derr T., Liu H., Xu X., Li Q. A Survey on Mamba: A New Frontier in Sequence Modeling // arXiv. 2024. arXiv:2408.01129v7.
4. Wang X., Wang S., Ding Y., Li Y., Wu W., Rong Y., Kong W., Huang J., Li S., Yang H., Wang Z., Jiang B., Li C., Wang Y., Tian Y., Tang J. State Space Model for New-Generation Network Alternative to Transformers: A Survey // arXiv. 2024. arXiv:2404.09516v1.
5. Somvanshi S., Islam M. M., Mimi M. S., Pollock S. B. B., Chhetri G., Das S. From S4 to Mamba: A Comprehensive Survey on Structured State Space Models // arXiv. 2025. arXiv:2503.18970v2.
6. Gu A., Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces // OpenReview (COLM). 2023.
7. Abreu W., Biscainho L. W. P. AEROMamba: An Efficient Architecture for Audio Super-Resolution Using Generative Adversarial Networks and State Space Models // LAMIR Workshop. 2024.
8. Asif A. A., Kashaniyan M., Yu S., Muñoz J. P., Jannesari A. PerfMamba: Performance Analysis and Pruning of Selective State Space Models // Proceedings of the 17th BenchCouncil International Symposium on Evaluation Science and Engineering (Bench). 2025. C. 1–17.
9. Rando S., Romani L., Migliarini M., Franco L., Gudovskiy D., Galasso F. SeRpEnt: Selective Resampling for Expressive State Space Models // Proceedings of the International Conference on Machine Learning (ICML). 2024.
10. Khamaiseh S. Y., Bagagem D., Al-Alaj A., Mancino M., Alomari H. W. Adversarial Deep Learning: A Survey on Adversarial Attacks and Defense Mechanisms on Image Classification // IEEE Access. 2022. Vol. 10. P. 14410–14430.

11. Котенко И.В., Саенко И.Б., Лаута О.С., Васильев Н.А., Садовников В.Е. Метод противодействия состязательным атакам на системы классификации изображений // Вопросы кибербезопасности. 2025. № 2 (66). С. 114–123. DOI: 10.21681/2311-3456-2025-2-114-123.
12. Silva S. H., Najafirad P. Opportunities and Challenges in Deep Learning Adversarial Robustness: A Survey // arXiv. 2020. arXiv:2007.00753v2.
13. Zheng W. A Survey on Artificial Intelligence Systems Robustness: Adversarial Attacks and Defenses // Journal of Electronic Research and Application. 2026. Vol. 10, No. 1. P. 182–191.
14. Wu B., Zhu Z., Liu L., Liu Q., He Z., Lyu S. Attacks in Adversarial Machine Learning: A Systematic Survey from the Life-cycle Perspective // arXiv. 2023. arXiv:2302.09457v2.
15. Kotyan S. A Reading Survey on Adversarial Machine Learning: Adversarial Attacks and Their Understanding // arXiv. 2023. arXiv:2308.03363v1.
16. Costa J. C., Roxo T., Proença H., Inácio P. R. M. How Deep Learning Sees the World: A Survey on Adversarial Attacks & Defenses // IEEE Access. 2024. Vol. 12. P. 1–24.
17. Ibitoye O., Abou-Khamis R., El Shehaby M., Matrawy A., Shafiq M. O. The Threat of Adversarial Attacks Against Machine Learning in Network Security: A Survey // arXiv. 2019. arXiv:1911.02621v3.
18. Pelekis S., Koutroubas T., Blika A., Berdelis A., Karakolis E., Ntanos C., Spiliotis E., Askounis D. Adversarial machine learning: a review of methods, tools, and critical industry sectors // Artificial Intelligence Review. 2025. Vol. 58. P. 226.
19. Wu B., Wei S., Zhu M., Zheng M., Zhu Z., Zhang M., Chen H., Yuan D., Liu L., Liu Q. Defenses in Adversarial Machine Learning: A Survey // arXiv. 2023. arXiv:2312.08890v1.
20. Lin J., Dang L., Rahouti M., Xiong K. Adversarial Attacks and Data Poisoning Attacks in Machine Learning // AI, Machine Learning and Deep Learning: A Security Perspective. Boca Raton: CRC Press, 2021.
21. Перминов А.И. SLAP – простая линейная атака на персептрон // Труды Института системного программирования РАН. 2024. Т. 36, вып. 3. С. 83–92. DOI: 10.15514/ISPRAS-2024-36(3)-6.

22. Qi B., Luo Y., Gao J., Li P., Tian K., Ma Z., Zhou B. Exploring Adversarial Robustness of Deep State Space Models // Advances in Neural Information Processing Systems (NeurIPS). 2024.
23. Намиот Д.Е., Романов В.Ю. Об улучшении робастности моделей машинного обучения // International Journal of Open Information Technologies. 2024. Т. 12, № 3. С. 89–90.
24. Чехонина Е.А., Костюмов В.В. Обзор состязательных атак и методов защиты для детекторов объектов // International Journal of Open Information Technologies. 2023. Т. 11, № 7. С. 11–20.
25. Ауси Р.М., Заргарян Е.В., Заргарян Ю.А. Глубокое обучение методам защиты от атак // Известия Южного федерального университета. Технические науки. 2023. № 2. С. 227–239. DOI: 10.18522/2311-3103-2023-2-227-239.

Приложение

В приложении представлен листинг фрагментов кода разработанного фреймворка. Полный код реализации содержит 6197 строки.

Код для конфигураторов и загрузчиков данных:

```
@dataclass
class DatasetConfig:
    name: str
    input_type: str
    num_classes: int
    batch_size: int
    max_len: int | None=None
    vocab_size: int | None = None
    pad_token_id: int = 0
    checkpoint_path: str = "best_model.pt"
    continuous_input_dim: int = 1

@dataclass
class TrainConfig:
    epochs: int = 7
    lr: float = 1e-3
    d_model: int = 128
    n_layers: int = 3
    dropout: float = 0.1
    patience: int = 2
    min_delta: float = 1e-3
    num_workers: int = 0
    use_subset: bool = False
    train_subset_size: int = 5000
    test_subset_size: int = 2000
    val_split: float = 0.2

    n_filters: int = 100
    filter_sizes: tuple = (3, 4, 5)

    nhead: int = 8
    dim_feedforward: int = 512

    delta_clip: bool = False

@dataclass
class AttackConfig:
    attack_type: str = "pgd"
    adv_steps: int = 3
    adv_lr: float = 1e-2
    adv_eps: float = 1e-1
    adv_alpha: float = 0.5
    patch_size: float = 0.3
    two_stage: bool = False
    sa_at_lambda: float = 0.0

DATASET_NAME = "imdb"
train_cfg = TrainConfig(epochs=7, lr=1e-3, d_model=128, n_layers=3, patience=2, num_workers=0)
atk_cfg = AttackConfig(attack_type="pgd")
```

```

class TextClassificationDataset(Dataset):
    def __init__(self, hf_split, tokenizer, text_col: str, label_col: str, max_len: int):
        self.data = hf_split
        self.tokenizer = tokenizer
        self.text_col = text_col
        self.label_col = label_col
        self.max_len = max_len

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        item = self.data[idx]
        enc = self.tokenizer(
            item[self.text_col],
            padding="max_length",
            truncation=True,
            max_length=self.max_len,
            return_tensors="pt",
        )
        input_ids = enc["input_ids"].squeeze(0).long()
        label = torch.tensor(item[self.label_col], dtype=torch.long)
        return input_ids, label

class SpeechDataset(Dataset):
    def __init__(self, ds, label_map):
        self.ds = ds
        self.label_map = label_map

    def __len__(self):
        return len(self.ds)

    def __getitem__(self, idx):
        waveform, sample_rate, label, *_ = self.ds[idx]
        # приведение длины до 16000 отсчетов (1 сек)
        if waveform.size(1) > 16000: waveform = waveform[:, :16000]
        else: waveform = torch.nn.functional.pad(waveform, (0, 16000 - waveform.size(1)))

        # (1, 16000) → (100, 160)
        seq = waveform.reshape(-1)[:16000].reshape(100, 160)
        return seq, torch.tensor(self.label_map[label])

```

```

def build_data loaders(dataset_name: str, train_cfg: TrainConfig):
    dataset_name = dataset_name.lower()
    tokenizer = None

    if dataset_name == "mnist":
        transform = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize((0.5, ), (0.5, )),
            transforms.Lambda(lambda t: t.view(-1).reshape(49, 16)) # убрать
        ])
        train_dataset = datasets.MNIST(root=DATA_ROOT, train=True, download=True, transform=transform)
        test_dataset = datasets.MNIST(root=DATA_ROOT, train=False, download=True, transform=transform)
        ds_cfg = DatasetConfig(
            name="mnist",
            input_type="continuous",
            num_classes=10,
            batch_size=128,
            max_len=49, # было 784
            continuous_input_dim=16, # добавить
            checkpoint_path="best_model_mnist.pt",
        )

    elif dataset_name == "smnist":
        transform = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize((0.1307, ), (0.3081, )),
            transforms.Lambda(lambda x: x.view(-1, 1)) # (784, 1)
        ])
        train_dataset = datasets.MNIST(root=DATA_ROOT, train=True, download=True, transform=transform)
        test_dataset = datasets.MNIST(root=DATA_ROOT, train=False, download=True, transform=transform)
        ds_cfg = DatasetConfig(
            name="smnist",
            input_type="continuous",
            num_classes=10,
            batch_size=128,
            max_len=784,
            checkpoint_path="best_model_smnist.pt",
        )

    elif dataset_name == "imdb":
        raw = load_dataset("imdb")
        tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
        train_dataset = TextClassificationDataset(raw["train"], tokenizer, "text", "label", max_len=256)
        test_dataset = TextClassificationDataset(raw["test"], tokenizer, "text", "label", max_len=256)
        ds_cfg = DatasetConfig(
            name="imdb",
            input_type="tokens",
            num_classes=2,
            batch_size=32,
            max_len=256,
            vocab_size=tokenizer.vocab_size,
            pad_token_id=tokenizer.pad_token_id or 0,
            checkpoint_path="best_model_imdb.pt",
        )

```

```

elif dataset_name == "cifar10":
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
        transforms.Lambda(lambda t: t.reshape(-1).reshape(64, 48)),
    ])
    train_dataset = datasets.CIFAR10(root=DATA_ROOT, train=True, download=True, transform=transform)
    test_dataset = datasets.CIFAR10(root=DATA_ROOT, train=False, download=True, transform=transform)
    ds_cfg = DatasetConfig(
        name="cifar10",
        input_type="continuous",
        num_classes=10,
        batch_size=32,
        max_len=64,
        continuous_input_dim=48,
        checkpoint_path="best_model_cifar10.pt",
    )

elif dataset_name == "speech_commands":
    train_dataset = torchaudio.datasets.SPEECHCOMMANDS(root=DATA_ROOT, download=True, subset='training')
    test_dataset = torchaudio.datasets.SPEECHCOMMANDS(root=DATA_ROOT, download=True, subset='testing')

    sc_root = Path(DATA_ROOT) / "SpeechCommands" / "speech_commands_v0.02"
    labels = sorted([d.name for d in sc_root.iterdir()
                     if d.is_dir() and not d.name.startswith('.')])
    label_map = {i: l for i, l in enumerate(labels)}

    train_dataset = SpeechDataset(train_dataset, label_map)
    test_dataset = SpeechDataset(test_dataset, label_map)

    ds_cfg = DatasetConfig(
        name="speech_commands",
        input_type="continuous",
        num_classes=35,
        batch_size=64,
        max_len=100,
        continuous_input_dim=160,
        checkpoint_path="best_model_speech.pt",
    )

```

Код реализации модели Mamba:

```

class UniversalTransformerClassifier(nn.Module):
    def __init__(
        self,
        input_type: str,
        num_classes: int,
        d_model: int = 128,
        n_layers: int = 3,
        nhead: int = 8,
        dim_feedforward: int = 512,
        dropout: float = 0.1,
        vocab_size: int | None = None,
        continuous_input_dim: int = 1,
        pad_token_id: int = 0,
        pooling: str = "mean",
        use_gating_filter: bool = True,
        inference_noise_std: float = 0.05
    ):
        super().__init__()
        self.input_type = input_type
        self.pooling = pooling
        self.pad_token_id = pad_token_id
        self.inference_noise_std = inference_noise_std
        self.use_gating_filter = use_gating_filter

```

```

if input_type == "tokens":
    self.input_layer = nn.Embedding(vocab_size, d_model, padding_idx=pad_token_id)
else:
    self.input_layer = nn.Linear(continuous_input_dim, d_model)

if self.use_gating_filter:
    self.pre_filter = nn.Conv1d(d_model, d_model, kernel_size=3, padding=1, groups=d_model)

encoder_layer = nn.TransformerEncoderLayer(
    d_model=d_model, nhead=nhead, dim_feedforward=dim_feedforward,
    dropout=dropout, batch_first=True
)
self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=n_layers)

self.max_pos = 1024
self.pos_embedding = nn.Parameter(torch.zeros(1, self.max_pos, d_model))
nn.init.trunc_normal_(self.pos_embedding, std=0.02)

self.classifier = nn.Linear(d_model, num_classes)

```

```

def get_embeddings(self, x):
    if self.input_type == "tokens":
        emb = self.input_layer(x.long())
    else:
        if x.dim() == 4: x = x.view(x.size(0), -1, 1)
        elif x.dim() == 2: x = x.unsqueeze(-1)
        emb = self.input_layer(x.float())

    if not self.training and self.inference_noise_std > 0:
        emb = emb + torch.randn_like(emb) * self.inference_noise_std
    return emb

def forward_from_embeddings(self, embeddings, input_ids=None):
    x = embeddings
    if self.use_gating_filter:
        x = x.transpose(1, 2)
        x = F.relu(self.pre_filter(x))
        x = x.transpose(1, 2)

    seq_len = x.size(1)
    x = x + self.pos_embedding[:, :seq_len, :]

    src_key_padding_mask = (input_ids == self.pad_token_id) if (self.input_type == "tokens" and input_ids is not None) else None
    x = self.transformer(x, src_key_padding_mask=src_key_padding_mask)

    if self.pooling == "mean":
        mask = (~src_key_padding_mask).unsqueeze(-1).float() if src_key_padding_mask is not None else None
        x = (x * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1.0) if mask is not None else x.mean(dim=1)
    else:
        x = x[:, -1]
    return self.classifier(x)

def forward(self, x):
    return self.forward_from_embeddings(self.get_embeddings(x), input_ids=x if self.input_type == "tokens" else None)

```

Код функции-интерфейса для создания моделей:

```
def build_model(ds_cfg: DatasetConfig, train_cfg: TrainConfig, model_type: str = "mamba", use_defense: bool = False):
    common_params = {
        "input_type": ds_cfg.input_type,
        "num_classes": ds_cfg.num_classes,
        "d_model": train_cfg.d_model,
        "vocab_size": ds_cfg.vocab_size,
        "pad_token_id": ds_cfg.pad_token_id,
        "use_gating_filter": use_defense,
        "inference_noise_std": 0.05 if use_defense else 0.0,
        "continuous_input_dim": getattr(ds_cfg, "continuous_input_dim", 1),
    }

    if model_type == "mamba":
        return UniversalMambaClassifier(**common_params, n_layers=train_cfg.n_layers)
    elif model_type == "transformer":
        return UniversalTransformerClassifier(**common_params, n_layers=train_cfg.n_layers,
                                             nhead=train_cfg.nhead, dim_feedforward=train_cfg.dim_feedforward)
    elif model_type == "cnn":
        return UniversalCNNClassifier(**common_params, filter_sizes=train_cfg.filter_sizes)

    raise ValueError(f"Unknown model type: {model_type}")
```

Фрагмент блока генерации возмущений:

```
def get_adversarial_examples(model, images, labels, attack_type="pgd", eps=8/255, patch_size=0.3, steps=10):
    model.eval()
    is_emb = getattr(model, "is_embedding", False)

    if is_emb:
        emb_eps = eps

    if attack_type == "pgd":
        if is_emb:
            return _pgd_embedding(model, images, labels, eps=emb_eps, steps=steps)
        adversary = torchattacks.PGD(model, eps=eps, alpha=2/255, steps=steps)
        return adversary(images, labels)

    if attack_type == "r_fgsm":
        e = emb_eps if is_emb else eps
        adv = images.clone().detach() + torch.zeros_like(images).uniform_(-e, e)
        if not is_emb:
            adv = torch.clamp(adv, 0, 1)
            adv.requires_grad = True
            loss = F.cross_entropy(model(adv), labels)
            grad = torch.autograd.grad(loss, adv)[0]
            adv = adv.detach() + e * grad.sign()
            return adv if is_emb else torch.clamp(adv, 0, 1)

    if attack_type == "hispa":
        return apply_hispa_attack(model, images, labels, eps=(emb_eps if is_emb else eps))

    if attack_type in ("sliding_patch", "info_gap"):
        if is_emb:
            adv = images.clone()
            B, L, D = adv.shape
            frac = 0.35 if attack_type == "sliding_patch" else float(patch_size)
            p = max(1, int(L * frac))
            for i in range(B):
                s = torch.randint(0, max(1, L - p), (1,)).item()
                if attack_type == "sliding_patch":
                    adv[i, s:s+p, :] = torch.randn(p, D, device=adv.device) * images.std()
                else:
                    adv[i, s:s+p, :] = images.mean() + images.std()
            return adv.detach()
        adv = images.clone()
        B, C, Lseq = images.shape[0], images.shape[1], images.shape[2]
        frac = 0.35 if attack_type == "sliding_patch" else float(patch_size)
        p = max(1, int(Lseq * frac))
        for i in range(B):
            s = torch.randint(0, max(1, Lseq - p), (1,)).item()
            if attack_type == "sliding_patch":
                adv[i, :, s:s+p] = torch.randn_like(adv[i, :, s:s+p]) * 5.0
            else:
                adv[i, :, s:s+p] = 1.0 + torch.randn((C, p), device=images.device) * 0.5
        return torch.clamp(adv, 0, 1)
```

Универсальная функция для обучения моделей:

```
def train_model(model, train_loader, val_loader, ds_cfg, train_cfg, device,
                atk_cfg: Optional[AttackConfig] = None,
                checkpoint_path="best_model.pt",
                model_type: str = "mamba"):
    import time as _time

    adversarial = atk_cfg is not None
    model = model.to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=train_cfg.lr)
    criterion = nn.CrossEntropyLoss()
    early_stopping = EarlyStopping(patience=train_cfg.patience,
                                   min_delta=train_cfg.min_delta, mode="max")

    history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
    best_path = Path(checkpoint_path)
    total_train_time = 0.0

    for epoch in range(1, train_cfg.epochs + 1):
        t0 = _time.time()
        if adversarial:
            train_loss, train_acc = train_one_epoch_adversarial(
                model, train_loader, optimizer, criterion, device, atk_cfg, epoch=epoch)
        else:
            train_loss, train_acc = train_one_epoch(
                model, train_loader, optimizer, criterion, device)

        epoch_time = _time.time() - t0
        total_train_time += epoch_time

        if isinstance(model, UniversalMambaClassifier) and train_cfg.delta_clip:
            stabilize_mamba_delta(model)

        val_loss, val_acc = evaluate(model, val_loader, criterion, device)

        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)

    improved = early_stopping.step(val_acc)

    if improved:
        save_checkpoint(
            best_path, model, optimizer, epoch, history,
            ds_cfg, train_cfg,
            atk_cfg=atk_cfg,
            training_time_sec=total_train_time,
            model_type=model_type,
        )
        msg = f"saved best -> {best_path}"
    else:
        msg = f"no improvement ({early_stopping.counter}/{train_cfg.patience})"

    mode_name = f"ADV ({atk_cfg.attack_type})" if adversarial else "CLEAN"
    print(f"[{mode_name}] Epoch {epoch:02d}/{train_cfg.epochs} | "
          f"train_loss={train_loss:.4f} | val_loss={val_loss:.4f} | "
          f"val_acc={val_acc:.4f} | {epoch_time:.1f}s | {msg}")

    if early_stopping.should_stop:
        print(f"Early stopping at epoch {epoch}. Best val_acc={early_stopping.best_score:.4f}")
        break

    checkpoint = load_best_model(best_path, model, device)
    print(f"Loaded best model from epoch {checkpoint['epoch']} "
          f"with val_acc={checkpoint['training_summary']['best_val_acc']:.4f}")

    return history, checkpoint
```


Пример кода для обучения и тестирования Mamba на датасете Speech commands в Full Defense режиме:

```
gc.collect(); torch.cuda.empty_cache()

train_cfg_speech_commands_full_mamba = TrainConfig(
    epochs=7, lr=1e-3, d_model=128, n_layers=3,
    dropout=0.1, patience=2, min_delta=1e-3,
    num_workers=0, val_split=0.2,
    delta_clip=True
)

_speech_commands_full_mamba_model = build_model(ds_cfg, train_cfg_speech_commands_full_mamba, model_type="mamba", use_defense=True)
_speech_commands_full_mamba_hist, _speech_commands_full_mamba_ckpt = train_model(
    model=_speech_commands_full_mamba_model,
    train_loader=train_loader,
    val_loader=val_loader,
    ds_cfg=ds_cfg,
    train_cfg=train_cfg_speech_commands_full_mamba,
    device=device,
    atk_cfg=TRAIN_ATK_ADV,
    checkpoint_path="mamba_speech_commands_full.pt",
    model_type="mamba",
)
print(f"\n Обучение завершено: mamba_speech_commands_full.pt")
print(f" best_val_acc = {_speech_commands_full_mamba_ckpt['training_summary']['best_val_acc']:.4f}")

_criterion = nn.CrossEntropyLoss()
_results_speech_commands_full_mamba = {}

for _atk_name in ["pgd", "r_fgsm", "hispa", "sliding_patch", "info_gap"]:
    _test_atk = AttackConfig(attack_type=_atk_name, adv_eps=8/255,
                            patch_size=0.35 if 'patch' in _atk_name else 0.45)
    _, _adv_acc = evaluate_adversarial(
        _speech_commands_full_mamba_model, test_loader, _criterion, device, _test_atk)
    _results_speech_commands_full_mamba[_atk_name] = round(float(_adv_acc), 4)
    print(f' {_atk_name:20s} adv_acc={_adv_acc:.4f}')

_, _clean_acc_speech_commands_full_mamba = evaluate(_speech_commands_full_mamba_model, test_loader, _criterion, device)
_results_speech_commands_full_mamba['clean'] = round(float(_clean_acc_speech_commands_full_mamba), 4)

_sd_speech_commands_full_mamba = None
try:
    _sd_speech_commands_full_mamba = compute_state_drift(_speech_commands_full_mamba_model, test_loader,
        AttackConfig(attack_type='pgd', adv_eps=8/255), device)
    _results_speech_commands_full_mamba['state_drift'] = round(float(_sd_speech_commands_full_mamba), 6)
    print(f' state_drift {_sd_speech_commands_full_mamba:.6f}')
except Exception as _e:
    print(f' state_drift N/A: {_e}')

_ckpt_speech_commands_full_mamba = torch.load('mamba_speech_commands_full.pt', map_location='cpu')
_ckpt_speech_commands_full_mamba['training_summary']['test_results'] = _results_speech_commands_full_mamba
_ckpt_speech_commands_full_mamba['training_summary']['clean_test_acc'] = _results_speech_commands_full_mamba['clean']
torch.save(_ckpt_speech_commands_full_mamba, 'mamba_speech_commands_full.pt')
print(f' ✓ checkpoint обновлён → mamba_speech_commands_full.pt')
print(_results_speech_commands_full_mamba)
```